Universität Bonn

Mitschrift zur Vorlesung

Randomisierte und Approximative Algorithmen

gehalten von

Prof. Dr. Thomas Kesselheim, Prof. Dr. Heiko Röglin

geTEXed von

Daniel Emse

Winter 2025/26

Anmerkungen und Korrekturen

Falls du Anmerkungen/Korrekturen hast, melde dich bei mir (`daniel.emse@uni-bonn.de`).

Inhaltsverzeichnis

Vorlesung **1**

I Randomisierte und Approximative Algorithmen **2**

1 Einleitung **3**

2 Greedy-Algorithmen **4**
2.1 Vertex Cover 4
2.2 Set Cover 5
2.3 Scheduling auf identischen Maschinen 6
2.4 Rucksackproblem 8

3 Runden und dynamische Programmierung **10**
3.1 Rucksackproblem 10
3.1.1 Dynamische Programmierung 10
3.1.2 Approximationsschema 11
3.2 Scheduling auf identischen Maschinen 12
3.2.1 Bin-Packing mit konstant vielen Objektgrößen 13
3.2.2 Approximationsschema 14
3.2.3 Untere Schranke für Approximierbarkeit 15

4 Randomisierte Approximationsalgorithmen **17**
4.1 Grundlagen der Wahrscheinlichkeitsrechnung 17
4.1.1 Diskrete Wahrscheinlichkeitsräume 17
4.1.2 Unabhängigkeit und bedingte Wahrscheinlichkeiten 18
4.2 Zufallsvariablen und Erwartungswerte 19
4.2.1 Analyse von RandMaxCut 22
4.2.2 Binomialverteilung und geometrische Verteilung 23
4.2.3 Vertex Cover 25
4.3 Max-SAT 26
4.3.1 Derandomisierung 27
4.3.2 Verbesserter Algorithmus 28

5 Lineare Programmierung und Runden **31**
5.1 Max-SAT 31
5.1.1 Randomisiertes Runden 31
5.1.2 Kombination beider Algorithmen 34
5.1.3 Nichtlineares randomisiertes Runden 35
5.2 Facility Location ohne Kapazitäten 36
5.2.1 Reduktion auf Set Cover 37
5.2.2 Metrisches Facility Location 37
5.3 Scheduling auf allgemeinen Maschinen 40

6 Semidefinite Programmierung und Runden **44**
6.1 Semidefinite Programmierung 44

6.2 Maximum-Cut Problem 46
6.3 Max-2SAT 48

8 Primal-Duale Algorithmen **51**
8.1 Duale Lineare Programme 51
8.2 Set Cover 53
8.3 Feedback Vertex Set 54
8.4 Das Kürzeste-Wege-Problem 56
8.5 Steinerwaldproblem 58

II Online-Algorithmen **61**

11 Einleitung **62**

12 Paging **63**
12.1 Deterministische Algorithmen 63
12.1.1 Markierungsalgorithmen 63
12.1.2 Untere Schranken 64
12.2 Randomisierte Algorithmen 65
12.2.1 Potentialfunktionen 65
12.2.2 Analyse von RANDOM 66
12.2.3 Analyse von MARK 67
12.2.4 Untere Schranke für randomisierte Online-Algorithmen 67

13 Das k-Server-Problem **69**
13.1 Untere Schranke für deterministische Algorithmen 69
13.2 Das k-Server-Problem auf Linien und Bäumen 70
13.2.1 Analyse von DC auf der Linie 70
13.2.2 Analyse von DC auf allgemeinen Bäumen 72
13.2.3 Anwendung des DC-Algorithmus 73

14 Approximation von Metriken **74**
14.1 Approximation durch Baummetriken 74
14.1.1 Hierarchische Partitionen und Baummetriken 74
14.1.2 Erzeugung von hierarchischen Partitionen 75

Anhang **A**

Liste der Definitionen **B**

Liste der Aussagen **C**

Liste der Algorithmen **E**

Index **F**

Vorlesung

Teil I

Randomisierte und Approximative Algorithmen

Kapitel 1

Einleitung

Definition 1.1: Optimierungsproblem, Approximationsalgorithmus

Ein *Optimierungsproblem* Π besteht aus

- einer Menge von Instanzen $\mathcal{I}_\Pi$
- für jede Instanz $I \in \mathcal{I}_\Pi$
  - einer Menge $\mathcal{S}_I$ von Lösungen
  - einer Zielfunktion $f_I : \mathcal{S}_I \to \mathbb{R}_{\geq 0}$.

Ein *Approximationsalgorithmus* A nimmt I als Eingabe und berechnet $A(I) \in \mathcal{S}_I$ (in polynomieller Zeit).
Sei $w_A(I) = f_I(A(I))$. Ein Approximationsalgorithmus erreicht Approximationsfaktor r, falls

$$w_A(I) \leq r \cdot \mathrm{OPT}(I) \quad \forall I \in \mathcal{I}_\Pi \qquad \text{(für Minimierungsprobleme)},$$

bzw.

$$w_A(I) \geq \frac{1}{r} \cdot \mathrm{OPT}(I) \quad \forall I \in \mathcal{I}_\Pi \qquad \text{(für Maximierungsprobleme)}.$$

Kapitel 2

Greedy-Algorithmen

2.1 Vertex Cover

[14.10.2025, Vorlesung 1]

Problem 2.1: Vertex Cover

Input: Ungerichteter Graph $G = (V, E)$
Output: Alle $V' \subseteq V$, sodass $\forall e = (u, v) \in E$ gilt $u \in V'$ oder $v \in V'$
Objective: Minimiere $|V'|$

Definition 2.2: Matching

Ein *Matching* $M \subseteq E$ eines Graphen $G = (V, E)$ ist eine Teilmenge der Kanten, sodass es keinen Knoten gibt, der zu mehr als einer Kante aus M inzident ist.
Es heißt *inklusionsmaximal*, wenn für alle Kanten $e \in E \setminus M$ gilt: $M \cup \{e\}$ ist kein Matching.

Algorithmus 2.3 MATCHING-VC

Input: ungerichteter Graph $G = (V, E)$
Output: Vertex Cover $V' \subseteq V$ mit $|V'| \leq 2 \cdot \mathrm{OPT}(G)$

1: ▷ *Berechne inklusionsmaximales Matching $M \subseteq E$:* ◁
2: $M := \emptyset$
3: **for all** $e = \{u, v\} \in E$ **do**
4: **if** weder u noch v ist inzident zu einer Kante in M **then**
5: $M := M \cup \{e\}$
6: $V(M) := \{v \in V \mid v \text{ ist inzident zu einer Kante in } M\}$
7: **return** $V(M)$

Satz 2.4: Matching-VC

Matching-VC hat Laufzeit $\mathcal{O}(|V| + |E|)$ für Graphen in Adjazenzlistendarstellung. Er berechnet ein Vertex Cover der Größe höchstens $2 \cdot \mathrm{OPT}(G)$, wobei $\mathrm{OPT}(G)$ die Größe des kleinsten Vertex Covers ist.

Beweis. **Laufzeit:** In Schritt 1 wird jede Kante genau einmal betrachtet, $\mathcal{O}(|V| + |E|)$.

In Schritt 2 wird jeder Knoten höchstens einmal betrachtet, $\mathcal{O}(|V|)$. Gemeinsam ergibt sich $\mathcal{O}(|V| + |E|)$.

Korrektheit: Angenommen, $e = (u, v) \in E$ wurde nicht von $V(M)$ abgedeckt. Das heißt, es wären weder $u \in V(M)$ noch $v \in V(M)$. Dann wäre $M \cup \{e\}$ ein Matching und e würde in Schritt 1 zu M hinzugefügt werden. Widerspruch zur Inklusionsmaximalität von M. Also ist $V(M)$ ein Vertex Cover.

Approximation: Sei V^* ein minimales Vertex Cover. V^* muss alle Kanten $e \in E$ abdecken, also auch alle $e \in M$.

Außerdem haben in M keine zwei Kanten denselben Endpunkt. Deshalb ist $|M| \leq |V^*|$.

Für unsere Lösung gilt $|V(M)| = 2 \cdot |M| \leq 2 \cdot \mathrm{OPT}(G)$.

□

2.2 Set Cover

Problem 2.5: Set Cover

Input: Menge S mit n Elementen, m Teilmengen $S_1, \ldots, S_m \subseteq S$ mit $\bigcup_{i=1}^{m} S_i = S$, Kosten $c_1, \ldots, c_m \in \mathbb{N}$.
Output: Alle $A \subseteq \{1, \ldots, m\}$ mit $\bigcup_{i \in A} S_i = S$
Objective: Minimiere $\sum_{i \in A} c_i$

Algorithmus 2.6 GREEDY-SC

Input: Grundmenge S mit n Elementen, m Teilmengen $S_1, \ldots, S_m \subseteq S$ mit $\bigcup_{i=1}^{m} S_i = S$, Kosten $c_1, \ldots, c_m \in \mathbb{N}$.
Output: $A \subseteq \{1, \ldots, m\}$ mit $\bigcup_{i \in A} S_i = S$ und $\sum_{i \in A} c_i$ minimal

1: $A := \emptyset, C := \emptyset$
2: **while** $C \neq S$ **do**
3: Wähle S_i mit minimalen relativen Kosten $r_i(C) = \frac{c_i}{|S_i \setminus C|}$
4: Setze $\mathrm{price}(x) := r_i(C)$ für alle $x \in S_i \setminus C$
5: $A := A \cup \{i\}$
6: $C := C \cup S_i$
7: **return** A

Satz 2.7: Greedy-SC

GREEDY-SC ist ein H_n-Approximationsalgorithmus für Set Cover, wobei

$$H_n = \sum_{i=1}^{n} \frac{1}{i}$$

die n-te harmonische Zahl ist.

Untere Schranke für Greedy-SC: Sei $\varepsilon > 0$ sehr klein. Betrachte n Knoten und $m = n + 1$ Mengen, wobei $S_i = \{i\}$ mit Kosten $\frac{1}{n+1-i}$ ist (für $i \in S$) und $S_{n+1} = S$ mit Kosten $1 + \varepsilon$. Dann wählt GREEDY-SC wählt alle Einzelnen, OPT die gesamte Menge.

[14.10.2025, Vorlesung 1]
[15.10.2025, Vorlesung 2]

Beweis. Beobachtung: Es gilt

$$c(A) = \sum_{x \in S} \mathrm{price}(x).$$

Sei $x_1, \ldots, x_n$ die Reihenfolge der Elemente in S, wie sie vom GREEDY-SC abgedeckt werden (Beim Abdecken von mehreren Elementen in gleicher Iteration wähle beliebige Reihenfolge).

Lemma 2.8: Preis in Greedy-SC

Für alle $k \in \{1, \dots, n\}$ gilt

$$\text{price}(x_k) \leq \frac{\text{OPT}}{n-k+1}.$$

Beweis: Lemma 2.8. Betrachte ein beliebiges $k \in \{1, \dots, n\}$. Sei $i \in \{1, \dots, m\}$ der Index der Menge S_i, mit der x_k erstmalig abgedeckt wird. Bezeichne A die Auswahl der Mengen, *bevor* x_k abgedeckt wird.

Sei $C := \bigcup_{i \in A} S_i$. Es gilt $|C| \leq k-1$. Eine optimale Auswahl A^* von Mengen deckt ganz S ab, insbesondere auch $S \setminus C$, wobei $|S \setminus C| \geq n-k+1$.

Es gilt

$$\begin{aligned}\text{OPT} = \sum_{j \in A^*} c_j &\geq \sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} c_j = \sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} |S_j \setminus C| \underbrace{\frac{c_j}{|S_j \setminus C|}}_{= r_j(C) \geq r_i(C)} \\ &\geq r_i(C) \underbrace{\sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} |S_j \setminus C|}_{\geq |S \setminus C|} \geq \underbrace{r_i(C)}_{= \text{price}(x_k)} \cdot (n-k+1)\end{aligned}$$

wobei $r_j(C) \geq r_i(C)$ wegen der Greedy-Auswahl und

$$\sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} |S_j \setminus C| \geq |S \setminus C|,$$

weil A^* ganz $S \setminus C$ abdeckt. □

Nun gilt mit Lemma 2.8

$$c(A) = \sum_{k=1}^{n} \text{price}(x_k) \leq \sum_{k=1}^{n} \frac{\text{OPT}}{n-k+1} = \text{OPT} \sum_{i=1}^{n} \frac{1}{i} = \text{OPT} \cdot H_n$$

und Satz 2.7 ist gezeigt. □

2.3 Scheduling auf identischen Maschinen

Problem 2.9: Scheduling auf identischen Maschinen

Input: Menge $J = \{1, \dots, n\}$ von Jobs, Menge $M = \{1, \dots, m\}$ von Maschinen, Größen $p_j > 0$ für jeden Job $j \in J$

Output: Schedule $\pi : J \to M$, Last von Maschine i in Schedule π: $L_i(\pi) = \sum_{\substack{j \in J \\ \pi(j) = i}} p_j$,

Makespan $C(\pi) = \max_{i \in M} L_i(\pi)$

Objective: Finde Schedule π mit kleinstem Makespan

Algorithmus 2.10 LEASTLOADED

Input: Siehe Problem 2.9

Output: Siehe Problem 2.9

1: Betrachte Jobs in Reihenfolge $1, \dots, n$

2: Weise jedem Job der Maschine mit bislang kleinster Last zu

Satz 2.11: LeastLoaded

LEASTLOADED ist ein $(2-\frac{1}{m})$-Approximationsalgorithmus für Scheduling auf identischen Maschinen.

Beweis. Sei π^* ein optimaler Schedule. Dann ist

$$C(\pi^*) \geq \frac{1}{m}\sum_{j\in J} p_j,$$

denn

$$C(\pi^*) = \max_{i\in M} \sum_{\substack{j\in J\\ \pi(j)=i}} p_j \geq \frac{1}{m}\sum_{i\in M}\sum_{\substack{j\in J\\ \pi(j)=i}} p_j = \frac{1}{m}\sum_{j\in J} p_j.$$

Außerdem gilt $C(\pi^*) \geq \max_{j\in J} p_j$.

Sei π der Schedule, der LEASTLOADED berechnet. Sei i die Maschine mit größter Last. Sei j der Job, der zuletzt zu i hinzugefügt wurde. Zu diesem Zeitpunkt war Maschine i diejenige mit der geringsten Last. Ihre Last war höchstens $\frac{1}{m}\sum_{k=1}^{j-1} p_k$. Es gilt

$$C(\pi) = L_i(\pi) \leq \frac{1}{m}\underbrace{\sum_{k=1}^{j-1} p_k}_{\leq \sum_{k\in J\setminus\{j\}} p_k} + p_j \leq \underbrace{\frac{1}{m}\sum_{k\in J} p_k}_{\leq C(\pi^*)} + \left(1-\frac{1}{m}\right)\underbrace{p_j}_{\leq C(\pi^*)} \leq \left(2-\frac{1}{m}\right) C(\pi^*)$$

und die Abschätzung ist gezeigt. □

Bemerkung 2.12. $2-\frac{1}{m}$ ist eine untere Schranke für LEASTLOADED.

Beweis. Sei $n = m(m-1)+1$ und $p_1 = \cdots = p_{m(m-1)} = 1$ und $p_{m(m-1)+1} = m$.

LEASTLOADED führt auf jeder Maschine $m-1$ Jobs der Größe 1 durch. Auf einer Maschine wird zusätzlich ein Job der Größe m ausgeführt. Somit ist der Makespan $m-1+m = 2m-1$.

Eine optimale Belegung führt auf $m-1$ Maschinen je m Jobs der Größe 1 aus und auf der letzten Maschine einen Job der Größe m, wir haben also einen Makespan von m, und der Faktor stimmt. □

Algorithmus 2.13 LONGESTPROCESSINGTIME (LPT)

Input: Siehe Problem 2.9
Output: Siehe Problem 2.9

1: Sortiere Jobs so, dass $p_1 \geq p_2 \geq \cdots \geq p_n$.
2: Führe LEASTLOADED auf den sortierten Jobs aus.

Satz 2.14: LPT

LONGESTPROCESSINGTIME ist ein $\frac{4}{3}$-Approximationsalgorithmus.

Beweis. Angenommen, es gäbe eine Instanz, sodass LPT einen Makespan von mehr als $\frac{4}{3}\cdot \mathrm{OPT}$ erreicht. Betrachte unter diesen Instanzen eine mit geringster Anzahl Jobs.

Sei i eine Maschine mit höchster Last in LPT-Schedule π. Sei j der letzte Job, der Maschine i zugewiesen wird. Es ist $j = n$, da sonst $p_1, \ldots, p_j$ ein Gegenbeispiel mit weniger Jobs wäre. Es gilt

$$C(\pi) = L_i(\pi) \leq \frac{1}{m}\sum_{k=1}^{n-1} p_k + p_n \leq \mathrm{OPT} + p_n.$$

Aus $C(\pi) > \frac{4}{3}\,\mathrm{OPT}$ folgt $p_n > \frac{1}{3}\,\mathrm{OPT}$. Da $p_1 \geq \cdots \geq p_n$ folgt $p_j > \frac{1}{3}\,\mathrm{OPT}$ für alle $j \in J$. Also erhält im optimalen Schedule jede Maschine höchstens zwei Jobs.

Sei k die Anzahl Jobs mit Größe $\frac{2}{3}\,\mathrm{OPT}$. Im optimalen Schedule sind k Jobs alleine auf einer Maschine und $n-k$ Jobs höchstens zu zweit.

Fall 1: $k = n$. Also ist $n \leq m$, Job n wird also einer leeren Maschine zugewiesen.

Fall 2: $k < n$. Zum Zeitpunkt bevor Job n hinzugefügt wird, hat LPT mindestens eine Maschine mit höchstens einem Job und dieser hat Größe höchstens $\frac{2}{3}\,\mathrm{OPT}$. Wegen $p_n \leq \frac{2}{3}\,\mathrm{OPT}$ ist die Last dieser Maschine im Anschluss höchstens $\frac{4}{3}\,\mathrm{OPT}$.

Beides ist ein Widerspruch zur Annahme. ↯ □

Bemerkung 2.15. $\frac{4}{3}$ ist eine untere Schranke für LONGESTPROCESSINGTIME.

Beweis. Übungsaufgabe. □

Wir werden später sehen, dass für alle $\varepsilon > 0$ ein $(1+\varepsilon)$-Approximationsalgorithmus existiert.

[15.10.2025, Vorlesung 2]
[21.10.2025, Vorlesung 3]

2.4 Rucksackproblem

Problem 2.16: Rucksackproblem (Knapsack Problem, KP)

Input: Kapazität $t \in \mathbb{N}$, Nutzen $p_1, \ldots, p_n \in \mathbb{N}$, Gewichte $w_1, \ldots, w_n \in \{1, \ldots, t\}$
Output: Teilmenge $A \subseteq \{1, \ldots, n\}$ mit $w(A) := \sum_{i \in A} w_i \leq t$
Objective: Maximiere $p(A) := \sum_{i \in A} p_i$

Algorithmus 2.17 GREEDY-KP

Input: Siehe Problem 2.16
Output: Siehe Problem 2.16

```
1: Sortiere Objekte nach Effizienz, sodass p_1/w_1 ≥ ··· ≥ p_n/w_n
2: B := ∅, i := 1
3: while i ≤ n and w(B) + w_i ≤ t do
4:     B := B ∪ {i}
5:     i := i + 1
6: Sei i* ein Objekt mit dem größten Nutzen
7: if p(B) < p_{i*} then
8:     A := {i*}
9: else A := B
10: return A
```

Satz 2.18: Greedy-KP

GREEDY-KP ist ein $\frac{1}{2}$-Approximationsalgorithmus.

Beweis. Sei $B := \{1, \ldots, i\}$ die im Algorithmus berechnete Menge. Falls $i = n$, gibt es nichts zu zeigen.

Falls $i < n$, gilt $w(B) + w_{i+1} > t$.

Behauptung: Es gilt $p(B) + p_{i+1} \geq \text{OPT}$:

Angenommen, $\text{OPT} > p(B) + p_{i+1}$. Dann gibt es eine Menge $A' \subseteq \{1, \ldots, n\}$, für die gilt, dass $w(A') \leq t < w(B) + w_{i+1}$ und $p(A') = \text{OPT} > p(B) + p_{i+1}$.

Betrachte $J := \{1, \ldots, i+1\} \setminus A'$ und $J' := A' \setminus \{1, \ldots, i+1\}$. Nun gilt

$$p(B) + p_{i+1} < p(A') = (p(B) + p_{i+1}) + p(J') - p(J) \implies p(J') > p(J)$$

und analog

$$w(B) + w_{i+1} > w(A') = (w(B) + w_{i+1}) + w(J') - w(J) \implies w(J') < w(J).$$

Sei $r = \frac{p_{i+1}}{w_{i+1}}$, dann

$$p(J) \geq r \cdot w(J), \qquad p(J') \leq r \cdot w(J').$$

Zusammen ergibt sich

$$p(J) \geq r \cdot w(J) > r \cdot w(J') \geq p(J') > p(J). \qquad \lightning$$

Damit ist die Behauptung gezeigt und es gilt $p(B) + p_{i+1} \geq \text{OPT}$.

Somit gilt

$$p(A) = \max\{p(B), p_{i^*}\} \geq \frac{1}{2}(p(B) + p_{i^*}) \geq \frac{1}{2}(p(B) + p_{i+1}) \geq \frac{1}{2}\,\text{OPT}$$

und die Aussage ist gezeigt. □

Kapitel 3

Runden und dynamische Programmierung

Definition 3.1: Approximationsschema, PTAS, FPATS

Ein *Approximationsschema* A für ein Optimierungsproblem Π ist ein Algorithmus, der zu jeder Eingabe (I, ε) mit $I \in \mathcal{I}_\Pi, \varepsilon > 0$ eine Lösung $A(I,\varepsilon) \in \mathcal{S}_I$ berechnet und es gilt

$$f(A(I,\varepsilon)) \leq (1+\varepsilon)\,\mathrm{OPT}(I) \qquad \text{bei Minimierungsproblemen, bzw.}$$
$$f(A(I,\varepsilon)) \geq (1-\varepsilon)\,\mathrm{OPT}(I) \qquad \text{bei Maximierungsproblemen.}$$

A heißt *polynomielles Approximationsschema* (PTAS), wenn die Laufzeit für jedes ε polynomiell in $|I|$ beschränkt ist.
A heißt *voll-polynomielles Approximationsschema* (FPTAS), wenn die Laufzeit polynomiell in $\frac{1}{\varepsilon}$ und $|I|$ ist.

Beispiel 3.2.

- $\Theta(|I|^{1/\varepsilon})$ ist PTAS, aber kein FPTAS
- $\Theta\left(\frac{|I|^{100}}{\varepsilon^{10}}\right)$ ist ein FPTAS

3.1 Rucksackproblem

3.1.1 Dynamische Programmierung

Sei $W(i,p)$ *geringste* Gewicht unter allen Teilmengen von $\{1,\dots,i\}$, die Nutzen mindestens p erreichen, bzw. ∞, wenn das nicht möglich ist.

Falls $I \subseteq \{1,\dots,i\}$ gilt: Falls $\sum_{i\in I} p_i \geq p$, dann ist $\sum_{i\in I} w_i \geq W(i,p)$.

Es gilt

$$W(0,p) = \begin{cases} 0 & p \leq 0 \\ \infty & p > 0, \end{cases}$$

$$W(1,p) = \begin{cases} 0 & p \leq 0 \\ w_1 & 0 < p \leq p_1 \\ \infty & p > p_1. \end{cases}$$

Sei $P := \max_{\{1,\dots,n\}} p_i$. Uns interessieren $W(i,p)$ für $i = 0,\dots,n$ und $p = 0,\dots,nP$.

Rekursionsformel für $W(i,p)$:

Sei $I \subseteq \{1,\dots,i\}$ diejenige Teilmenge mit minimalem Gewicht unter allen mit $\sum_{j\in I} p_j \geq p$.

Fall 1: $i \notin I$. Dann gilt $W(i,p) = W(i-1,p)$.

Fall 2: $i \in I$. Dann betrachte $I \setminus \{i\}$. Es ist $W(I \setminus \{i\}) = W(i,p) - w(i)$ und $p(I \setminus \{i\}) \geq p - p_i$. Daraus folgt

$$W(i-1, p-p_i) \leq W(i,p) - w_i.$$

Umgekehrt sei $I' \subseteq \{1,\dots,i-1\}$ diejenige Menge, die $W(i-1,p-p_i)$ liefert. Nun gilt

$$W(I' \cup \{i\}) = W(i-1,p-p_i) + w_i, \qquad p(I' \cup \{i\}) = p(I') + p_i \geq p.$$

Daraus folgt, dass $W(i,p) \leq W(i-1,p-p_i) + w_i$.

Umstellen liefert

$$W(i,p) = W(i-1,p-p_i) + w_i.$$

Insgesamt ergibt sich die Rekursion

$$W(i,p) = \min\{W(i-1,p), W(i-1,p-p_i) + w_i\}, \qquad W(0,p) = \begin{cases} \infty & p > 0 \\ 0 & p \leq 0. \end{cases}$$

Algorithmus 3.3 DYNKP

Input: Siehe Problem 2.16
Output: Siehe Problem 2.16

1: $P := \max_{i\in\{1,\dots,n\}} p_i$
2: **for** $i = 1$ **to** n **do**
3: **for** $p = 1$ **to** nP **do**
4: $W(i,p) = \min\{W(i-1,p), W(i-1,p-p_i) + w_i\}$
5: **return** maximales $p \in \{1,\dots,nP\}$ mit $W(n,p) \leq t$.

Satz 3.4: DynKP

DYNKP liefert in Zeit $\Theta(n^2P)$ den maximal erreichbaren Nutzen.

3.1.2 Approximationsschema

Algorithmus 3.5 FPTAS-KP

Input: Siehe Problem 2.16, $\varepsilon > 0$
Output: Siehe Problem 2.16

1: $P := \max_{i\in\{1,\dots,n\}} p_i$
2: $K := \frac{\varepsilon P}{n}$
3: **for** $i = 1$ **to** n **do**
4: $p'_i = \lfloor \frac{p_i}{K} \rfloor$
5: Benutze DYNKP auf $p'_1,\dots,p'_n, w_1,\dots,w_n, t$

Satz 3.6: FPTAS-KP

FPTAS-KP hat eine Laufzeit von $\mathcal{O}(\frac{n^3}{\varepsilon})$ und berechnet eine $1-\varepsilon$-Approximation.

Beweis.

Laufzeit: DYNKP braucht Zeit $\Theta(n^2P')$, wobei $P' = \max_{i\in\{1,\dots,n\}} p'_i$. Es gilt für alle i:

$$p'_i = \left\lfloor \frac{p_i}{K} \right\rfloor \leq \left\lfloor \frac{P}{K} \right\rfloor = \left\lfloor \frac{n}{\varepsilon} \right\rfloor \leq \frac{n}{\varepsilon},$$

also ist auch $P' \leq \frac{n}{\varepsilon}$.

Approximationsfaktor: Sei I' eine optimale Lösung auf $p'_1, \ldots, p'_n$ und I eine optimale Lösung auf $p_1, \ldots, p_n$.

I' ist auch eine optimale Lösung auf $\{p_i^*\} := \{K \cdot p'_i\}$.

Beispiel 3.7. Für $n = 4, P = 50, \varepsilon = \frac{4}{5}, K = \frac{\varepsilon P}{n} = 10$. Für $p_1 = 33$ erhalten wir $p'_1 = 3$ bzw. $p_1^* = 30$.

Es gilt $p_i^* = K \cdot \lfloor \frac{p_i}{K} \rfloor \geq K \cdot (\frac{p_i}{K} - 1) = p_i - K$ und $p_i^* \leq K \cdot \frac{p_i}{K} = p_i$, also $p_i^* \in [p_i - K, p_i]$.

Für alle $J \subseteq \{1, \ldots, n\}$ gilt

$$p^*(J) := \sum_{i \in J} p_i^* \leq \sum_{i \in J} p_i = p(J)$$

und

$$p^*(J) = \sum_{i \in J} p_i^* \geq \sum_{i \in J} (p_i - K) = \sum_{i \in J} p_i - |J| \cdot K \geq p(J) - nK.$$

Damit ist $p(I') \geq p^*(I') \geq p^*(I) \geq p(I) - nK$, da I' optimal hinsichtlich p^* ist.

Weiterhin ist $p(I) \geq P$, also gilt

$$\frac{p(I')}{\mathrm{OPT}} \geq \frac{p(I) - nK}{p(I)} \geq 1 - \frac{nK}{P} = 1 - \varepsilon$$

und wir erhalten $1 - \varepsilon$ als Approximationsfaktor.

□

[21.10.2025, Vorlesung 3]
[22.10.2025, Vorlesung 4]

3.2 Scheduling auf identischen Maschinen

Erster Ansatz für Approximationsschema: Wir nennen einen Job $j \in J$ *klein*, falls

$$p_j \leq \varepsilon \cdot \frac{1}{m} \sum_{k=1}^{n} p_k,$$

sonst heißt er *groß*. Wir betrachten folgenden Algorithmus:

Algorithmus 3.8 GrossKleinScheduling

Input: Siehe Problem 2.9, $\varepsilon > 0$
Output: Siehe Problem 2.9

1: Berechne optimalen Schedule auf großen Jobs ▷ *Bspw. durch Ausprobieren aller Schedules*
2: Ergänze kleine Jobs mit LeastLoaded

Satz 3.9: GroßKleinScheduling

GrossKleinScheduling ist ein PTAS mit Laufzeit $m^{\frac{m}{\varepsilon}}$.

Beweis. **Laufzeit:** Für Algorithmus 3.8 gibt es höchstens $\frac{m}{\varepsilon}$ große Jobs, denn jeder ist mindestens $\varepsilon \frac{1}{m} \sum_{k=1}^{n} p_k$ groß. Damit gibt es maximal $m^{\frac{m}{\varepsilon}}$ mögliche Schedules. Damit hat Algorithmus 3.8 polynomielle Laufzeit für jedes $\varepsilon > 0$, wenn m konstant ist.

Approximationsfaktor: Betrachte die Maschine mit der größten Last.

Fall 1: Diese Maschine erhält nur große Jobs in unserem Schedule. Dann ist der Makespan durch die kleinen Jobs unverändert. Unser Schedule ist optimal auf den großen Jobs, und somit auch optimal auf allen Jobs.

Fall 2: Diese Maschine erhält auch einen kleinen Job. Im Moment der Zuweisung des letzten solchen Jobs durch LEASTLOADED ist die Last höchstens die $\frac{1}{m}\sum_{k=1}^{n} p_k$. Der letzte Job ist ein kleiner Job, somit haben wir am Ende eine Gesamtlast von höchstens

$$(1+\varepsilon)\frac{1}{m}\sum_{k=1}^{n} p_k \leq (1+\varepsilon)\,\mathrm{OPT}\,.$$

□

3.2.1 Bin-Packing mit konstant vielen Objektgrößen

Problem 3.10: Bin-Packing

Input: Objektgrößen $s_1, \ldots, s_n \in \mathbb{N}$, Kapazität $T \in \mathbb{N}$
Output: Partitionen $C_1, \ldots, C_\ell$ der Menge $\{1, \ldots, n\}$ mit $\sum_{j \in C_i} s_j \leq T$ für alle i
Objective: Minimiere ℓ

Spezialfall: Wir wählen als Eingabe $(s_1, \ldots, s_z) \in \mathbb{N}^z, (n_1, \ldots, n_z) \in \mathbb{N}^z, T \in \mathbb{N}$. Dabei haben wir jeweils n_i Objekte der Größe s_i.

Wir bezeichnen mit $\mathrm{BINS}(i_1, \ldots, i_z)$ den optimalen Zielfunktionswert, wenn es $i_j \leq n_j$ Objekte der Größe s_j gibt. Uns interessiert $\mathrm{BINS}(n_1, \ldots, n_z)$. Berechne aller Werte $\mathrm{BINS}(i_1, \ldots, i_z)$ für $(i_1, \ldots, i_z) \in \mathbb{N}_0^z$ mit $i_j \leq n_j$. Wenn z konstant ist, gibt es höchstens $(n+1)^z \in \mathcal{O}(n^z)$ viele solche Vektoren.

Definiere

$$C := \left\{ (i_1, \ldots, i_z) \in \mathbb{N}_0^z \;\middle|\; \sum_{j=1}^{z} i_j s_j \leq T \right\} \setminus \{(0, \ldots, 0)\}.$$

Wir bemerken:

- $\mathrm{BINS}(0, \ldots, 0) = 0$
- $\mathrm{BINS}(i_1, \ldots, i_z) = 1$ für $(i_1, \ldots, i_z) \in C$

Für die allgemeine Größe von $\mathrm{BINS}(i_1, \ldots, i_z)$ betrachte ein optimales Packing. Der erste Bin erhalte $(c_1, \ldots, c_z) \in C$ viele Objekte. So erhalten die übrigen Bins insgesamt $(i_1 - c_1, \ldots, i_z - c_z)$ Objekte und es gilt

$$\mathrm{BINS}(i_1, \ldots, i_z) = 1 + \mathrm{BINS}(i_1 - c_1, \ldots, i_z - c_z).$$

Als allgemeine Rekursionsformel ergibt sich

$$\mathrm{BINS}(i_1, \ldots, i_z) = 1 + \min_{(c_1, \ldots, c_z) \in C} \mathrm{BINS}(i_1 - c_1, \ldots, i_z - c_z) \qquad \text{für alle } (i_1, \ldots, i_z) \neq (0, \ldots, 0)$$

und $\mathrm{BINS}(0, \ldots, 0) = 0$.

Wir berechnen Werte von $\mathrm{BINS}(i_1, \ldots, i_z)$ aufsteigend nach $\sum_{j=1}^{z} i_j$. Insgesamt werden $\mathcal{O}(n^z)$ viele Vektoren mit jeweils Laufzeit $\mathcal{O}(n^z)$ pro Vektor, damit ergibt sich eine Gesamtlaufzeit von $\mathcal{O}(n^{2z})$, also folgender Satz:

Satz 3.11: Bin-Packing

Das Bin-Packing-Problem mit z verschiedenen Objektgrößen kann für Instanzen mit n Objekten in Zeit $\mathcal{O}(n^{2z})$ gelöst werden (wobei die Konstante in der $\mathcal{O}$-Notation von z abhängt.)

3.2.2 Approximationsschema

Zusätzliche Eingabe: T. Der Algorithmus liefert entweder einen Schedule mit Makespan höchstens $(1+\varepsilon)T$ oder er beweist, dass $\mathrm{OPT} > T$.

Wir bezeichnen einen Job j als *groß*, falls $p_j > \varepsilon T$. Wir runden die Größen aller großen Jobs auf Vielfache von $\varepsilon^2 T$ ab, somit gibt es höchstens $\frac{1}{\varepsilon^2}$ viele Jobgrößen. Auf diesen bestimmen wir eine optimale Bin-Packing-Lösung. Wir erhalten als Antwort entweder eine Packung der großen Jobs auf m Maschinen mit Makespan $\leq T$ (bzgl. der gerundeten Jobgrößen), oder dass es nicht möglich ist (dann brechen wir ab). Für die kleinen Jobs verwenden wir anschließend LeastLoaded.

Für den Makespan gilt insgesamt:

- Erhält eine Maschine einen kleinen Job, betrachte den letzten derartigen Job. Dann gilt

$$L_i(\pi) \leq \underbrace{\left(\frac{1}{m}\sum_{k=1}^{n} p_k\right)}_{\leq T} + p_j \leq (1+\varepsilon)T.$$

- Erhält eine Maschine keinen kleinen Job, dann ist die Last durch die gerundeten Jobgrößen höchstens T. Sei p'_j die abgerundete Jobgröße und p_j die tatsächliche Jobgröße. Dann gilt

$$p_j \geq \varepsilon T, \qquad p'_j \geq p_j - \varepsilon^2 T \geq p_j - \varepsilon p'_j \implies p_j \leq (1+\varepsilon)p'_j,$$

  unter der Voraussetzung, dass $\frac{1}{\varepsilon} \in \mathbb{N}$.

Somit sind nicht gerundete Jobgrößen höchstens um den Faktor $(1+\varepsilon)$ größer und somit ist die Last der Maschine $\leq (1+\varepsilon)T$ bezüglich der tatsächlichen Jobgrößen.

Hier der Algorithmus nochmal in Pseudocode:

Algorithmus 3.12 $B_\varepsilon(T)$

1: Sei $J' = \{j \in J \mid p_j > \varepsilon T\}$ die Menge der großen Jobs
2: Für $j \in J$ sei p'_j der Wert p_j auf das nächste Vielfache von $\varepsilon^2 T$ abgerundet. Sei I' die Instanz, die nur aus Jobs mit den Größen p'_j mit $j \in J'$ besteht.
3: Konstruiere eine Instanz für Bin-Packing mit Objektgrößen $\{p'_j \mid j \in J'\}$ und Kapazität T. Bezeichne a die minimale Anzahl an Bins, die für diese Instanz benötigt werden. Berechne a und die dazugehörige Verteilung π' der Objekte mit Hilfe des dynamischen Programms aus Satz 3.11.
4: **if** $a > m$ **then**
5: | **return** $\mathrm{OPT}(I) \geq \mathrm{OPT}(I') > T$
6: **else**
7: | Erweitere π' auf alle Jobs aus J durch Hinzufügen der Jobs aus $J \setminus J'$ mittels LeastLoaded.
8: | **return** resultierenden Schedule π

Somit haben wir gezeigt:

Lemma 3.13: $B_\varepsilon(T)$

Es gibt einen Algorithmus, der in Zeit $\mathcal{O}(m^{\frac{2}{\varepsilon^2}})$ entweder beweist, dass es keinen Schedule mit Makespan T gibt oder einen Schedule mit Makespan $(1+\varepsilon)T$ berechnet.

Für einen Optimalen Makespan gilt nun

$$\begin{aligned}\mathrm{OPT} &\geq \left\lceil \frac{1}{m}\sum_{k=1}^{n} p_k \right\rceil \\ \mathrm{OPT} &\geq p_{\max} \\ \mathrm{OPT} &\leq \left\lceil \frac{1}{m}\sum_{k=1}^{n} p_k \right\rceil + p_{\max}.\end{aligned}$$

Setze initial $L := \max\left\{\left\lceil \frac{1}{m}\sum_{k=1}^{n} p_k \right\rceil, p_{\max}\right\}$ und $U := \left\lceil \frac{1}{m}\sum_{k=1}^{n} p_k \right\rceil + p_{\max}$. Dabei haben wir folgende Invarianten:

1. $L \leq \mathrm{OPT}$
2. Wir können einen Schedule mit Makespan höchstens $(1+\varepsilon)U$ in Zeit $\mathcal{O}(m^{\frac{2}{\varepsilon^2}})$ berechnen.

Nun berechne $T := \left\lfloor \frac{L+U}{2} \right\rfloor$ und prüfe, ob ein Schedule mit Makespan T berechnet werden kann. Falls ja, setze $U := T$, sonst setze $L := T+1$. Es gilt $U - L \leq \frac{p_{\max}}{2^i}$ nach i Iterationen, somit gilt $U - L \leq \frac{1}{2}$ nach $\lfloor \log_2(p_{\max}) \rfloor + 1$ Iterationen. Dann muss $L = U$ gelten und wir erhalten als Resultat folgenden Satz:

Satz 3.14: PTAS Scheduling

Für Scheduling auf identischen Maschinen existiert ein PTAS mit einer Laufzeit von $\mathcal{O}(m^{\frac{2}{\varepsilon^2}} \log(p_{\max}))$.

[22.10.2025, Vorlesung 4]
[28.10.2025, Vorlesung 5]

3.2.3 Untere Schranke für Approximierbarkeit

Definition 3.15: Stark NP-schwer

Sei Π ein Optimierungsproblem, in dessen Instanzen ganze Zahlen enthalten sind. Π heißt *stark NP-schwer*, wenn es ein Polynom q gibt, sodass Π eingeschränkt auf Instanzen I, in denen alle Zahlen durch $q(|I|)$ beschränkt sind, bereits NP-schwer ist.

Scheduling auf identischen Maschinen ist tatsächlich stark NP-schwer.

Satz 3.16: Scheduling hat keinen FPTAS

Unter der Annahme $\mathrm{P} \neq \mathrm{NP}$ existiert kein FPTAS für Scheduling auf identischen Maschinen.

Beweis. Angenommen, es gäbe einen FPTAS. Weil Scheduling auf identischen Maschinen stark NP-schwer ist, gibt es ein Polynom q, sodass es bereits NP-schwer ist, das Problem mit Jobgrößen $p_1 \leq \cdots \leq p_n \in \{1, \ldots, q(n)\}$ zu lösen.

Berechne mit dem FPTAS eine $(1+\varepsilon)$-Approximation, wobei $\varepsilon = \frac{1}{2nq(n)}$. Die Laufzeit ist polynomiell in n und $\frac{1}{\varepsilon} = 2nq(n)$, also beides polynomiell in n. Für den Approximationsfaktor

erhalten wir $w_A(I, \varepsilon) \leq (1 + \varepsilon)\,\mathrm{OPT}$. Es gilt

$$\mathrm{OPT}(I) \leq \sum_{i=1}^{n} p_i \leq n \cdot q(n)$$

und folglich

$$(1 + \varepsilon)\,\mathrm{OPT} = \mathrm{OPT} + \underbrace{\varepsilon\,\mathrm{OPT}}_{\leq \frac{1}{2}} \leq \mathrm{OPT}(I) + \varepsilon n q(n) = \mathrm{OPT} + \frac{1}{2}.$$

Weil alle Jobgrößen ganzzahlig sind, sind auch $w_A(I, \varepsilon)$ und $\mathrm{OPT}(I)$ ganze Zahlen. Da die Differenz höchstens $\frac{1}{2}$ ist, ist $w_A(I, \varepsilon) = \mathrm{OPT}(I)$. Also löst der FPTAS das Problem exakt in polynomieller Zeit. Also muss P = NP sein. $\lightning$ zur Annahme □

Kapitel 4

Randomisierte Approximationsalgorithmen

Problem 4.1: Maximum-Cut Problem

Input: Ungerichteter Graph $G = (V, E)$, Kantengewichte $w : E \to \mathbb{R}_{>0}$
Output: Menge $U \subseteq V$. Wir nennen $(U, V \setminus U)$ *Schnitt* des Graphen G
Objective: Maximiere $\sum_{x \in U} \sum_{y \in V \setminus U} w(x, y)$ mit $w(x, y) = 0$, falls $\{x, y\} \notin E$.

Algorithmus 4.2 RandMaxCut

Input: Wie in Problem 4.1
Output: Wie in Problem 4.1

```
1: U := ∅
2: for v ∈ V do
3:     Wirf eine Münze
4:     if Münze zeigt Kopf then
5:         Füge v zu U hinzu
6: return U
```

4.1 Grundlagen der Wahrscheinlichkeitsrechnung

4.1.1 Diskrete Wahrscheinlichkeitsräume

Definition 4.3: Wahrscheinlichkeitsraum

Sei Ω eine endliche oder abzählbare Menge (*Ergebnismenge*).
Elemente von Ω heißen *Elementarereignisse*, Teilmengen von Ω heißen *Ereignisse*.
Für $A \subseteq \Omega$ bezeichne $\overline{A} := \Omega \setminus A$ das *Gegenereignis*.
Ein(-e) *Wahrscheinlichkeit(-smaß)* ist eine Funktion $\mathbb{P} : 2^\Omega \to [0, 1]$ mit $\mathbb{P}[\Omega] = 1$ und $\mathbb{P}$ *σ-additiv*, d.h. für $A_1, A_2, \ldots \subseteq \Omega$ mit $A_i \cap A_j = \emptyset$ für $i \neq j$ gilt

$$\mathbb{P}\left[\bigcup_{i=1}^{\infty} A_i\right] = \sum_{i=1}^{\infty} \mathbb{P}[A_i].$$

Das Paar $(\Omega, \mathbb{P})$ heißt (diskreter) *Wahrscheinlichkeitsraum*.

Es reicht, $\mathbb{P}[a]$ für alle $a \in \Omega$ zu definieren, denn

$$\mathbb{P}[\{a_1, \ldots, a_n\}] = \sum_{i=1}^{n} \mathbb{P}[\{a_i\}].$$

Beispiel 4.4. Bei einem Würfelwurf setze $\Omega = \{1, \ldots, 6\}$. Dann ist $\mathbb{P}[i] = \frac{1}{6}$ für alle $i \in \Omega$. Es ist auch $\mathbb{P}[\text{„gerade Zahl“}] = \mathbb{P}[\{2, 4, 6\}] = 3 \cdot \frac{1}{6} = \frac{1}{2}$.

Wir modellieren nun einen Wahrscheinlichkeitsraum für RandMaxCut:

Sei $V = \{v_1, \dots, v_n\}$. Setze $\Omega = \{0,1\}^n$. Wir berechnen den Schnitt als

$$U = \{v_i \in V \mid \omega_i = 1\}.$$

Für alle $e = \{v_i, v_j\} \in E$, ist die Wahrscheinlichkeit, dass e über den Schnitt läuft, gerade $\frac{1}{2}$, denn es ist

$$\mathbb{P}[e \text{ läuft über den Schnitt } (U, V \setminus U)] = \mathbb{P}[\{\omega \in \Omega \mid \omega_i \neq \omega_j\}] = \frac{2^{n-1}}{2^n} = \frac{1}{2}.$$

4.1.2 Unabhängigkeit und bedingte Wahrscheinlichkeiten

Definition 4.5: Unabhängige Ereignisse

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum. Zwei Ereignisse $A, B \subseteq \Omega$ heißen *unabhängig*, falls

$$\mathbb{P}[A \cap B] = \mathbb{P}[A] \cdot \mathbb{P}[B].$$

Allgemeiner: Ereignisse $A_1, \dots, A_k \subseteq \Omega$ heißen *unabhängig*, wenn für alle $I \subseteq \{1, \dots, k\}$ gilt

$$\mathbb{P}\left[\bigcap_{i \in I} A_i\right] = \prod_{i \in I} \mathbb{P}[A_i].$$

Beispiel 4.6.

- Sei $\Omega = \{(i,j) \mid i,j \in \{1,\dots,6\}\}$. Dann ist $\mathbb{P}[(i,j)] = \frac{1}{36}$.

  Seien $A :=$ „Erster Würfel zeigt eine gerade Zahl“ und $B :=$ „Zweiter Würfel zeigt ≥ 3“. Dann ist $\mathbb{P}[A] = \frac{1}{2}, \mathbb{P}[B] = \frac{2}{3}$ und $\mathbb{P}[A \cap B] = \frac{1}{3} = \mathbb{P}[A] \cdot \mathbb{P}[B]$. Also sind A und B unabhängig.
- Für eine endlihce Folge von Wahrscheinlichkeitsräumen $(\Omega_1, \mathbb{P}_1), \dots, (\Omega_n, \mathbb{P}_n)$ definieren wir den *Produktraum* durch $\Omega := \Omega_1 \times \cdots \times \Omega_n$ und $\mathbb{P}[(x_1, \dots, x_n)] := \mathbb{P}_1[x_1] \cdot \ldots \cdot \mathbb{P}_n[x_n]$.

In RandMaxCut ist der Wahrscheinlichkeitsraum eigentlich ein Produktraum von n Münzwürfen $(\{0,1\}, \mathbb{P})$ mit $\mathbb{P}[0] = \mathbb{P}[1] = \frac{1}{2}$. Seien $e, e' \in E$ ohne gemeinsame Endpunkte. Dann ist

$$\mathbb{P}[\text{„}e \text{ und } e' \text{ laufen über Schnitt“}] = \frac{1}{4} = \mathbb{P}[\text{„}e \text{ läuft über Schnitt“}] \cdot \mathbb{P}[\text{„}e' \text{ läuft über Schnitt“}].$$

Definition 4.7: Bedingte Wahrscheinlichkeit

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum. Seien $A, B \subseteq \Omega$ mit $\mathbb{P}[B] > 0$. Dann bezeichne

$$\mathbb{P}[A \mid B] := \frac{\mathbb{P}[A \cap B]}{\mathbb{P}[B]}$$

die *bedingte Wahrscheinlichkeit* von A gegeben B.

Beispiel 4.8. Wir betrachten zwei unanbhängige Würfelwürfe. Sei $A :=$ „Erster Würfel zeigt 6“ und $B :=$ „Summe beider Würfe ist 10“. Dann sind

$$\mathbb{P}[A] = \frac{1}{6}, \quad \mathbb{P}[B] = \frac{3}{36}, \quad \mathbb{P}[A \cap B] = \frac{1}{36} \quad \text{und} \quad \mathbb{P}[A \mid B] = \frac{1}{3}.$$

4.2 Zufallsvariablen und Erwartungswerte

Definition 4.9: Zufallsvariable

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum. eine Abbildung $X : \Omega \to \mathbb{R}$ heißt (diskrete, reelle) *Zufallsvariable*. Wir schreiben

$$\mathbb{P}[X = a] = \mathbb{P}[\{\omega \in \Omega \mid X(\omega) = a\}]$$

und allgemeiner

$$\mathbb{P}[X \in A] = \mathbb{P}[\{\omega \in \Omega \mid X(\omega) \in A\}].$$

Beispiel 4.10. Betrachte zwei unabhängige Würfelwürfe. Sei $X : \Omega := \{1, \dots, 12\}$ definiert durch $X(\omega) := \omega_1 + \omega_2$ für alle $\omega = (\omega_1, \omega_2) \in \Omega$.

Definition 4.11: Unabhängigkeit (Zufallsvariablen)

Seien $X, Y : \Omega \to \mathbb{R}$ Zufallsvariablen. X und Y heißen *unabhängig*, wenn für alle $x, y \in \mathbb{R}$ gilt

$$\mathbb{P}[(X = x) \cap (Y = y)] = \mathbb{P}[X = x] \cdot \mathbb{P}[Y = y].$$

Allgemeiner seien $X_1, \dots, X_n : \Omega \to \mathbb{R}$ Zufallsvariablen. Diese heißen *unabhängig*, wenn für alle $I \subseteq \{1, \dots, n\}, x_i \in \mathbb{R}$ gilt

$$\mathbb{P}\left[\bigcap_{i \in I} (X_i = x)\right] = \prod_{i \in I} \mathbb{P}[X_i = x].$$

Definition 4.12: Erwartungswert

Sei $X : \Omega \to \mathbb{R}$ eine diskrete Zufallsvariable. Falls die Reihe

$$\sum_{\omega \in \Omega} |X(\omega)| \cdot \mathbb{P}[\omega]$$

konvergiert, heißt

$$\mathbb{E}[X] := \sum_{\omega \in \Omega} X(\omega) \cdot \mathbb{P}[\omega]$$

Erwartungswert von X.

[28.10.2025, Vorlesung 5]
[29.10.2025, Vorlesung 6]

Sei X eine Zufallsvariable, für die der Erwartungswert gemäß Definition 4.12 existiert. Bezeichnen wir mit $R = X(\Omega)$ die abzählbare Teilmenge der reelen Zahlen, die von X angenommen werden können, können wir den Erwartungswert auch schreiben als

$$\mathbb{E}[X] = \sum_{x \in R} x \cdot \mathbb{P}[X = x].$$

Beispiel 4.13.

- Für die Zufallsvariable X, die die Summe zweier unabhängiger Würfelwürfe modelliert, ist

$$\mathbb{E}[X] = \sum_{x=2}^{12} x \cdot \mathbb{P}[X = x] = \frac{1}{36}(2+2\cdot 3+3\cdot 4+4\cdot 5+5\cdot 6+6\cdot 7+5\cdot 8+4\cdot 9+3\cdot 10+2\cdot 11+12) = 7.$$

- Für die Zufallsvariable X, die beschreibt, wie oft in zehn Münzwürfen „Zahl“ geworfen wird, ist es anhand der Definition schon schwieriger, den Erwartungswert zu berechnen. Mit Mühe erhält man

$$\mathbb{E}[X] = 0 \cdot \frac{1}{1024} + 1 \cdot \frac{10}{1024} + 2 \cdot \frac{45}{1024} + \cdots + 10 \cdot \frac{1}{1024} = 5.$$

- Wieder betrachten wir den Algorithmus RANDMAXCUT (Algorithmus 4.2) auf dem obigen Beispiel, in dem die drei Kanten e_1, e_2 und e_3 ein Dreieck bilden. Sei wieder X die Zufallsvariable, die die Anzahl an Kanten beschreibt, die über den Schnitt laufen. Da X nur die Werte 0 oder 2 annehmen kann, gilt

$$\mathbb{E}[X] = 0 \cdot \mathbb{P}[X = 0] + 2 \cdot \mathbb{P}[X = 2] = 2 \cdot \frac{3}{4} = \frac{3}{2}.$$

Im obigen Beispiel für RANDMAXCUT haben wir die erwartete Anzahl an Kanten, die über den Schnitt laufen, ausgerechnet. Für Graphen mit vielen Knoten und Kanten wäre eine ähnliche Rechnung sehr aufwändig, da wir in diesem Beispiel alle möglichen Realisierungen einzeln betrachtet haben. Um das Ausrechnen von Erwartungswerten zu vereinfachen, lernen wir einige wichtige Rechenregeln kennen.

Satz 4.14: Linearität des Erwartungswerts

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum und seien $X, Y : \Omega \to \mathbb{R}$ Zufallsvariablen mit existierenden Erwartungswerten. Dann gilt:

(a) *Für $c, d \in \mathbb{R}$ existiert der Erwartungswert von $(cX + d) : \Omega \to \mathbb{R}$ mit $(cX + d)(\omega) := c \cdot X(\omega) + d$ für alle $\omega \in \Omega$. Es gilt $\mathbb{E}[cX + d] = c \cdot \mathbb{E}[X] + d$.*

(b) *Der Erwartungswert der Zufallsvariablen $(X + Y) : \Omega \to \mathbb{R}$ mit $(X + Y)(\omega) := X(\omega) + Y(\omega)$ für alle $\omega \in \Omega$ existiert und es gilt $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$.*

(c) *Sind X und Y unabhängig, so existiert der Erwartungswert der Zufallsvariablen $(XY) : \Omega \to \mathbb{R}$ mit $(XY)(\omega) := X(\omega) \cdot Y(\omega)$ für alle $\omega \in \Omega$ und es gilt $\mathbb{E}[XY] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.*

Beweis.

(a): Aus Definition 4.12 folgt

$$\begin{aligned}\mathbb{E}[cX + d] &= \sum_{\omega \in \Omega} (cX(\omega) + d) \cdot \mathbb{P}[\{\omega\}] \\ &= c \cdot \sum_{\omega \in \Omega} X(\omega) \cdot \mathbb{P}[\{\omega\}] + d \cdot \sum_{\omega \in \Omega} \mathbb{P}[\{\omega\}] \\ &= c \cdot \mathbb{E}[X] + d.\end{aligned}$$

Die Existenz des Erwartungswerts von $cX + d$ folgt aus der Existenz des Erwartungswerts von X.

(b): Es gilt, auch mit Definition 4.12:

$$\begin{aligned}\mathbb{E}[X + Y] &= \sum_{\omega \in \Omega} (X(\omega) + Y(\omega)) \cdot \mathbb{P}[\{\omega\}] \\ &= \sum_{\omega \in \Omega} X(\omega) \cdot \mathbb{P}[\{\omega\}] + \sum_{\omega \in \Omega} Y(\omega) \cdot \mathbb{P}[\{\omega\}] \\ &= \mathbb{E}[X] + \mathbb{E}[Y].\end{aligned}$$

(c): Seien $R = X(\Omega)$ und $S = Y(\Omega)$. Dann gilt die abzählbaren Mengen von Werten, die X

und Y annehmen können. Es folgt mit Definition 4.12:

$$\begin{aligned}\mathbb{E}[XY] &= \sum_{\omega\in\Omega} X(\omega)\cdot Y(\omega)\cdot \mathbb{P}[\{\omega\}] \\ &= \sum_{x\in R}\sum_{y\in S} x\cdot y\cdot \mathbb{P}[(X=x)\cap(Y=y)] \\ &= \sum_{x\in R}\sum_{y\in S} x\cdot y\cdot \mathbb{P}[X=x]\cdot\mathbb{P}[Y=y] \\ &= \left(\sum_{x\in R} x\cdot\mathbb{P}[X=x]\right)\cdot\left(\sum_{y\in S} y\cdot\mathbb{P}[Y=y]\right) \\ &= \mathbb{E}[X]\cdot\mathbb{E}[Y],\end{aligned}$$

wobei wir in der dritten Gleichheit die Unabhängigkeit von X und Y verwendet haben. Die Existenz des Erwartungswerts von XY folgt aus der Existenz der Erwartungswerte von X und Y.

□

Satz 4.15: Markov

Sei $(\Omega,\mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum und sei $X:\Omega\to\mathbb{R}_{\geq 0}$ eine Zufallsvariable mit existierendem Erwartungswert. Dann gilt für alle $a>0$:

$$\mathbb{P}[X\geq a]\leq\frac{\mathbb{E}[X]}{a}.$$

Wir können die Markov-Ungleichung umformulieren, um eine noch einprägsamere Form zu erhalten. Sei $X:\Omega\to\mathbb{R}_{\geq 0}$ eine Zufallsvariable. Dann gilt für alle $a\geq 1$

$$\mathbb{P}[X\geq a\cdot\mathbb{E}[X]]\leq\frac{1}{a}.$$

Beweis von Satz 4.15. Es gilt

$$\mathbb{E}[X]=\sum_{\omega\in\Omega} X(\omega)\cdot\mathbb{P}[\{\omega\}]\geq\sum_{\substack{\omega\in\Omega\\ X(\omega)\geq a}} X(\omega)\cdot\mathbb{P}[\{\omega\}]\geq\sum_{\substack{\omega\in\Omega\\ X(\omega)\geq a}} a\cdot\mathbb{P}[\{\omega\}]=a\cdot\mathbb{P}[X\geq a].$$

Daraus folgt die Behauptung. □

Mithilfe der Markov-Ungleichung können wir beispielsweise die Wahrscheinlichkeit abschätzen, dass wir in einer Folge von n unabhängigen, fairen Münzwürfen mindestens $(3n/4)$-mal „Zahl“ werfen. Sei X die Zufallsvariable, die die Anzahl an „Zahl“ in den n Münzwürfen beschreibt. Dann ist $\mathbb{E}[X]=\frac{n}{2}$. Mit der Markov-Ungleichung folgt für $a=\frac{3}{2}$:

$$\mathbb{P}\left[X\geq\frac{3n}{4}\right]=\mathbb{P}\left[X\geq a\cdot\mathbb{E}[X]\right]\leq\frac{1}{a}=\frac{2}{3}.$$

Tatsächlich fällt die obige Wahrscheinlichkeit sogar exponentiell schnell mit n.

4.2.1 Analyse von RandMaxCut

Definition 4.16: Randomisierter Approximationsalgorithmus

Ein *randomisierter Approximationsalgorithmus* A für ein Minimierungs- oder Maximierungsproblem Π erreicht einen *Approximationsfaktor* von $r \geq 1$ bzw. $r \leq 1$, wenn

$$\mathbb{E}[w_A(I)] \leq r \cdot \mathrm{OPT}(I) \quad \text{bzw.} \quad \mathbb{E}[w_A(I)] \geq r \cdot \mathrm{OPT}(I)$$

für alle Instanzen $I \in \mathcal{I}_\Pi$ gilt. Wir sagen dann, dass A ein *randomisierter r-Approximationsalgorithmus* ist.
Oft hängt der Approximationsfaktor von der Eingabelänge ab. Dann ist $r : \mathbb{N} \to [0, \infty)$ eine Funktion und es muss

$$\mathbb{E}[w_A(I)] \leq r(|I|) \cdot \mathrm{OPT}(I) \quad \text{bzw.} \quad \mathbb{E}[w_A(I)] \geq r(|I|) \cdot \mathrm{OPT}(I)$$

für alle Instanzen $I \in \mathcal{I}_\Pi$ gelten, wobei $|I|$ die Länge der Eingabe I bezeichne.

Satz 4.17: RandMaxCut

RandMaxCut (Algorithmus 4.2) ist ein randomisierter $\frac{1}{2}$-Approximationsalgorithmus für das Maximum-Cut-Problem.

Beweis. Sei $G = (V, E)$ ein ungereichteter Graph mit Kantengewichten w. Sei $U \subseteq V$ die zufällige Teilmenge von V, die RandMaxCut auswählt. Sei X die Zufallsvariable, die das Gesamtgewicht der Kanten, die über den Schnitt laufen, angibt. Außerdem definieren wir für jede Kante $e \in E$ die Zufallsvariable X_e durch

$$X_e := \begin{cases} w(e), & \text{falls } e \text{ über den Schnitt läuft,} \\ 0, & \text{sonst.} \end{cases}$$

Dann gilt per Definition $X = \sum_{e \in E} X_e$. Mit der Linearität des Erwartungswerts (Satz 4.14 (b)) folgt

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{e \in E} X_e\right] = \sum_{e \in E} \mathbb{E}[X_e].$$

Da X_e nur die Werte 0 und $w(e)$ annehmen kann, gilt

$$\begin{aligned} \mathbb{E}[X_e] &= 0 \cdot \mathbb{P}[X_e = 0] + w(e) \cdot \mathbb{P}[X_e = w(e)] \\ &= w(e) \cdot \mathbb{P}[e \text{ läuft über den Schnitt}] \\ &= w(e) \cdot \frac{1}{2}. \end{aligned}$$

Für den letzten Schritt haben wir verwendet, dass die Wahrscheinlichkeit, dass eine Kante über den Schnitt läuft, gerade $\frac{1}{2}$ ist. Insgesamt erhalten wir somit

$$\mathbb{E}[X] = \sum_{e \in E} \mathbb{E}[X_e] = \sum_{e \in E} w(e) \cdot \frac{1}{2} = \frac{1}{2} \cdot \sum_{e \in E} w(e) = \frac{W}{2}$$

für $W = \sum_{e \in E} w(e)$. Außerdem gilt $\mathrm{OPT} \leq W$, da kein Schnitt mehr als das Gesamtgewicht aller Kanten erreichen kann. Somit folgt wie gewünscht

$$\mathbb{E}[X] = \frac{W}{2} \geq \frac{\mathrm{OPT}}{2}.$$

□

4.2.2 Binomialverteilung und geometrische Verteilung

Seien $Y_1, \dots, Y_n$ unabhängige Bernoulli-Zufallsvariablen mit demselben Parameter $p \in [0,1]$. Die Zufallsvariable $X = Y_1 + \cdots + Y_n$ heißt *binomialverteilt* mit Parametern n und p.

Satz 4.18: Binomialverteilung

Sei X eine binomialverteilte Zufallsvariable mit Parametern n und p. Dann gilt $\mathbb{E}[X] = pn$ und für alle $j \in \{0, \dots, n\}$:

$$\mathbb{P}[X = j] = \binom{n}{j} p^j (1-p)^{n-j}.$$

Beweis. Ausrechnen. □

Sei nun $Y_1, Y_2, \dots$ eine Folge unabhängiger Bernoulli-Zufallsvariablen mit demselben Parameter $p \in [0,1]$. Wir betrachten nun die Frage, wie lange wir auf den ersten Erfolg warten müssen. Diese wird durch eine *geometrisch verteilte Zufallsvariable mit Parameter p* modelliert. Eine solche Zufallsvariable X nimmt den Wert $n \in \mathbb{N}$ an, wenn $Y_1 = \cdots = Y_{n-1} = 0$ und $Y_n = 1$. Die Wahrscheinlichkeit hierfür ist $(1-p)^{n-1}p$. Wir definieren für eine geometrische Zufallsvariable X mit Parameter p dementsprechend $\mathbb{P}[X = n] = (1-p)^{n-1}p$ für alle $n \in \mathbb{N}$. Um den Erwatungswert einer geometrischen Zufallsvariablen zu bestimmen, leiten wir zunächst eine nützliche alternative Darstellung des Erwartungswerts her, die nur für Zufallsvariablen gilt, die nur Werte aus $\mathbb{N}_0$ annehmen können.

Lemma 4.19: Erwartungswert natürliche Zufallsvariable

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum und sei $X : \Omega \to \mathbb{N}_0$ eine Zufallsvariable. Dann gilt

$$\mathbb{E}[X] = \sum_{i=1}^{\infty} \mathbb{P}[X \geq i].$$

Beweis. Es gilt

$$\begin{aligned}
\sum_{i=1}^{\infty} \mathbb{P}[X \geq i] &= \sum_{i=1}^{\infty} \sum_{j=1}^{\infty} \mathbb{P}[X = j] \\
&= \sum_{j=1}^{\infty} \sum_{i=1}^{j} \mathbb{P}[X = j] \\
&= \sum_{j=1}^{\infty} j \cdot \mathbb{P}[X = j] = \mathbb{E}[X],
\end{aligned}$$

wobei wir ausgenutzt haben, dass wir bei absolut konvergenten Reihen die Reihenfolge der Summanden beliebig vertauschen können. □

Satz 4.20: Geometrische Verteilung

Sei X eine geometrisch verteilte Zufallsvariable mit Parameter p. Dann gilt $\mathbb{E}[X] = \frac{1}{p}$.

Beweis. Für die geometrische Zufallsvariable X und $j \in \mathbb{N}$ gilt

$$\mathbb{P}[X \geq j] = \sum_{i=j}^{\infty} \mathbb{P}[X = i] = \sum_{i=j}^{\infty} (1-p)^{i-1} p = (1-p)^{j-1},$$

wobei die letzte Gleichung durch Nachrechnen folgt, oder durch Überlegen, dass $X \geq j$ genau dann gilt, wenn die ersten $(j-1)$ Bernoulli-Versuche Fehlschläge sind. Mit Lemma 4.19 folgt somit

$$\mathbb{E}[X] = \sum_{i=1}^{\infty} \mathbb{P}[X \geq i] = \sum_{i=1}^{\infty} (1-p)^{i-1} = \frac{1}{1-(1-p)} = \frac{1}{p},$$

wobei wir in der letzten Gleichheit die Formel für die geometrische Reihe verwendet haben. □

Eine weitere wichtige Eigenschaft geometrisch verteilter Zufallsvariablen ist die *Gedächtnislosigkeit*. Das bedeutet, dass die Wahrscheinlichkeit eines Erfolgs unanbhängig davon ist, wie viele Misserfolge es bis jetzt gegeben hat.

Lemma 4.21: Gedächtnislosigkeit geometrischer Verteilung

Sei X eine geometrisch verteilte Zufallsvariable mit Parameter p. Dann gilt für alle $n \in \mathbb{N}$ und $k \in \mathbb{N}_0$:

$$\mathbb{P}[X = n + k \mid X > k] = \mathbb{P}[X = n].$$

Beweis. Es gilt

$$\begin{aligned}\mathbb{P}[X = n+k \mid X > k] &= \frac{\mathbb{P}[(X = n+k) \cap (X > k)]}{\mathbb{P}[X > k]} = \frac{\mathbb{P}[X = n+k]}{\mathbb{P}[X > k]} \\ &= \frac{(1-p)^{n+k-1} \cdot p}{(1-p)^k} = (1-p)^{n-1} p = \mathbb{P}[X = n].\end{aligned}$$ □

Das *Coupon Collector's Problem* tritt oft in der Analyse randomisierter Algorithmen auf und lässt sich anhant von Sammelbildern beschreiben, wie sie zum Beispiel zu jeder Fußballweltmeisterschaft verkauft werden. Wir gehen davon aus, dass es insgesamt n verschiedene solcher Bilder gibt. Diese kann man nicht gezielt erwerben, sondern man kann nur verschlossene Tüten kaufen, in denen sich jeweils unabhängig ein uniform zufällig ausgewähltes Sammelbild befindet. Wir bezeichnen mit X die Zufallsvariable, die angibt, wie viele Tüten wir kaufen müssen, bis wir jedes Bild mindestens einmal in unserer Sammlung haben.

Satz 4.22: Coupon Collector's Problem

Für die Zufallsvariable X aus dem Coupon Collector's Problem gilt $\mathbb{E}[X] = n\ln(n) + \Theta(n)$.

Beweis. Wir zerlegen den Prozess in n Phasen. Die erste Phase beginnt mit dem ersten Sammelbild, das wir kaufen. Phase i für $i \in \{2, \dots, n\}$ beginnt direkt, nachdem wir das $(i-1)$-te verschiedene Sammelbild gekauft haben, und endet, sobald wir das i-te verschiedene Sammelbild gekauft haben. Bezeichnen wir mit X_i die Anzahl an Sammelbildern, die wir in Phase i kaufen, dann gilt $X = X_1 + X_2 + \cdots + X_n$. Außerdem ist jedes X_i geometrisch verteilt mit Parameter

$$p_i = \frac{n-i+1}{n},$$

denn die Wahrscheinlichkeit, eines der $n - i + 1$ Sammelbilder zu kaufen, die nicht zu den $i-1$ bereits erworbenen Bildern gehören, beträgt in Phase i in jedem Versuch $\frac{n-i+1}{n}$. Mit der Linearität des Erwartungswertes und Satz 4.20 folgt somit

$$\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i] = \sum_{i=1}^{n} \frac{1}{p_i} = \sum_{i=1}^{n} \frac{n}{n-i+1} = n \cdot \sum_{i=1}^{n} \frac{1}{i} = n \cdot (\ln(n) + \Theta(1)) = n\ln(n) + \Theta(n).$$

Im letzten Schritt haben wir dabei ausgenutzt, dass für die n-te harmonische Zahl H_n gerade $\sum_{i=1}^{n} \frac{1}{i} = \ln(n) + \Theta(1)$ gilt. □

4.2.3 Vertex Cover

Wir betrachten nun einen simplen Algorithmus RandVertexCover für das Vertex-Cover-Problem. Für eine Eingabe $G = (V, E)$ startet dieser Algorithmus mit einer leeren Menge U und geht alle Kanten aus E in einer beliebigen Reihenfolge durch. Für jede Kante $e \in E$ überprüft der Algorithmus, ob bereits mindestens ein Endknoten von e in U enthalten ist. Falls ja, so macht der Algorithmus mit der nächsten Kante weiter. Ansonsten wählt er uniform zufällig einen der beiden Endknoten von e aus und fügt diesen zu U hinzu. Anschließend macht er mit der nächsten Kante weiter.

Wir betrachten sein Verhalten auf einem Sterngraphen $G = (V, E)$. Für einen solchen Graphen ist $V = \{v, u_1, \dots, u_{n-1}\}$ und die Menge E besteht nur aus den Kanten (v, u_i) für alle i. Das optimale Vertex Cover besteht nur aus dem Knoten v, der alle Kanten abdeckt. Bei jeder Kante fügt RandVertexCover mit einer Wahrscheinlichkeit von $\frac{1}{2}$ den Knoten v zur Menge U hinzu. Sobald dies einmal passiert ist, sind alle Kanten des Graphen abgedeckt und es werden keine weiteren Knoten der Menge U hinzugefügt. Sei nun X eine geometrisch verteilte Zufallsvariable mit Parameter $p = \frac{1}{2}$. Dann ist die Anzahl an Knoten, die RandVertexCover zu U hinzufügt, verteilt wie die Zufallsvariable $\min\{X, n-1\}$, da der Algorithmus maximal $|E| = n - 1$ viele Knoten zu U hinzufügen kann. Es gilt

$$\mathbb{E}[\min\{X, n-1\}] \leq \mathbb{E}[X] = \frac{1}{p} = 2 = 2 \cdot \mathrm{OPT}$$

und somit handelt es sich bei RandVertexCover bei diesem Beispiel um einen randomisierten 2-Approximationsalgorithmus.

Satz 4.23: RandVertexCover

RandVertexCover ist ein randomisierter 2-Approximationsalgorithmus für das Vertex-Cover-Problem.

Beweis. Aus der Beschreibung des Algorithmus folgt direkt, dass er stets ein Vertex Cover berechnet. Zu zeigen ist also nur, dass dieses im Erwartungswert höchstens doppelt so viele Knoten enthält wie ein minimales Vertex Cover. Sei $G = (V, E)$ eine beliebige Eingabe und sei $Z = \{z_1, \dots, z_k\} \subseteq V$ ein minimales Vertex Cover von G. Wir bezeichnen mit $V_i \subseteq V$ die Menge, die aus z_i und den zu z_i benachbarten Knoten besteht. Die Mengen V_i sind im Allgemeinen nicht paarweise disjunkt. Ferner bezeichne $E_i \subseteq E$ die Menge der zu z_i inzidenten Kanten. Da es sich bei Z um ein Vertex Cover handelt, gilt $E = E_1 \cup \dots \cup E_k$.

Es bezeichne X_i die Anzahl an Knoten aus V_i, die RandVertexCover bei Betrachtung der Kanten aus E_i zu U hinzufügt. Es seien $e_1, \dots, e_\ell$ die Kanten aus E_i und wir nehmen an, dass RandMaxCut diese in diese Reihenfolge betrachtet. Zwischendurch betrachtet RandMaxCut im Allgemeinen auch andere Kanten aus $E \setminus E_i$. Wenn der Algorithmus den Knoten z_i bei der Betrachtung einer Kante $e_j \in E_i$ der Menge U hinzufügt, so wird bei allen weiteren Kanten aus E_i kein weiterer Knoten zu U hinzugefügt. Die Situation ist ähnlich zu obigem Sterngraphen, allerdings sind die X_i nicht identisch verteilt wie $\min\{X, |E_i|\}$ für eine geometrisch verteilte Zufallsvariable X mit Parameter $p = \frac{1}{2}$, da zwischen den Betrachtungen der Kanten aus E_i auch Kanten aus $E \setminus E_i$ betrachtet werden können, die den Zustand des Algorithmus verändern. Allerdings gilt dennoch

$$\mathbb{E}[X_i] \leq \mathbb{E}[\min\{X, |E_i|\}] \leq \mathbb{E}[X] = 2.$$

Wegen $E_1 \cup \dots \cup E_k = E$ gilt $|U| = X_1 + \dots + X_k$ und somit folgt mit der Linearität des Erwartungswerts (Satz 4.14 (*b*)):

$$\mathbb{E}[|U|] = \mathbb{E}[X_1 + \dots + X_k] = \sum_{i=1}^{k} \mathbb{E}[X_i] \leq \sum_{i=1}^{k} 2 = 2k = 2 \cdot \mathrm{OPT}\,. \qquad \square$$

4.3 Max-SAT

Die Eingabe besteht aus einer aussagenlogischen Formel in konjunktiver Normalform (KNF). Das Ziel ist es, eine Belegung der Variablen zu finden, die möglichst viele Klauseln erfüllt. In der gewichteten Variante besitzt jede Klausel ein Gewicht und das Ziel ist es, eine Belegung zu finden, die das Gesamtgewicht der erfüllten Klauseln maximiert. Aus der NP-Schwere des Entscheidungsproblems SAT folgt direkt, dass auch das Max-SAT-Problem NP-schwer ist.

Wir nennen eine Formel der Form x oder $\neg x$ für eine Variable x ein *Literal*. Ein Literal der Form x nennen wir ein *positives Literal*, ein Literal der Form $\neg x$ ein *negatives Literal*. Wir nennen die Variablen, die in der Eingabe vorkommen, im Folgenden $x_1, \dots, x_n$. Eine *Klausel* C ist eine *Disjunktion* (Oder-Verknüpfung) von Literalen über diesen Variablen und eine aussagenlogische Formel φ ist in *konjunktiver Normalform* (KNF) , wenn sie eine *Konjunktion* (Und-Verknüpfung) von Klauseln ist. Oft fassen wir Klauseln auch einfach als Menge von Literalen auf. Eine Formel in KNF ist dann von der Gestalt

$$\varphi = \bigwedge_{i=1}^{m} \left(\bigvee_{\ell \in C_l} \ell \right),$$

wobei $C_1, \dots, C_m$ die Klauseln von φ sind.

Problem 4.24: Max-SAT

Input: aussagenlogische Formel φ in KNF mit Variablen $x_1, \dots, x_n$ und Klauseln $C_1, \dots, C_m$; Gewichte $w_1, \dots, w_m \in (0, \infty)$
Output: Belegung der Variablen $x_1, \dots, x_m$
Objective: Maximiere das Gesamtgewicht der durch die Belegung erfüllten Klauseln

Genau wie schon für die Maximum-Cut- und Vertex Cover-Probleme geben wir für Max-SAT zunächst einen sehr einfachen Approximationsalgorithmus RandMaxSAT an. Dieser wählt einfach eine uniform zufällige Belegung α der Variablen $x_1, \dots, x_n$. Das bedeutet, er setzt jede Variable unabhängig von den anderen mit einer Wahrscheinlichkeit von $\frac{1}{2}$ auf 0 bzw. 1.

Satz 4.25: RandMaxSAT

RandMaxSAT ist ein randomisierter $\frac{1}{2}$-Approximationsalgorithmus für Max-SAT.

Beweis. Wir definieren zunächst für jede Klausel C_j eine Zufallsvariable

$$Y_j := \begin{cases} w_j & \text{falls } \alpha \text{ Klausel } C_j \text{ erfüllt,} \\ 0 & \text{sonst.} \end{cases}$$

Dann können wir das Gesamtgewicht Y der durch α erfüllten Klauseln als die Summe $Y_1 + \cdots + Y_m$ schreiben und es gilt

$$\mathbb{E}[Y] = \sum_{j=1}^{m} w_j \cdot \mathbb{P}[\alpha \text{ erfüllt } C_j].$$

Die Klausen C_j wird nur dann nicht durch die zufällige Belegung α erfüllt, wenn alle Variablen, die als positives Literal in C_j vorkommen, auf 0 gesetzt sind und alle Variablen, die als negatives Literal in C_j vorkommen, auf 1 gesetzt sind. Es bezeichne $\ell_j = |C_j|$ die Anzahl an Literalen in C_j. Da die oben genannte Belegung der Variablen aus C_j die einzige ist, die die Klausel nicht

erfüllt, gilt

$$\mathbb{P}[\alpha \text{ erfüllt } C_j \text{ nicht}] \leq \left(\frac{1}{2}\right)^{\ell_j} \leq \frac{1}{2}.$$

Der erste Schritt ist dabei eine Ungleichung und keine Gleichung, da die Klausel C_j für eine Variable x sowohl das Literal x als auch das Literal $\neg x$ enthalten kann. Da eine solche Klausel durch jede beliebige Belegung erfüllt wird, beträgt die Wahrscheinlichkeit, dass C_j durch α nicht erfüllt wird, in diesem Fall 0. damit folgt insgesamt

$$\mathbb{E}[Y] = \sum_{j=1}^{m} w_j \cdot \mathbb{P}[\alpha \text{ erfüllt } C_j] \geq \sum_{j=1}^{m} w_j \cdot \frac{1}{2} = \frac{1}{2} \sum_{j=1}^{m} w_j \geq \frac{1}{2} \cdot \text{OPT},$$

wobei die letzte Schranke daraus folgt, dass das Gesamtgewicht aller Klauseln eine obere Schranke für OPT ist. □

Damit folgt, dass der Approximationsfaktor von RandMaxSAT besser wird, je länger die Klauseln sind. Eine Klausel mit ℓ Literalen ist nur mit Wahrscheinlichkeit von höchstens $(1/2)^\ell$ nicht durch die zufällige Belegung erfüllt. Betrachten wir beispielsweise das 3-SAT-Problem, bei dem alle Klauseln genau drei Literale enthalten, so ergibt sich ein Approximationsfaktor von $\frac{7}{8}$. Es ist bekannt, dass dies sogar im Wesentlichen der beste erreichbare Approximationsfaktor für 3-SAT unter der Annahme P≠NP ist.

RandMaxCut und seine Analyse sind vollkommen analog zu dem einfachen Approximationsalgorithmus für das Maximum-Cut Problem. Wir zeigen, wie der Algorithmus *derandomisiert* werden kann. Das bedeutet, wir möchtendie eigentlich randomisierten Entscheidungnen von RandMaxSAT (d.h. die zufällige Belegung α) nun deterministisch so treffen, dass der Algorithmus den gleichen Approximationsfaktor erreicht. Danach werden wir sehen, wie wir mit einem besseren randomisierten Ansatz einen besseren Approximationsfaktor erreichen können.

4.3.1 Derandomisierung

Definition 4.26: Bedingte Erwartung

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum und $A \subseteq \Omega$ ein Ereignis mit $\mathbb{P}[A] > 0$. Außerdem sei $X : \Omega \to \mathbb{R}$ eine diskrete Zufallsvariable mit Werten aus $M \subseteq \mathbb{R}$. Der *bedingte Erwartungswert von X gegeben A* existiert, falls die Reihe $\sum_{x \in M} |x| \cdot \mathbb{P}[X = x \mid A]$ konvergiert und ist dann definiert als

$$\mathbb{E}[X \mid A] = \sum_{x \in M} x \cdot \mathbb{P}[X = x \mid A].$$

Es gelten folgende Rechenregeln:

$$\mathbb{E}[X] = \mathbb{P}[A] \cdot \mathbb{E}[X \mid A] + \mathbb{P}[\overline{A}] \cdot \mathbb{E}[X \mid \overline{A}] \tag{4.1}$$

und (für B unabhängig von A)

$$\mathbb{E}[X \mid A] = \mathbb{P}[B] \cdot \mathbb{E}[X \mid A \wedge B] + \mathbb{P}[\overline{B}] \cdot \mathbb{E}[X \mid A \wedge \overline{B}]. \tag{4.2}$$

In der Derandomisierung von RandMaxSAT wählen wir die Belegung von $x_1, \ldots, x_n$ Schritt für Schritt. In Schritt i bestimmen wir die Belegung von x_i und haben davor schon die Belegung von $x_1, \ldots, x_{i-1}$ fexstgelegt. Wir betrachten zunächst den ersten Schritt, in dem wir die Belegung von x_1 festlegen. Mit Y bezeichnen wir wieder die Zufallsvariable, die das Gesamtgewicht der

erfüllten Klauseln bei einer uniform zufälligen Belegung α beschreibt. Gemäß (4.1) gilt

$$\mathbb{E}[Y] = \mathbb{P}[\alpha_1 = 0] \cdot \mathbb{P}[Y \mid \alpha_1 = 0] + \mathbb{P}[\alpha_1 = 1] \cdot \mathbb{E}[Y \mid \alpha_1 = 1] = \frac{\mathbb{E}[Y \mid \alpha_1 = 0] + \mathbb{E}[Y \mid \alpha_1 = 1]}{2}.$$

Aus dieser Gleichung folgt direkt, dass einer der beiden bedingten Erwartungswerte mindestens den Wert $\mathbb{E}[Y]$ annehmen muss. Sei $b_1 \in \{0,1\}$ mit $\mathbb{E}[Y \mid \alpha_1 = b_1] \geq \mathbb{E}[Y]$. Wir setzen x_1 nun auf b_1. Wir fahren nach dem Schema fort und betrachten Schritt i, in dem wir die Belegung von x_i festlegen und gehen davon aus, dass wir bereits Belegungen $b_1, \dots, b_{i-1}$ der Variablen $x_1, \dots, x_{i-1}$ gewählt haben, sodass gilt

$$\mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}] \geq \mathbb{E}[Y].$$

Mit (4.2) gilt dann

$$\begin{aligned}
\mathbb{E}[Y] &\leq \mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}] \\
&= \mathbb{P}[\alpha_i = 0] \cdot \mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}, \alpha_i = 0] \\
&\quad + \mathbb{P}[\alpha_i = 1] \cdot \mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}, \alpha_i = 1] \\
&= \frac{\mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}, \alpha_i = 0] + \mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}, \alpha_i = 0]}{2}.
\end{aligned}$$

Mit demselben Argument muss einer der beiden bedingten Erwartungswerte mindestens den Wert $\mathbb{E}[Y]$ haben. Wir setzen x_i nun auf das b_i, für das das der Fall ist. Auf diese Weise können wir induktiv fortfahren und deterministisch eine Belegung $b_1, \dots, b_n$ der Variablen $x_1, \dots, x_n$ finden, sodass das Gesamtgewicht der erfüllten Klauseln mindestens $\mathbb{E}[Y]$ beträgt.

Wir müssen nun noch zeigen, dass dieses Verfahren effizient ausgeführt werden kann. Mit Linearität des Erwartungsertes folgt (was auch für bedingte Erwartungen gilt)

$$\mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_i = b_i] = \sum_{j=1}^{m} w_j \cdot \mathbb{P}[\alpha \text{ erfüllt } C_j \mid \alpha_1 = b_1, \dots, \alpha_i = b_i].$$

Diese bedingten Wahrscheinlichkeiten können wie folgt bestimmt werden: Erfüllt eine der Variablen $x_1, \dots, x_i$ mit der Belegung $b_1, \dots, b_i$ bereits Klausel C_j, so beträgt die bedingte Wahrscheinlichkeit 1. Ansonsten sind alle Literale in C_j, die aus $x_1, \dots, x_i$ bestehen, in der Belegung $b_1, \dots, b_i$ falsch. Diese können aus der Klausel getrichen werden. Bleiben danach noch ℓ Literale übrig, so beträgt die bedingte Wahrscheinlichkeit, dass die Klausel durch die zufällige Belegung erfüllt wird, $1 - (1/2)^\ell \geq 1/2$. (Wir nehmen an, dass nicht gleichzeitig x und $\neg x$ in der Klausel vorkommen, da diese sonst mit Wahrscheinlichkeit 1 erfüllt wäre.)

Wir erhalten insgesamt einen deterministischen $\frac{1}{2}$-Approximationsalgorithmus für Max-SAT. Die Methode der bedingten Erwartungswerte zur Derandomisierung lässt sich auch auf viele andere randomisierte Algorithmen erweitern. Insbesondere können wir mit dieser Methode auch RandMaxCut derandomisieren.

4.3.2 Verbesserter Algorithmus

Wir werden nun sehen, dass der Approximationsfaktor $\frac{1}{2}$ durch einen einfachen Trick verbessert werden kann. Wir setzen nun jede Variable mit Wahrscheinlichkeit $p > \frac{1}{2}$ unabhängig von den anderen Variablen auf 1. Für eine geeignete Wahl von p führt das zu einem besseren Approximationsfaktor. Zunächst betrachten wir nur Formeln, die keine Klauseln enthalten, die nur aus einem einzigen negativen Literal bestehen. Solche Klauseln nennen wir im Folgenden *negative Einheitsklauseln*.

Lemma 4.27: Zufällige Belegung mit Wahrscheinlichkeit p

Sei φ eine Formel in KNF über den Variablen $x_1, \dots, x_n$ mit den Klauseln $C_1, \dots, C_m$. Die Formel φ enthalte keine negativen Einheitsklauseln. Eine zufällige Belegung α der Variablen $x_1, \dots, x_n$, die jede Variable unabhängig mit Wahrscheinlichkeit $p \geq \frac{1}{2}$ auf 1 setzt, erfüllt jede einzelne Klausel mit einer Wahrscheinlichkeit von mindestens $\min\{p, 1-p^2\}$.

Beweis. Sei C eine beliebige Klausel. Falls es sich bei C um eine Klausel der Länge 1 handelt, so besteht sie laut Voraussetzung aus einem positiven Literal. Das bedeutet, die C wird von der zufälligen Belegung mit einer Wahrscheinlichkeit von p erfüllt. Betrachten wir nun den Fall, dass es sich bei C um eine Klausel der Länge $\ell \geq 2$ handelt. Wir bezeichnen mit a und b die Anzahl an positiven bzw. negativen Literalen in C, also ist $\ell = a + b$. Enthält C für eine Variable sowohl das entsprechend positive als auch das negative Literal, ist C mit Wahrscheinlichkeit 1 erfüllt. Ansonsten gibt es genau eine Belegung der in C vorkommenden Variablen, die C nicht erfüllt: Alle positiven Literale sind auf 0 gesetzt und alle negativen Literale auf 1. Die Wahrscheinlichkeit, dass diese Belegung gewählt wird, beträgt $p^b(1-p)^a$. Folglich beträgt die Wahrscheinlichkeit, dass C erfüllt ist,

$$1 - p^b(1-p)^a \geq 1 - p^{a+b} = 1 - p^\ell \geq 1 - p^2,$$

wobei wir $1 - p \leq p$ genutzt haben. □

Satz 4.28: Approximationsalgorithmus durch zufällige Belegung mit Wahrscheinlichkeit p

Für Max-SAT ohne negative Einheitsklauseln erhalten wir einen $\min\{p, 1-p^2\}$-Approximationsalgorithmus, indem wir jede Variable mit Wahrscheinlichkeit $p \geq \frac{1}{2}$ unabhängig auf 1 setzen.

Beweis. Sei φ eine Formel ohne negative Einheitsklauseln und Y das Gesamtgewicht der Klauseln durch die zufällige Belegung α. Dann gilt

$$\begin{aligned}\mathbb{E}[Y] &= \sum_{j=1}^{m} w_j \cdot \mathbb{P}[\alpha \text{ erfüllt } C_j] \geq \sum_{j=1}^{m} w_j \cdot \min\{p, 1-p^2\} \\ &= \min\{p, 1-p^2\} \cdot \sum_{j=1}^{m} w_j \geq \min\{p, 1-p^2\} \cdot \mathrm{OPT}\,.\end{aligned}$$

□

$\min p, 1-p^2$ ist maximal, wenn $p = 1-p^2$. In $[0,1]$ ist das für $p = \frac{\sqrt{5}-1}{2} \approx 0.618$ der Fall.

Nun zeigen wir noch, wie wir die Annahme eliminieren können, dass es keine negativen Einheitsklauseln gibt: Sei φ eine beliebige KNF-Formel. Wir erhalten eine zu φ äquivalente Formel, indem wir für eine Variable x jedes Vorkommen von x durch $\neg x$ und jedes $\neg x$ durch x ersetzen. Das bedeutet, dass oBdA das Gewicht der Einheitsklausel x mindestens so groß ist wie das Gewicht der Einheitsklausel $\neg x$ (mit der impliziten Konvention, dass Klauseln, die nicht in φ enthalten sind, Gewicht 0 haben). Insbesondere bedeutet das, dass nicht der Fall eintreten kann, dass φ die Einheitsklausel $\neg x$ mit positiven Gewicht, aber nicht die Einheitsklausel x enthält. Für eine Variable x_i bzeichnen wir im Folgenden mit v_i das Gewicht der negativen Einheitsklausel $\neg x$.

Lemma 4.29: OPT ist höchstens alle Gewichte minus negative Einheitsklauseln

Es gilt $\mathrm{OPT} \leq \sum_{j=1}^{m} w_j - \sum_{i=1}^{n} v_i$.

Beweis. Jede Belegung erfüllt für jede Variable x_j entweder die Einheitsklausel x_j oder die Einheitsklausel $\neg x_j$. Im besten Fall erfüllt eine Belegung alle Klauseln der Länge mindestens zwei und für jede Variable x_j die Einheitsklausel mit größerem Gewicht. Aus dieser Überlegung folgt direkt das Lemma. □

Satz 4.30: randomisierter $\frac{\sqrt{5}-1}{2}$-Approximationsalgorithmus

Für MAX-SAT erhalten wir einen $\min\{p, 1-p^2\}$-Approximationsalgorithmus, indem wir jede Variable mit Wahrscheinlichkeit $p \geq \frac{1}{2}$ unabhängig auf 1 setzen. Dabei nehmen wir oBdA an, dass für jede Variable x das Gewicht der Einheitsklausel x mindestens so groß ist wie das Gewicht der Einheitsklausel $\neg x$. Für $p = \frac{\sqrt{5}-1}{2}$ ergibt sich ein Approximationsfaktor von $\frac{\sqrt{5}-1}{2} \approx 0.618$.

Beweis. Sei φ eine beliebige KNF-Formel. Sei U die Teilmenge der Klauseln von φ, die alle Klauseln aus φ bis auf die negativen Einheitsklauseln enthält. Das Gesamtgewicht der Klauseln in U beträgt $\sum_{j=1}^{m} w_j - \sum_{i=1}^{n} v_i$.

Wir setzen nun jede Variable unanbhängig voneinander mit Wahrscheinlichkeit p auf 1. Sei C eine beliebige Klausel aus U. Handelt es sich bei C um eine positive Einheitsklausel, so ist sie mit Wahrscheinlichkeit p erfüllt. Enthält C für eine Variable x die Literale x und $\neg x$, so ist C mit Wahrscheinlichkeit 1 erfüllt. Ansonsten bezeichne $\ell \geq 2$ die Anzahl an Literalen in C. Ferner seien a und b die Anzahlen an positiven bzw. negativen Literalen in C. Die Wahrscheinlichkeit, dass C von der zufälligen Belegung der Variablen erfüllt wird, beträgt analog zum Argument aus Lemma 4.27

$$1 - p^b(1-p)^a \geq 1 - p^{a+b} = 1 - p^\ell \geq 1 - p^2.$$

Satz 4.30 folgt nun wieder aus der Linearität des Erwartungswertes:

$$\begin{aligned}
\mathbb{E}[Y] &\geq \sum_{C_j \in U} w_j \cdot \mathbb{P}[\alpha \text{ erfüllt } C_j] \geq \sum_{C_j \in U} w_j \cdot \min\{p, 1-p^2\} \\
&= \min\{p, 1-p^2\} \cdot \left(\sum_{j=1}^{m} w_j - \sum_{i=1}^{n} v_i \right) \\
&\geq \min\{p, 1-p^2\} \cdot \mathrm{OPT}\,.
\end{aligned}$$

□

Auch der Algorithmus aus Satz 4.30 kann mit bedingten Erwartungen derandomisiert werden.

[04.11.2025, Vorlesung 7]
[05.11.2025, Vorlesung 8]

Kapitel 5

Lineare Programmierung und Runden

Problem 5.1: Lineares Programm (LP)

Input: $c \in \mathbb{R}^d, b \in \mathbb{R}^m, A \in \mathbb{R}^{m \times d}$
Output: $x \in \mathbb{R}^d$ mit $Ax \leq b$
Objective: Minimiere/maximiere $c \cdot x$

Beispiel 5.2 (MaxFlow)**.** Sei $G = (V, E)$ ein Flussnetzwerk mit Quelle $s \in V$ und Senke $t \in V$ und Kapazitäten $c : E \to \mathbb{N}_0$. Setze x_e auf den tatsächlichen Fluss für alle $e \in E$. Dann wollen wir die Funktion

$$\sum_{e=(s,v)} x_e - \sum_{e=(v,s)} x_e$$

maximieren unter folgenden Bedingungen:

$$\begin{aligned}
&\forall e \in E : x_e \geq 0 && \text{(Fluss nicht negativ)},\\
&\forall e \in E : x_e \leq c(e) && \text{(Fluss nicht größer als Kapazität)},\\
&\forall v \in V \setminus \{s,t\} : \sum_{e=(u,v)} x_e - \sum_{e=(v,u)} x_e = 0 && \text{(Flusserhaltung)}.
\end{aligned}$$

Lineare Programme können in polynomieller Zeit gelöst werden. Im Folgenden werden *ganzzahlige lineare Programme* eine große Rolle spielen – lineare Programme, bei denen zusätzlich als Nebenbedingung gefordert ist, dass die Werte einiger oder aller Variablen ganzzahlig sein müssen. Das Finden einer solchen Variablenbelegung ist NP-schwer, was z. B. daraus folgt, dass das Rucksackproblem als ILP formuliert werden kann.

5.1 Max-SAT

Mittels LPs können wir einen besseren Approximationsfaktor erreichen.

5.1.1 Randomisiertes Runden

Sei φ eine Eingabe für Max-SAT mit Variablen $x_1, \ldots, x_n$, Klauseln $C_1, \ldots, C_m$ und Gewichten $w_1, \ldots, w_m$. Zunächst formulieren wir die Aufgabe, eine Belegung zu finden, für die das Gesamtgewicht der erfüllten Klausen maximal wird, als ILP:

$$\begin{aligned}
\max \quad & \sum_{j=1}^{m} w_j z_j && \\
s.t. \quad & \sum_{i \in P_j} y_i + \sum_{i \in N_j} (1 - y_i) \geq z_j, && \forall j \in \{1, \ldots, m\},\\
& y_i \in \{0,1\}, && \forall i \in \{1, \ldots, n\},\\
& 0 \leq z_j \leq 1, && \forall j \in \{1, \ldots, m\},
\end{aligned}$$

wobei $z_j = 1$ genau dann gelten soll, wenn die durch $y_1, \dots, y_n$ codierte Belegung die Klausel C_j erfüllt. Mit P_j und N_j sind die Indizes der Variablen gemeint, die positiv bzw. negativ in C_j vorkommen.

Wir haben nun das Problem, eine optimale Variablenbelegung zu finden, auf das Lösen eines ILP reduziert. Dies ist auch NP-schwer. Wir werden dieses ILP aber *relaxieren*, d.h. die Bedingung, dass nur bestimmte Werte angenommen werden dürfen, aufgeben:

$$\begin{aligned} \max \quad & \sum_{j=1}^{m} w_j z_j & & \\ s.t. \quad & \sum_{i \in P_j} y_i + \sum_{i \in N_j} (1 - y_i) \geq z_j, & & \forall j \in \{1, \dots, m\}, \\ & 0 \leq y_i \leq 1, & & \forall i \in \{1, \dots, n\}, \\ & 0 \leq z_j \leq 1, & & \forall j \in \{1, \dots, m\}. \end{aligned}$$

Im Folgenden seien Z^*_{LP} und Z^*_{ILP} die optimalen Zielfunktionswerte des LP bzw. des ILP. Es gilt dann $Z^*_{\text{ILP}} = \text{OPT}$, wobei OPT das Gesamtgewicht der Klauseln angibt, die in einer optimalen Belegung der Aussagenvariablen $x_1, \dots, x_n$ erfüllt sind. Es gilt außerdem offensichtlich $Z^*_{\text{LP}} \geq Z^*_{\text{ILP}}$.

Es bezeichne (y^*, z^*) mit $y^* = (y^*_1, \dots, y^*_n)$ und $z^* = (z^*_1, \dots, z^*_n)$ eine optimale Lösung des linearen Programms. Der Algorithmus RandomizedRounding sieht wie folgt aus:

Algorithmus 5.3 RandomizedRounding

Input: Wie in Problem 4.24
Output: Wie in Problem 4.24

1: Löse das relaxierte LP
2: Setze jede Aussagenvariable x_i mit Wahrscheinlichkeit y^*_i auf 1 und sonst auf 0

Satz 5.4: RandomizedRounding

RandomizedRounding ist ein randomisierter $\left(1 - \frac{1}{e}\right)$-Approximationsalgorithmus für Max-SAT.

Beweis. Wir betrachten wieder für jede Klausel separat die Wahrscheinlichkeit, dass sie durch die zufällige Belegung der Variablen erfüllt wird. Mit obiger Notation erhalten wir für jede Klausel C_j

$$\mathbb{P}[C_j \text{ nicht erfüllt}] = \prod_{i \in P_j} (1 - y^*_i) \cdot \prod_{i \in N_j} y^*_i. \tag{5.1}$$

Dies folgt daraus, dass es nur eine Belegung der in C_j vorkommenden Variablen gibt, die C_j nicht erfüllt: Alle positiv auftretenden Variablen auf 0, alle negativ auftretenden auf 1. Die rechte Seite beschreibt genau die Wahrscheinlichkeit, dass diese Belegung gewählt wird. Wir verwenden die AM-GM-Ungleichung $\prod_{i=1}^n a_i \leq \left(\frac{1}{n} \sum_{i=1}^n a_i\right)^n$ für alle $a_1, \dots, a_n \in \mathbb{R}_{\geq 0}$. Angewendet auf (5.1)

ergibt sich mit $|C_j| = |P_j| + |N_j|$

$$\begin{aligned}\prod_{i\in P_j}(1-y_i^*)\cdot\prod_{i\in N_j}y_i^* &\le \left(\frac{1}{|C_j|}\left(\sum_{i\in P_j}(1-y_i^*)+\sum_{i\in y_i^*}\right)\right)^{|C_j|}\\ &= \left(\frac{1}{|C_j|}\left(|C_j|+\sum_{i\in P_j}(-y_i^*)+\sum_{i\in N_j}(y_i^*-1)\right)\right)^{|C_j|}\\ &= \left(1-\frac{1}{|C_j|}\left(\sum_{i\in P_j}y_i^*+\sum_{i\in N_j}(1-y_i^*)\right)\right)^{|C_j|}\end{aligned}$$

und mit der Nebenbedingung

$$\sum_{i\in P_j}y_i^*+\sum_{i\in N_j}(1-y_i^*)\ge z_j,$$

die das LP garantiert, ergibt sich insgesamt

$$\mathbb{P}[C_j \text{ nicht erfüllt}] \le \left(1-\frac{z_j^*}{|C_j|}\right)^{|C_j|}$$

und dementsprechend

$$\mathbb{P}[C_j \text{ erfüllt}] \ge 1-\left(1-\frac{z_j^*}{|C_j|}\right)^{|C_j|}.$$

Betrachte die Funktion $f(x) = 1-\left(1-\frac{x}{|C_j|}\right)^{|C_j|}$ genauer. Die zweite Ableitung auf $[0,1]$ ist für jedes $|C_j| \ge 1$ nicht positiv, folglich ist die Funktion konkav und die Gerade g, die durch die Punkte $(0, f(0))$ und $(1, f(1))$ verläuft, liegt an keiner Stelle über der Funktion f. Wegen $f(0) = 0$ entspricht g der Funktion

$$g(x) = f(1)\cdot x = \left(1-\left(1-\frac{1}{|C_j|}\right)^{|C_j|}\right)\cdot x$$

und es gilt insgesamt

$$\mathbb{P}[C_j \text{ erfüllt}] \ge f(z_j^*) \ge g(z_j^*) = \left(1-\left(1-\frac{1}{|C_j|}\right)^{|C_j|}\right)\cdot z_j^*.$$

Bezeichnen wir wieder mit Y das Gesamtgewicht der erfüllten Klauseln, so ergibt sich mit der Linearität des Erwartungswertes

$$\begin{aligned}\mathbb{E}[Y] &= \sum_{j=1}^m w_j\mathbb{P}[C_j \text{ erfüllt}] \ge \sum_{j=1}^m w_j\left(1-\left(1-\frac{1}{|C_j|}\right)^{|C_j|}\right)\cdot z_j^*\\ &\ge \min_{k\ge1}\left(1-\left(1-\frac{1}{k}\right)^k\right)\cdot\sum_{j=1}^m w_j z_j^* = \min_{k\ge1}\left(1-\left(1-\frac{1}{k}\right)^k\right)\cdot Z_{\mathrm{LP}}^*\\ &\ge \min_{k\ge1}\left(1-\left(1-\frac{1}{k}\right)^k\right)\cdot \mathrm{OPT}\,.\end{aligned}$$

Die Funktion $1-\left(1-\frac{1}{k}\right)^k$ ist streng monoton fallend und geht von oben gegen $1-\frac{1}{e}$. Damit erhalten wir

$$\mathbb{E}[Y] \ge \left(1-\frac{1}{e}\right)\cdot \mathrm{OPT}\,. \qquad \square$$

Es gilt $\left(1-\frac{1}{e}\right) \approx 0.632$ und somit erreicht RANDOMIZEDROUNDING einen etwas besseren Approximationsfaktor als die Algorithmen aus Kapitel 4.3. Kombinieren macht es aber noch besser:

5.1.2 Kombination beider Algorithmen

Betrachten wir RANDMAXSAT und RANDOMIZEDROUNDING genauer, so stellen wir fest, dass der Approximationsfaktor von RANDMAXSAT besser wird, je länger die Klauseln wird, und der Approximationsfaktor von RANDOMIZEDROUNDING besser wird, je kürzer die Klauseln sind. Konkret gilt für jede Klausel, dass sie durch RANDOMIZEDROUNDING mit einer Wahrscheinlichkeit von mindestens $\left(1-\left(1-\frac{1}{|C_j|}\right)^{|C_j|}\right) \cdot z_j^*$ erfüllt wird und durch RANDMAXSAT von mindestens $1-2^{-|C_j|}$.

Algorithmus 5.5 BETTEROFTWO

Input: Wie in Problem 4.24
Output: Wie in Problem 4.24

1: Führe RANDOMIZEDROUNDING aus
2: Führe RANDMAXSAT aus
3: Gib die bessere Belegung zurück

Satz 5.6: BetterOfTwo

BETTEROFTWO ist ein randomisierter $\frac{3}{4}$-Approximationsalgorithmus für MAX-SAT.

Beweis. Wir bezeichnen mit Y_1 und Y_2 das Gesamtgewicht der durch RANDOMIZEDROUNDING und RANDMAXSAT erfüllten Klauseln. Da BETTEROFTWO beide Algorithmen ausführt und anschließend das bessere Ergebnis wählt, interessieren wir uns für den Erwartungswert von $\max\{Y_1, Y_2\}$. Es gilt

$$\mathbb{E}[\max\{Y_1, Y_2\}] \geq \mathbb{E}\left[\frac{Y_1+Y_2}{2}\right] = \frac{\mathbb{E}[Y_1]+\mathbb{E}[Y_2]}{2}.$$

Einsetzen der Infos aus den Beweisen von Sätzen 5.4 und 4.25 liefert

$$\begin{aligned}\frac{\mathbb{E}[Y_1]+\mathbb{E}[Y_2]}{2} &\geq \frac{1}{2}\sum_{j=1}^{m} w_j \left(1-\left(1-\frac{1}{|C_j|}\right)^{|C_j|}\right) \cdot z_j^* + \frac{1}{2}\sum_{j=1}^{m} w_j \left(1-2^{-|C_j|}\right) \\ &\geq \sum_{j=1}^{m} w_j z_j^* \left(\frac{1}{2}\left(1-\left(1-\frac{1}{|C_j|}\right)^{|C_j|}\right) + \frac{1}{2}\left(1-2^{-|C_j|}\right)\right).\end{aligned}$$

Wir betrachten nun die Funktion

$$f(x) = \frac{1}{2}\left(1-\left(1-\frac{1}{x}\right)^{x}\right) + \frac{1}{2}\left(1-2^{-x}\right).$$

Diese Funktion nimmt für alle ganzzahligen Werte mindestens den Wert $\frac{3}{4}$ an:

$$f(1) = \frac{1}{2}+\frac{1}{2}\cdot\frac{1}{2} = \frac{3}{4} \quad \text{und} \quad f(2) = \frac{1}{2}\left(1-\frac{1}{4}\right)+\frac{1}{2}\left(1-\frac{1}{4}\right) = \frac{3}{4}.$$

Für $x \geq 3$ gilt

$$f(x) \geq \frac{1}{2}\left(1-\frac{1}{e}\right)+\frac{1}{2}\left(1-2^{-3}\right) \approx 0.753 \geq \frac{3}{4}.$$

Damit gilt insgesamt

$$\mathbb{E}[\max\{Y_1, Y_2\}] \geq \frac{3}{4} \sum_{j=1}^{m} w_j z_j^* = \frac{3}{4} \cdot Z_{\text{LP}}^* \geq \frac{3}{4} \cdot \text{OPT}\,. \qquad \square$$

5.1.3 Nichtlineares randomisiertes Runden

Algorithmus 5.7 RANDROUNDING$_f$

Input: Wie in Problem 4.24, Funktion $f : [0,1] \to [0,1]$
Output: Wie in Problem 4.24

1: Löse das relaxierte LP
2: Setze jede Aussagenvariable x_i mit Wahrscheinlichkeit $f(y_i^*)$ auf 1 und sonst auf 0
3: ▷ *Für $f(x) = x$ ist dies einfach* RANDOMIZEDROUNDING. ◁

Wir analysieren den Approximationsfaktor von RANDROUNDING$_f$ für eine beliebige Funktion $f : [0,1] \to [0,1]$ mit

$$1 - 4^{-x} \leq f(x) \leq 4^{x-1} \tag{5.2}$$

für alle $x \in [0,1]$. Wir werden sehen, dass damit jede Klausel C_j mit Wahrscheinlichkeit von mindestens $\frac{3}{4} z_j^*$ erfüllt ist.

Satz 5.8: RandRounding$_f$

Für jede Funktion f mit Eigenschaft (5.2) ist RANDROUNDING$_f$ *ein randomisiserter $\frac{3}{4}$-Approximationsalgorithmus für* MAX-SAT.

Beweis. Wir zeigen für jede Klausel, dass sie mit Wahrscheinlichkeit von mindestens $\frac{3}{4} z_j^*$ erfüllt ist. Der Satz folgt dann aus dieser Aussage analog zum Beweis von Satz 5.4.

Sei C_j eine beliebige Klausel. Wieder bezeichnen P_j und N_j die Mengen der Variablen, die positiv bzw. negativ in C_j vorkommen. Die Klausel C_j ist nur dann nicht erfüllt, wenn alle positiv auftretenden Variablen auf 0 und alle neagtiv auftretenden Variablen auf 1 gesetzt sind. Daraus folgt

$$\mathbb{P}[C_j \text{ nicht erfüllt}] = \prod_{i \in P_j} (1 - f(y_i^*)) \cdot \prod_{i \in N_j} f(y_i^*) \leq \prod_{i \in P_j} 4^{-y_i^*} \cdot \prod_{i \in N_j} 4^{y_i^* - 1}$$

durch (5.2). Es gilt mit der Ungleichung

$$\sum_{i \in P_j} y_i^* + \sum_{i \in N_j} (1 - y_i^*) \geq z_j^*$$

aus dem LP gerade

$$\prod_{i \in P_j} 4^{-y_i^*} \cdot \prod_{i \in N_j} 4^{y_i^* - 1} = 4^{-\left(\sum_{i \in P_j} y_i^* + \sum_{i \in N_j} (1 - y_i^*)\right)} \leq 4^{-z_j^*}.$$

Es gilt somit

$$\mathbb{P}[C_j \text{ erfüllt}] \geq 1 - 4^{-z_j^*}.$$

Da die Funktion $g(x) = 1 - 4^{-x}$ in $[0,1]$ konkav ist, ist die Gerade h, die durch die Punkte $(0, g(0))$ und $(1, g(1))$ geht, eine untere Schranke (analog zum Beweis von Satz 5.4). Wegen $g(0) = 0$ gilt $h(x) = g(1) \cdot x$ und somit insgesamt

$$\mathbb{P}[C_j \text{ erfüllt}] \geq 1 - 4^{-z_j^*} = g(z_j^*) \geq h(z_j^*) = g(1) \cdot z_j^* = \left(1 - \frac{1}{4}\right) z_j^* = \frac{3}{4} z_j^*.$$

Wieder bezeichnen wir mit Y das Gesamtgewicht der erfüllten Klauseln. Insgesamt ergibt sich

$$\begin{aligned}\mathbb{E}[Y] &= \sum_{j=1}^{m} w_j \cdot \mathbb{P}[C_j \text{ erfüllt}] \\ &\geq \sum_{j=1}^{m} w_j \cdot \frac{3}{4} z_j^* = \frac{3}{4} \sum_{j=1}^{m} w_j z_j^* \\ &= \frac{3}{4} \cdot Z_{\text{LP}}^* \geq \frac{3}{4} \cdot \text{OPT}\,. \end{aligned}$$

□

Wir fragen uns nun, ob ein besserer Approximationsfaktor als $\frac{3}{4}$ erreicht werden kann.

Beispiel 5.9 (Instanz für Max-SAT)**.** Wir betrachten die Formel

$$\varphi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2),$$

wobei alle Klauseln das Gewicht 1 haben. Man sieht leicht, dass jede Belegung der Aussagenvariablen genau drei der Klauseln erfüllt. Somit gilt $\text{OPT} = 3$. In der optimalen Lösung des LP gilt $y_1^* = y_2^* = \frac{1}{2}$ und $z_j = 1$ für alle Klauseln. Folglich ist $Z_{\text{LP}}^* = 4$.

Es gibt also Eingaben φ für Max-SAT für die $\text{OPT} \leq \frac{3}{4} Z_{\text{LP}}^*$ gilt, also können wir keinen besseren Approximationsfaktor als $\frac{3}{4}$ zeigen, wenn wir als obere Schranke nur Z_{LP}^* verwenden. Dafür haben wir aber folgende Verallgemeinerung:

Definition 5.10: Integrality Gap

Das *Integrality Gap* ist der Quotient aus dem optimalen Wert des ILPs und dem optimalen Wert des LPs, das durch Relaxierung des ILPs entsteht.
Haben wir für ein Optimierungsproblem eine Formulierung der Instanzen als ILP gefunden, so bezeichnen wir mit dem *Integrality Gap dieser Formulierung* das schlechteste Integrality Gap einer Instanz des Optimierungsproblems.

Bei Maximierungsproblemen betrachten wir also das kleinste Integrality Gap über alle Instanzen und bei Minimierungsproblemen das größte.

Allgemein gilt, dass wir für einen Approximationsalgorithmus keinen besseren Approximationsfaktor als das Integrality Gap zeigen können, wenn wir als einzige Schranke für den Wert einer optimalen Lösung die optimale Lösung des relaxierten LPs benutzen.

[05.11.2025, Vorlesung 8]
[11.11.2025, Vorlesung 9]

5.2 Facility Location ohne Kapazitäten

Problem 5.11: Facility Location (ohne Kapazitäten)

Input: Menge F von Standorten (*Facilities*), Menge D von Kunden; Distanzen $d : F \times D \to \mathbb{R}_{\geq 0}$, Kosten $f : F \to \mathbb{R}_{\geq 0}$. Wir schreiben d_{ij} für $d(i,j)$ und f_i für $f(i)$.
Output: Teilmenge $S \subseteq F$ mit $S \neq \emptyset$
Objective: Minimiere $\sum_{i \in S} f_i + \sum_{j \in D} \min_{i \in S} d_{ij}$

Es geht darum, eine optimale Menge von Fabrikstandorten zu wählen. Die Kosten, um einen Kunden von einer bestimmten Fabrik zu beliefern, sind durch d gegeben, wobei jeder Kunde

von der nächsten Fabrik beliefert wird. Die Gesamtkosten einer Menge von Fabrikstandorten ergeben sich durch die Kosten zum Eröffnen der Standorte (gegeben durch f) sowie die Kosten, um alle Kunten zu beliefern. Dieses Problem ist NP-schwer.

5.2.1 Reduktion auf Set Cover

Sei (D, F, d, f) eine Eingabe für Facility Location. Wir konstruieren daraus eine Menge für Set Cover, bei der die Kunden die abzudeckende Grundmenge S bilden. Die Familie der Teilmengen von S, die wir zur Abdeckung von S einsetzen können, heiße T. Die Familie T enthält dabei jede mögliche Teilmenge $A \subseteq S$ mit $A \neq \emptyset$ genau $|F|$ mal, und zwar für jedes $i \in F$ einmal mit Kosten $f_i + \sum_{j \in A} d_{ij}$. Somit können wir jeder Menge aus T einen Standort $i \in F$ zuordnen.

In einer optimalen Lösung für die konstruierte Instanz von Set Cover können keine zwei Mengen $S_1, S_2 \in T$ ausgewählt sein, die demselben Standort $i \in F$ zugeordnet sind, diese könnten sonst durch die Vereinigung $S_1 \cup S_2$ ersetzt werden, was die Kosten um f_i senken würde. Somit interessieren uns nur Lösungen für die Instanz von Set Cover, die für jedes $i \in F$ höchstens eine Menge aus T auswählen. Ebenso können wir annehmen, dass die ausgewählten Mengen aus T disjunkt sind, da wir ansonsten Elemente aus den Mengen entfernen könnten, ohne die Kosten zu erhöhen.

Zwischen den Teilmengen von T, die alle Elemente aus S abdecken, paarweise disjunkt sind und für kein $i \in F$ mehr als eine Menge enthalten, auf der einen Seite, und den Lösungen für die Instanz von Facility Location auf der anderen Seite, existiert eine naheliegende Bijektion. Modulo dieser Bijektion stimmen auch die optimalen Lösungen überein.

Wir wenden nun GreedySC (Algorithmus 2.6) auf die konstruierte Instanz an. Dieser wählt in jedem Schritt eine Menge aus T mit minimalen relativen Kosten aus und fügt diese der Lösung hinzu. Hat er mit den bislang gewählten Mengen aus T bereits eine Teilmenge $D' \subseteq D$ der Kunden abgedeckt, so bedeutet dies, dass er einen Standort $i \in F$ und eine Menge $A \subseteq D$ auswählt, für die

$$\frac{f_i + \sum_{j \in A} d_{ij}}{|A \setminus D'|} \tag{5.3}$$

minimal wird. Gemäß Satz 2.7 erreicht dieser Algorithmus einen Approximationsfaktor von $H_{|D|} \in \mathcal{O}(\log(|D|))$. Allerdings ist nicht klar, dass der Greedy-Algorithmus eine polynomielle Laufzeit besitzt, da T exponentiell groß ist. Dennoch kann in polynomieller Zeit ein Paar (i, A) mit $i \in F$ und $A \subseteq D$ gefunden werden, für das (5.3) minimal wird. Dazu beachte, dass für ein Paar (i, s) mit $i \in F$ und $s \in \{d, \dots, |D \setminus D'|\}$ in polynomieller Zeit eine Menge A mit $|A \setminus D'| = s$ gefunden werden kann, für die (5.3) minimal wird. Dazu genügt es, A als die Menge der s nächsten Nachbarn von i aus der Menge $D \setminus D'$ zu wählen. Der Algorutgnzs testet dann alle polynomiell vielen Wahlen von (i, s) mit $i \in F$ und $s \in \{1, \dots, |D \setminus D'|\}$ und findet so die Teilmenge mit den insgesamt kleinsten relativen Kosten. Es ergibt sich also folgendes Resultat:

Satz 5.12: Facility Location

Für das Facility-Location-Problem existiert ein $\mathcal{O}(\log(|D|))$-Approximationsalgorithmus.

5.2.2 Metrisches Facility Location

Wir nehmen nun an, dass $D \cup F$ in eine Metrik eingebettet ist, folglich also die Dreiecksungleichnug $d_{xy} + d_{yz} \geq d_{xz}$ für alle $x, y, z \in D \cup F$ gilt. Wir schreiben zunächst eine gegebene Eingabe als ILP. Dafür führen wir Variablen $x_{ij} \in \{0, 1\}$ und $y_i \in \{0, 1\}$ für alle $i \in F$ und $j \in D$ ein. Dabei soll $y_i = 1$ genau dann gelten, wenn Standort i eröffnet wird, und $x_{ij} = 1$ genau dann, wenn Kunde j von Standort i beliefert wird. Wir müssen noch garantieren, dass jeder Kunde

von einem *geöffneten* Standort beliefert wird und erhalten folgendes ILP:

$$\begin{aligned}
\min \quad & \sum_{i\in F} f_i y_i + \sum_{i\in F}\sum_{j\in D} d_{ij}x_{ij} & \\
s.t. \quad & \sum_{i\in F} x_{ij} \geq 1, & \forall j \in D,\\
& x_{ij} \leq y_i, & \forall i \in F, j \in D,\\
& y_i \in \{0,1\}, & \forall i \in F,\\
& x_{ij} \in \{0,1\}, & \forall i \in F, j \in D.
\end{aligned}$$

Da die Zielfunktion minimiert wird, ist sichergestellt, dass in der optimalen Lösung sogar $\sum_{i\in F} x_{ij} = 1$ gilt. Wir betrachten nun aber wieder die Relaxierung des ILP, indem wir nur noch $x_{ij} \in [0,1]$ und $y_i \in [0,1]$ fordern, dann erhalten wir ein LP, welches wir in polynomieller Zeit optimal lösen können. Sei (x^*, y^*) eine optimale Lösung dieses LPs. Wir werden diese Lösung wieder durch eine geeignete Rundung (dieses Mal aber deterministisch) ganzzahlig machen.

Wir bezeichnen mit

$$\mathrm{OPT}_{\mathrm{LP}} = \sum_{i\in F} f_i y_i^* + \sum_{i\in F}\sum_{j\in D} d_{ij}x_{ij}^*$$

die Kosten der optimalen Lösung des LPs. Ferner bezeichnen wir mit $F^* = \sum_{i\in F} f_i y_i^*$ die Kosten für das Eröffnen der Standorte und mit $D^* = \sum_{i\in F}\sum_{j\in D} d_{ij}x_{ij}^*$ die Kosten für das Beliefern der Kunden in der optimalen Lösung des LPs.

Für einen Kunden $j \in D$ seien $D_j^* = \sum_{i\in F} d_{ij}x_{ij}^*$ die Gesamtkosten, um diesen Kunden zu beliefern. Wir definieren die *Nachbarschaft* N_j von Kunden j als

$$N_j = \Big\{ i \in F \mid d_{ij} \leq 2D_j^* \text{ und } x_{ij}^* > 0 \Big\}.$$

Lemma 5.13: Kunde von Hälfte der Nachbarschaft beliefert

Für jeden Kunden $j \in D$ gilt

$$\sum_{i\in N_j} x_{ij}^* \geq \frac{1}{2}.$$

Beweis. Angenommen, dies ist nicht der Fall. Dann folgt aus $\sum_{i\in F} x_{ij}^* \geq 1$ als Nebenbedingung direkt $\sum_{i\in F\setminus N_j} x_{ij}^* > \frac{1}{2}$. Dies impliziert den Widerspruch

$$D_j^* = \sum_{i\in F} d_{ij}x_{ij}^* \geq \sum_{i\in F\setminus N_j} d_{ij}x_{ij}^* > 2D_j^* \sum_{i\in F\setminus N_j} x_{ij}^* > D_j^*. \qquad \square$$

Wir wandeln die Lösung (x^*, y^*) nun in eine ganzzahlige Lösung um. Dazu sei $j \in D$ ein Kunde mit minimalen Verbindungskosten D_j^*, und sei $k = \arg\min_{i\in N_j} f_i$ der günstigste Standort in der Nachbarstaft von Kunden j. Wir setzen $y_k'' = 1$ und $y_i'' = 0$ für alle $i \in N_j \setminus \{k\}$. Das heißt, Standort k wird eröffnet und alle anderen Standorte aus der Nachbarschaft von j bleiben geschlossen. Setze außerdem $x_{kj}'' = 1$ und $x_{ij}'' = 0$ für alle $i \neq k$. Das heißt, Kunde j wird in der ganzzahligen Lösung von Standort k beliefert. Ferner ordnen wir alle Kunden ℓ mit $N_j \cap N_\ell \neq \emptyset$ ebenfalls Standort k zu, setzen also $x_{k\ell}'' = 1$ und $x_{i\ell}'' = 0$ für alle $i \neq k$.

Solange es noch einen Kunden gibt, der keinem Standort zugewiesen wurde, fahren wir nach diesem Schema fort. Das heißt wir wählen einen Kunden $j \in D$, der unter allen noch nicht zugewiesenen Kunden minimale Verbindungskosten D_j^* besitzt. Für diesen Kunden wählen wir den günstigsten Standort k, öffnen diesen und weisen j sowie allen Kunden Standort k zu, die noch nicht zugewiesen sind und einen Standort in der Nachbarschaft haben, der auch in der Nachbarschaft von Kunde j liegt.

Wir zeigen, dass die ganzzahlige Lösung (x'', y'') eine 6-Approximation berechnet. Da diese effizient gefunden werden kann, ist obiges Verfahren ein 6-Approximationsalgorithmus für Facility Location.

Satz 5.14: Metrisches Facility Location

Die Gesamtkosten der ganzzahligen Lösung (x'', y'') sind höchstens 6-mal so groß wie die Kosten $\mathrm{OPT}_{\mathrm{LP}}$ der optimalen Lösung (x^*, y^*) des LPs.

Beweis. Sei D' die Menge aller Kunden, für die ein neuer Standort eröffnet wird. Bei der Betrachtung eines Kunden $j \in D'$ eröffnen wir Standort $k = \arg\min_{i \in N_j} f_i$. Die Kosten dafür betragen

$$f_k \leq 2 \sum_{i \in N_j} f_i y_i^*,$$

wobei wir ausgenutzt haben, dass $f_k \leq f_i$ für alle $i \in N_j$ gilt und dass nach Lemma 5.13

$$\sum_{i \in N_j} y_i^* \geq \sum_{i \in N_j} x_{ij}^* \geq \frac{1}{2}$$

gilt. Wir beobachten außerdem, dass die Nachbarschaften der Kunden in D' disjunkt sein müssten, denn wenn $N_j \cap N_{j'} \neq \emptyset$ und es wird für Kunde j ein neuer Standort eröffnet, dann wŕde j' ebenfalls diesem Standort zugeordnet werden. Somit folgt, dass jeder Standort $i \in F$ in höchstens einer Menge N_j mit $j \in D'$ enthalten sein kann.

Insgesamt lassen sich die Eröffnungskosten damit beschränken durch

$$\sum_{j \in D'} \min_{i \in N_j} f_i \leq \sum_{j \in D'} \sum_{i \in N_j} f_i 2 y_i^* \leq 2 \sum_{i \in F} f_i y_i^* = 2F^*,$$

wobei die zweite Ungleichung folgt, da jeder Standort $i \in F$ in höchstens einer Menge N_j mit $j \in D'$ enthalten sein kann.

Zuletzt müssen wir noch abschätzen, wie hoch die Verbindungskosten sind. Betrachten wir wieder einen Schritt, in dem Kunde $j \in D$ mit minimalen Verbindungskosten D_j^* ausgewählt wird. Dieser wird einem Standort k in seiner Nachbarschaft zugewiesen. Aus der Definition der Nachbarschaft folgt $d_{kj} \leq 2D_j^*$. Alle Kunden, die einen gemeinsamen Standort in der Nachbarschaft von j haben, weden nun ebenfalls dem Standort k zugewiesen. Sei ℓ ein solcher Kunde und $i \in N_j \cap N_\ell$. Nun gelten $d_{kj} \leq 2D_j^*$ und $d_{ij} \leq 2D_j^*$, da $k, i \in N_j$. Außerdem gilt $d_{i\ell} \leq 2D_\ell^*$, da $i \in N_\ell$. Nach Dreiecksungleichung gilt damit

$$d_{k\ell} \leq d_{kj} + d_{ij} + d_{i\ell} \leq 2D_j^* + 2D_j^* + 2D_\ell^* \leq 6D_\ell^*.$$

Im letzten Schritt haben wir ausgenutzt, dass Kunde $j \in D$ gemäß seiner Wahl unter allen noch nicht zugewiesenen Kunden minimale Verbindungskosten D_j^* besitzt und insbesondere also $D_j^* \leq D_\ell^*$ gilt. Somit betragen die Gesamtkosten der Lösung (x'', y'') höchstens

$$2F^* + \sum_{j \in D} 6D_j^* = 2F^* + 6D^* \leq 6(F^* + D^*) = 6\,\mathrm{OPT}_{\mathrm{LP}}\,. \qquad \square$$

Im Beweis fällt auf, dass in der gerundeten Lösung (x'', y'') das Eröffnen der Standorte höchstens doppelt so viel kostet wie in der optimalen Lösung des LPs, während das Beliefern der Kunden bis zu sechsmal so teuer sein kann. Durch eine geschicktere Wahl der Nachbarschaft können diese Beiträge balanciert werden, was insgesamt zu einem besseren Approximationsalgorithmus führt: Sei $\alpha = 1$. Wir definieren die α-Nachbarschaft von Kunde j als

$$N_j^{(\alpha)} = \left\{ i \in F \mid d_{ij} \leq \alpha D_j^* \text{ und } x_{ij}^* > 0 \right\}.$$

Dann ist $N_j = N_j^{(2)}$. Der Algorithmus läuft nun genauso wie oben, nur mit der $N_j^{(\alpha)}$-Nachbarschaft. Das Lemma würde entsprechend zeigen, dass

$$\sum_{i \in N_j^{(\alpha)}} x_{ij}^* \geq 1 - \frac{1}{\alpha}$$

gilt. Dann kann auch der Beweis von Satz 5.14 übertragen werden. Damit kann gezeigt werden, dass

$$\sum_{i \in F} f_i y_i' + \sum_{j \in D} 3\alpha D_j^* \leq \frac{\alpha}{\alpha - 1} \sum_{i \in F} f_i y_i^* + 3\alpha D^* \leq \max\left\{\frac{\alpha}{\alpha - 1}, 3\alpha\right\} \cdot \mathrm{OPT}_{\mathrm{LP}},$$

wobei dieser Wert minimal ist für $\alpha = \frac{4}{3}$ und wir erhalten einen 4-Approximationsalgorithmus.

[11.11.2025, Vorlesung 9]
[12.11.2025, Vorlesung 10]

5.3 Scheduling auf allgemeinen Maschinen

Problem 5.15: Scheduling auf allgemeinen Maschinen

Input: Menge $J = \{1, \ldots, n\}$ von Jobs; Menge $M = \{1, \ldots, m\}$ von Maschinen; Jobgröße $p_{ij} \in \mathbb{R}_{>0}$ für alle $i \in M$ und $j \in J$

Output: Abbildung (*Schedule*) $\pi : J \to M$

Objective: Es sei $L_i(\pi) = \sum_{\substack{j \in J \\ \pi(j) = i}} p_{ij}$ die Ausführungszeit von Maschine i in Schedule π. Minimiere den *Makespan* $C(\pi) = \max_{i \in M} L_i(\pi)$

Der einzige Unterschied zum Scheduling auf identischen Maschinen (Problem 2.9) besteht darin, dass ein Job $j \in J$ nicht eine feste Größe p_j, sondern auf jeder Maschine $i \in M$ eine individuelle Größe p_{ij} besitzt. Es ist bekannt, dass für P≠NP für Scheduling auf allgemeinen Maschinen kein Approximationsalgorithmus mit besseren Approximationsfaktor als $\frac{3}{2}$ existiert. Der beste bekannte Approximationsalgorithmus ist ein 2-Approximationsalgorithmus, welchen wir auch besprechen werden.

Dazu formulieren wir dies wieder als ILP. $x_{ij} \in \{0, 1\}$ gibt an, ob Job j Maschine i zugewiesen wird, und $t \in \mathbb{R}$ beschreibe den Makespan des Schedules:

$$\begin{aligned}
\min \quad & t \\
s.t. \quad & \sum_{i \in M} x_{ij} = 1, && \forall j \in J, \\
& \sum_{j \in J} p_{ij} x_{ij} \leq t, && \forall i \in M, \\
& x_{ij} \in \{0, 1\}, && \forall i \in M, j \in J.
\end{aligned}$$

Versuchen wir mithilfe dieses ILP einen Approximationsalgorithmus durch Relaxieren zu entwerden, stellen wir fest, dass dies nicht zum Erfolg führen kann, da das Integrality Gap sehr groß ist:

Beispiel 5.16 (Großes Integrality Gap)**.** Wir haben einen Job und m Maschinen. Der Job habe überall die Größe 1.

Dann besitzt die optimale ganzzahlige Lösung einen Makespan von 1. In der optimalen Lösung des relaxierten LP wird hingegen jeder Maschine ein $\frac{1}{m}$-Bruchteil zugewisen, also $x_{i1} = \frac{1}{m}$ für

alle $i \in M$, sodass die optimale Lösung des relaxierten LPs den Wert $\frac{1}{m}$ besitzt. Das Integrality Gap beträgt damit m.

Unter Verwendung der optimalen Lösung des relaxierten LP als untere Schranke können wir also keinen Approximationsalgorithmus zeigen, der einen besseren Approximationsfaktor als m hat.

Stattdessen gehen wir davon aus, dass wir als Eingabe bereits einen Makespan T erhalten. Wir testen dann, ob das relaxierte LP eine Lösung mit Wert höchstens T besitzt, wobei wir die obigen Nebenbedingungen sicher stellen. Sei dazu $S_T = \{(i,j) \in M \times J \mid p_{ij} \leq T\}$. Wir betrachten nun das folgende LP LP_T ohne Zielfunktion:

$$\sum_{i \in M} x_{ij} = 1, \qquad \forall j \in J, \tag{5.4}$$

$$\sum_{j \in J} p_{ij} x_{ij} \leq T, \qquad \forall i \in M, \tag{5.5}$$

$$x_{ij} = 0, \qquad \forall (i,j) \notin S_T, \tag{5.6}$$

$$x_{ij} \geq 0, \qquad \forall (i,j) \in S_T. \tag{5.7}$$

Wir haben bei diesem LP keine Zielfunktion angegeben, da wir nur testen möchten, ob überhaupt eine zulässige Lösung existiert. Dies ist ebenfalls in polynomieller Zeit möglich. Besitzt LP_T keine Lösung, so offensichtlich auch keine ganzzahlige. Das bedeutet, dass der Makespan OPT der optimalen Lösung größer als T sein muss. Mit einer binären Suche kann in polynomieller Zeit der Wert

$$T^* = \min\{T \mid \mathrm{LP}_T \text{ besitzt eine Lösung}\}$$

bestimmt werden. Für diesen gilt $T^* \leq \mathrm{OPT}$, da $T^* - \varepsilon < \mathrm{OPT}$ für jedes $\varepsilon > 0$ gilt. Wir zeigen nun, wie aus einer zulässigen Lösung für LP_{T^*} eine ganzzahlige Lösung mit Makespan höchstens $2T^* \leq 2\,\mathrm{OPT}$ konstruiert werden kann.

Wir müssen uns nun intensiver mit der LP-Struktur beschäftigen und könnnen diese nicht mehr nur als Blackbox behandeln:

In einem LP mit d Variablen entspricht jede Variablenbelegung einem Punkt im $\mathbb{R}^d$. Jede lineare Nebenbedingung definiert einen Halbraum im $\mathbb{R}^d$ und der Schnitt all dieser Halbräume entspricht der Menge der zulässigen Lösungen für das LP. Einen solchen Schnitt nennt man *konvexes Polyeder*, und das zu einem LP gehörende konvexe Polyeder nennen wir auch seinen *Lösungspolyeder*.

Wir betrachten den Rand konvexer Polyeder genauder. Sei $\mathcal{P} \subseteq \mathbb{R}^d$ ein beliebiges konvexes Polyeder und sei eine Hyperebene $\mathcal{A} = \{x \in \mathbb{R}^d \mid u \cdot x = w\}$ mit beliebigen $u \in \mathbb{R}^d$ und $w \in \mathbb{R}$ gegeben. Ist $\mathcal{P} \cap \mathcal{A} \neq \emptyset$ und gilt $\mathcal{P} \subseteq \mathcal{H}$, wobei $\mathcal{H} = \{x \in \mathbb{R} \mid u \cdot x \leq w\}$ den zu $\mathcal{A}$ gehörenden Halbraum bezeichnet, so sagen wir, dass $\mathcal{A}$ eine *stützende Hyperebene* von $\mathcal{P}$ ist. Für jede stützende Hyperebene nennen wir die Menge $\mathcal{P} \cap \mathcal{A}$ *Fläche von* $\mathcal{P}$. Eine nulldimensionale Fläche heißt *Ecke von* $\mathcal{P}$, eine eindimensionale Fläche heißt *Kante von* $\mathcal{P}$ und eine Fläche der Dimension $d-1$ heißt *Seitenfläche von* $\mathcal{P}$.

Die Ecken des Lösungspolyeders entsprechen den zulässigen Basislösungen. Geometrisch wird eine Ecke durch den Schnitt von d linear unabhängigen Hyperebenen definiert, in einer Ecke sind also mindestens d der Nebenbedingungen mit Gleichheit erfüllt. Eine Lösung x ist genau dann eine zulässige Basislösung, wenn kein $y \in \mathbb{R}^d \setminus \{0\}$ existiert, sodass $x + y \in \mathcal{P}$ und $x - y \in \mathcal{P}$ gilt. Außerdem gibt es zu jedem zulässigen und beschränkten LP eine zulässige Basislösung existiert, die optimal ist, wenn alle Variablen nur nichtnegative Werte annehmen können. Eine solche kann in polynomieller Zeit gefunden werden, wir können also in polynomieller Zeit eine zulässige Basislösung von LP_{T^*} berechnen.

Lemma 5.17: Zulässige Basislösung von LP_{T^*}

In einer zulässigen Basislösung von LP_{T^*} besitzen höchstens $n+m$ Variablen einen Wert echt größer als Null.

Beweis. Die Anzahl Variablen in LP_{T^*} beträgt nm. Das bedeutet, in einer zulässigen Basislösung müssen mindestens nm Nebenbedingungen mit Gleichheit erfüllt sein. Es gibt n Bedingungen vom Typ (5.4) und m Bedingungen vom Typ (5.5). Selbst wenn alle diese mit Gleichheit erfüllt sind, müssen noch mindestens $nm-(n+m)$ viele Bedingungen vom Typ (5.6) oder (5.7) mit Gleichheit erfüllt sein. □

Sei nun x eine zulässige Basislösung von LP_{T^*}. Zu dieser Lösung definieren wir den *Allokationsgraphen* G. Dabei handelt es sich um einen bipartiten Graphen mit Knotenmenge $M \cup J$ und Kantenmenge $E = \{(i,j) \in M \times J \mid x_{ij} > 0\}$. Eine Kante (i,j) in G besagt also, dass Job j in der zulässigen Basislösung x zumindest teilweise Maschine i zugewiesen wird. Wir nennen einen Graphen mit Knotenmenge V einen *Pseudobaum*, falls dieser zusammenhängend ist und höchstens $|V|$ Kanten enthält. Da jeder Spannbaum eines solchen Graphen genau $|V|-1$ Kanten enthält, handelt es sich bei einem Pseudobaum entweder um einen Spannbaum oder um einen Spannbaum mit einer zusätzlichen Kante, die einen Kreis schließt. Das folgende Korollar folgt direkt aus Lemma 5.17:

Korollar 5.18. *Falls der Allokationsgraph G zusammenhängend ist, so ist er ein Pseudobaum.*

Wir nennen einen Grapehn einen *Pseudowald*, wenn jede seiner Zusammenhangskomponenten ein Pseudobaum ist.

Lemma 5.19: Allokationsgraph ist Pseudowald

Der Allokationsgraph G ist ein Pseudowald.

Beweis. Es sei $G' = (V', E')$ eine beliebige Zusammenhangskomponente von G mit $V' = M' \cup J'$ für $M' \subseteq M$ und $J' \subseteq J$. Wir betrachten nun ein eingeschränktes Schedulingproblem, bei dem nur noch die Jobs aus J' und die Maschinen aus M' vorhanden sind. Wir bezeichnen mit LP'_{T^*} das zu LP_{T^*} analoge LP, das sich für die eingeschränkte Instanz ergibt. Außerdem bezeichnen wir mit x' die Einschränkung der Lösung x auf die Variablen, die in LP'_{T^*} noch auftreten. Dann ist x' eine zulässige Lösung von LP'_{T^*}.

Wir zeigen nun, dass x' sogar eine zulässige Basislösung von LP'_{T^*} ist: Angenommen, dies ist nicht der Fall. Dann gibt es $y' \neq 0$, sodass $x'+y'$ und $x'-y'$ zulässige Lösungen von LP'_{T^*} sind. Wir erweitern y' zu einem $y \in \mathbb{R}^{nm}$, indem wir $y_{ij} = 0$ für alle $i \notin M'$ oder $j \notin J'$ setzen. Dann sind $x+y$ und $x-y$ zulässige Lösungen von LP_{T^*} und x ist keine zulässige Basislösung von LP_{T^*}. ↯

Da x' eine zulässige Basislösung von LP'_{T^*} ist und G' zusammenhängend ist, ist er laut Korollar 5.18 ein Pseudobaum. □

Mithilfe des Allokationsgraphen runden wir die Lösung x nun in eine ganzzahlinge Lösung $\hat{x}$. Für einen Job $j \in J$, der bereits vollständig einer Maschine $i \in M$ zugewiesen ist (für den also $x_{ij} = 1$ gilt), übernehmen wir die Zuweisung in $\hat{x}$. Solange es noch einen solchen Job gibt, entfernen wir ihn und die entsprechende Kante aus dem Allokationsgraphen. Den Graphen, der danach übrig bleibt, nenenn wir H. Die Knotenmenge von H ist $J' \cup M$, wobei $J' \subseteq J$ die Menge der Jobs bezeichnet, die in der Lösung x auf mehr als eine Maschien verteilt werden.

In H berechnen wir ein einseitig perfektes Matching, d. h. eine Kantenmenge N, sodass jeder Job aus J' genau eine inzidente Kante in N und jede Maschine aus M höchstens eine inzidente Kante in N besitzt.

Lemma 5.20: H hat einseitig perfektes Matching

Der Graph H besitzt ein einseitiges perfektes Matching, welches in polynomieller Zeit gefunden werden kann.

Beweis. Zunächst entferne alle isolierten Maschinenknoten aus H. Nun sind alle Blätter im Pseudowald H Maschinenknoten, da Jobknoten mit Grad 1 bereits bei der Erstellung von H entfernt wurden. Solange es in H noch ein Blatt gibt, führe folgende Operation aus: Wähle ein beliebiges Blatt $i \in M$ aus H aus. Dieses besitzt genau einen adjazenten Knoten $j \in J$. Weise Job j Maschine i zu, füge also Kante (i, j) zu N hinzu. Anschließend entferne Knoten i und j und ihre inzidenten Kanten aus H. Lösche anschließend alle isolierten Maschinenknoten.

Werden aus einem Pseudobaum iterativ alle Blätter auf diese Art und Weise entfernt, so bleibt am Ende ein leerer Graph oder ein Kreis übrig. Da es bei H um einen bipartiten Pseudowald handelt, verbleibt somit am Ende eine (möglicherweise leere) Menge an Kreisen gerader Länge. Von jedem dieser Kreise nehmen wir jede zweite Kante zu N hinzul Durch dieses Verfahren stellen wir sicher, dass jeder Knoten aus J' genau eine inzidente Kante und jeder Knoten aus M höchstens eine inzidente Kante erhält. □

Das einseitig perfekte Matching N bildet zusammen mit den bereits in x festgelegten Jobs die ganzzahlige Lösung $\hat{x}$.

Satz 5.21: 2-Approximationsalgorithmus für Scheduling auf allgemeinen Maschinen

Der Makespan von $\hat{x}$ beträgt höchstens $2T^*$.

Beweis. Auf jeder Maschine $i \in M$ verursachen die Zuweisungen in x eine Ausführungszeit von höchstens T^*. Insbesondere beträgt die Gesamtausführungszeit aller Jobs, die in der Lösung x komplett Maschine i zugewiesen sind, höchstens T^*. All diese Jobs erhält Maschine i auch in der Lösung $\hat{x}$. Zusätzlich wird ihr beim Runden der fraktional zugewiesenen Jobs wegen Lemma 5.20 höchstens ein weiterer Job j zugewiesen. Für diesen Job j gilt, sofern er existiert, dass die Kante (i, j) zum Graphen G gehört. Demnach gilt $x_{ij} > 0$ und somit $p_{ij} \leq T^*$. Insgesamt ergibt sich damit für jede Maschine $i \in M$ eine Ausführungszeit von höchstens $2T^*$. □

[12.11.2025, Vorlesung 10]
[18.11.2025, Vorlesung 11]

Kapitel 6

Semidefinite Programmierung und Runden

Ein *Vektorprogramm* besitzt n Variablen $z_i \in \mathbb{R}^n$, jeweils also n-dimensionale, reelle Vektoren. Sowohl Zielfunktion als auch Nebenbedingungen sind linear in den Skalarprodukten dieser Vektoren. Ein VP mit m Nebenbedingungen lässt sich also wie folgt schreiben:

$$\begin{aligned} \max / \min \quad & \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij}(z_i \cdot z_j) & \\ s.t. \quad & \sum_{i=1}^{n}\sum_{j=1}^{n} a_{ijk}(z_i \cdot z_j) = b_k, & \forall k \in \{1,\dots,m\} \\ & z_i \in \mathbb{R}^n, & \forall i \in \{1,\dots,n\}. \end{aligned} \tag{6.1}$$

6.1 Semidefinite Programmierung

Ein bisschen LA-Wiederholung: Für eine Matrix $M \in \mathbb{R}^{m\times n}$ bezeichne $M^\top \in \mathbb{R}^{n\times m}$ die transponierte Matrix. M heißt *symmetrisch*, wenn $M = M^\top$ gilt. Alle Vektoren $v \in \mathbb{R}^n$ fassen wir im Folgenden als Spaltenvektoren auf, d. h. $v^\top v$ entspricht dem Skalarprodukt von v mit sich selbst und $vv^\top$ ist eine $(n \times n)$-Matrix.

Definition 6.1: Positiv semidefinit

Eine Matrix $M \in \mathbb{R}^{n\times n}$ heißt *positiv semidefinit*, wenn $x^\top Mx \geq 0$ für alle $x \in \mathbb{R}^n$ gilt. Wir schreiben dann $M \succeq 0$.

Folgende Aussage kann mit LA gezeigt werden:

Lemma 6.2: Eigenschaften positiv semidefiniter Matrizen

Für eine symmetrische Matrix $M \in \mathbb{R}^{n\times n}$ sind folgende Aussagen äquivalent:

(*i*) *M ist positiv semidefinit,*

(*ii*) *M hat keine negativen Eigenwerte,*

(*iii*) *Es gilt $M = V^\top V$ für eine Matrix $V \in \mathbb{R}^{n\times n}$.*

Wir lernen nun *semidefinite Programme*, eine Art von nichtlinearen Programmen kennen. Die Variablen eines solchen Programms bezeichnen wir mit x_{ij} für $i, j \in \{1,\dots,n\}$ und wir fordern $x_{ij} = x_{ji}$. Die Zielfunktion und Nebenbedingungen sind linear in diesen Variablen. Der einzige Unterschied zu einem LP ist eine zusätzliche Nebenbedingung, die nur solche Variablenbelegun-

gen zulässt, für die die Matrix $X = (x_{ij})$ positiv semidefinit aus. Es hat also die Form

$$\begin{aligned} \max / \min \quad & \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij}x_{ij} & & \\ s.t. \quad & \sum_{i=1}^{n}\sum_{j=1}^{n} a_{ijk}x_{ij} = b_k, & \forall k \in \{1,\dots,m\}, & \\ & x_{ij} = x_{ji}, & \forall i,j \in \{1,\dots,n\}, & \\ & X = (x_{ij}) \succeq 0. & & \end{aligned} \tag{6.2}$$

Lemma 6.3: Äquivalenz von Vektor- und semidefiniten Programmen

Das Vektorprogramm (6.1) *und das semidefinite Programm* (6.2) *sind in dem Sinne äquivalent, das eine Lösung für das eine in polynomieller Zeit in eine Lösung des anderen mit demselben Zielfunktionswert überführt werden kann und umgekehrt. Insbesondere können damit optimale Lösungen ineinander überführt werden.*

Beweis. Betrachte eine Lösung $z = (z_1, \dots, z_n)$ für das VP (6.1). Wir konstruieren eine Matrix $Z \in \mathbb{R}^{n\times n}$, wobei Spalte i durch den Vektor z_i gegeben ist. Es sei $X = (x_{ij}) = Z^\top Z \in \mathbb{R}^{n\times n}$. Dann gilt $x_{ij} = z_i \cdot z_j = x_{ji}$ und somit ist X symmetrisch. Mit Lemma 6.2 folgt dann, dass X positiv semidefinit ist. Die Matrix X ist somit eine zulässige Lösung für das SDP (6.2) und besitzt denselben Zielfunktionswert wie die Lösung z im VP.

Nun betrachten wir eine Lösung $X = (x_{ij})$ für das SDP. Um hieraus eine Lösung für das VP zu konstruieren, berechne eine Cholesky-Zerlegung von X, also eine Matrix Z mit $X = Z^\top Z$. Diese kann allgemein nicht in polynomieller Zeit berechnet werden, da die Einträge von Z irrational sein können. Es ist jedoch möglich, die Einträge in polynomieller Zeit mit einem kleinen, additiven Fehler zu approximieren. Da dieser Fehler beliebig klein gemacht werden kann, gehen wir nicht weiter darauf ein und nehmen an, dass die Zerlegung exakt berechnet wurde. Bezeichnen wir mit z_i idie i-te Spalte von Z, so ist $z = (z_1, \dots, z_n)$ eine zulässige Lösung für das VP mit demselben Zielfunktionswert wie die Lösung X des SDP, denn es gilt $x_{ij} = z_i \cdot z_j$. □

SDPs können im Allgemeinen nicht in polynomieller Zeit optimal gelöst werden, da die optimale Lösung irrational sein kann. Ein weiteres Problem ist, dass es SDPs gibt, in denen alle Koeffizienten kleine Zahlen sind, der Wert der optimalen Lösung jedoch doppelt exponentiell von der Anzahl der Variablen abhängt. In den für uns relevanten SDPs tritt dieses Problem jedoch nicht auf, und mit der Ellipsoidmethode erhält man folgenden Satz:

Satz 6.4: Fehlerabschätzung optimale Lösung von SDPs

Die optimale Lösung eines semidefiniten Programms kann für jedes $\varepsilon > 0$ mit einem additiven Fehler von höchstens ε berechnet werden. Die Laufzeit hierfür ist polynomiell in der Eingabegröße und $\log\left(\frac{R}{\varepsilon}\right)$ beschränkt, sofern die Menge der zulässigen Lösung nichtleer und im Hyperwürfel $[-R, R]^n$ enthalten ist.

Mit Lemma 6.3 überträgt sich dies auch auf Vektorprogramme.

6.2 Maximum-Cut Problem

Wir kommen zurück zum Maximum-Cut Problem, und zwar auf seine (relaxierte) Darstellung als Vektorprogramm:

$$\begin{aligned} \max \quad & \frac{1}{2}\sum_{i=1}^{n-1}\sum_{j=i+1}^{n} w_{ij}(1 - z_i \cdot z_j) & \\ s.t. \quad & z_i \cdot z_i = 1, & \forall i \in \{1,\dots,n\}, \\ & z_i \in \mathbb{R}^n, & \forall i \in \{1,\dots,n\} \end{aligned} \tag{6.3}$$

Mithilfe von Satz 6.4 können wir in polynomieller Zeit die optimale Lösung dieser Relaxierung mit einem beliebg kleinen, addtiven Fehler berechnen. Wir ignorieren diesen additiven Fehler und gehen davon aus, dass wir die optimale Lösung $z = (z_1, \dots, z_n)$ berechnet haben und bezeichnen mit W^* den Wert dieser Lösung. Da (6.3) eine Relaxierung des ganzzahligen quadratischen Problems ist, gilt $W^* \geq \text{OPT}$, wobei OPT den Wert des optimalen Schnittes berechnet.

Das Ziel ist es nun, die optimale Lösung des VPs zu einer ganzzahligen Lösung zu runden, sodass sich die Zielfunktion nur um einen konstanten Faktor verschlechtert. Das Rundungsverfahren, das wir einsetzen werden, ist randomisiert, und es basiert auf der folgenden Beobachtung: Der Beitrag der Kante zwischen den Knoten v_i und v_j zur optimalen Lösung des VPs beträge

$$\frac{w_{ij}}{2}(1 - z_i \cdot z_j) = \frac{w_{ij}}{2}(1 - \|z_i\|\|z_j\|\cos(\theta_{ij})) = \frac{w_{ij}}{2}(1 - \cos(\theta_{ij})),$$

wobei θ_{ij} den Winkel zwischen den Vektoren z_i und z_j bezeichnet. Der Beitrag steigt in dem relevanten Intervall $[0, \pi]$ für θ_{ij} also streng monoton von 0 (für $\theta_{ij} = 0$) auf w_{ij} (für $\theta_{ij} = \pi$).Damit wir durch das Runden nicht allzu viel verlieren, sollte die Wahrscheinlichkeit, dass v_i und v_j in der gerundeten Lösung durch den Schnitt getrennt werden, mit dem Winkel θ_{ij} ansteigen: Je größer θ_{ij} ist, desto größer sollte die Wahrscheinlichkeit sein, dass v_i und v_j voneinander getrennt werden.

Das gewünschte Verfahren erhalten wir auf einfache Art und Weise: Die Vektoren z_i liegen aufgrund der Nebenbedingungen $z_i \cdot z_i = 1$ auf der Einheitssphäre $\mathbb{S}^{n-1} = \{x \in \mathbb{R}^n \mid \|x\| = 1\}$. Wir teilen diese Sphäre nun in zwei Hälften, indem wir eine uniform zufällige Hyperebene durch den Mittelpunkt von $\mathbb{S}^{n-1}$, also den Koordinatenursprung, legen. Diese Hyperebene partitioniert die Menge der Vektoren z_i und damit auch die Menge der Knoten v_i in zwei Teile:

Algorithmus 6.5 SDPMAXCUT

Input: Wie in Problem 4.1
Output: Wie in Problem 4.1

1: Löse das Vektorprogramm (6.3)
2: Wähle $r \in \mathbb{S}^{n-1}$ uniform zufällig
3: Setze $U = \{v_i \mid z_i \cdot r \geq 0\}$
4: **return** $(U, V \setminus U)$

Wir müssen uns noch überlegen, wie ein uniform zufälliger Vektor $r \in \mathbb{S}^{n-1}$ effizient ausgewählt werden. Dazu erzeugen wir zunächst n Zahlen $\hat{r}_1, \dots, \hat{r}_n$ unabhängig zufällig gemäß einer Standardnormalverteilung. Streng genommen ist dies natürlich nicht in endlicher Zeit nötig, da eine standardnormalverteilte Zahl mit Wahrscheinlichkeit 1 irrational ist. Bei einer Implementierung muss man sich also mit einer Approximation begnügen und die Zahlen $\hat{r}_1, \dots, \hat{r}_n$ nach endlich vielen Dezimalstellen abschneiden. Standardnormalverteilte Zufallsvariablen, die nach einer endlichen Anzahl an Stellen abgeschnitten sind, können effizient erzeugt werden. Wir ignorieren im Folgenden den Einfluss dieser Approximation, da er beliebig klein gemacht werden kann, und gehen der Einfachheit halber davon aus, dass es sich bei $\hat{r}_1, \dots, \hat{r}_n$ um nicht-abgeschnittene standardnormalverteilte Zufallsvariablen handelt. Es sei dann $\hat{r} = (\hat{r}_1, \dots, \hat{r}_n)$ und $r = \frac{\hat{r}}{\|\hat{r}\|}$.

Lemma 6.6: Uniformverteilung auf $\mathbb{S}^{n-1}$

Der Vektor r ist uniform zufällig auf der Sphäre $\mathbb{S}^{n-1}$ verteilt.

Beweis. Aus der Definition der Standardnormalverteilung und der Tatsache, dass die Zufallsvariablen $\hat{r}_1, \dots, \hat{r}_n$ unanbhängig sind, ergibt sich, dass die Dichte des Vektors $\hat{r}$ an der Stelle $(x_1, \dots, x_n) \in \mathbb{R}^n$

$$\prod_{i=1}^{n} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x_i^2}{2}\right) = \frac{1}{(2\pi)^{\frac{n}{2}}} \exp\left(-\frac{1}{2}\sum_{i=1}^{n} x_i^2\right)$$

beträgt. Diese Dichte hängt nur davon ab, wie weit der Punkt $(x_1, \dots, x_n)$ vom Koordinatenursprung entfernt ist, nicht aber von seiner genauen Position. Demzufolge ist die Dichte von $\hat{r}$ radialsymmetrisch. Der Vektor $r = \frac{\hat{r}}{\|\hat{r}\|}$ hat dieselbe Richtung wie der Vektor $\hat{r}$. Aus der Radialsymmetrie folgt somit das Lemma. □

[18.11.2025, Vorlesung 11]
[19.11.2025, Vorlesung 12]

Eine wichtige Eigenschaft von Zufallsvektoren ist, dass Projektionen auf Unterräume wieder normalverteilt sind: Seien $e_1, e_2 \in \mathbb{R}^n$ zwei orthogonale Einheitsvektoren. Dann sind die Projektionen von $\hat{r}$ auf e_1 und e_2 unabhängige, standardnormalverteilte Zufallsvariablen. Es bezeichne r' die Projektion von $\hat{r}$ auf die von e_1 und e_2 aufgespannte Ebene. Da die Projektionen von $\hat{r}$ auf e_1 und e_2 unabhängige standardnormalverteilte Zufallsvariablen sind, folgt mit Lemma 6.6, dass der Vektor $\frac{r'}{\|r'\|}$ uniform zufällig auf der Sphäre $\mathbb{S}^1$, also dem Rand des Einheitskreises, in der durch e_1 und e_2 aufgespannten Ebene verteilt ist.

Lemma 6.7: Wahrscheinlichkeit, dass Knoten getrent werden

Für jedes Paar v_i und v_j von Knoten gilt

$$\mathbb{P}[v_i \text{ und } v_j \text{ werden durch den Schnitt } (U, V \setminus U) \text{ getrennt}] = \frac{\theta_{ij}}{\pi}.$$

Beweis. Für $z_i = z_j$ beträgt die Wahrscheinlichkeit, dass v_i und v_j getrennt werden, gleich null. Die Aussage ist also wahr.

Sei nun $z_i \neq z_j$. Wir projizieren den Vektor $\hat{r}$ auf die durch z_i und z_j aufgespannte Ebene. Diese Projektion heiße r'. Wie oben ist $\frac{r'}{\|r'\|}$ ein uniform zufälliger Einheitsvektor in dieser Ebene. Um zu bestimmen, wann v_i und v_j getrennt werden, beachte folgendes: Wir zeichnen zu den Vektoren z_i und z_j jeweils eine orthogonale Gerade durch den Koordinatenursprung. Die zu z_i orthogonale Gerade berühre den Einheitskreis in den Punkten A und C, die zu z_j orthogonale Gerade in den Punkten B und D. Durch diesse Punkte wird der Einheitskreis in vier Sektoren eingeteilt.

Die Knoten v_i und v_j werden genau dann durch den Schnitt getrennt, wenn der Vektor $\frac{r'}{\|r'\|}$ in den Kreisvektor zwischen A und B oder zwischen C und D fällt. Wir gehen dabei oBdA davon aus, dass von z_i ausgehend der Abstand zu z_j im Uhrzeigersinn mindestens so groß ist wie gegen den Uhrzeigersinn.

Sei $r'' = \frac{r'}{\|r'\|}$. Die Wahrscheinlichkeit, dass r'' exakt einen der Punkte A, B, C oder D trifft, ist 0. Wir können davon ausgehen, dass dieses Ereignis nicht eintritt.

- Fällt r'' in den Kreisvektor AB, so gilt $r' \cdot z_i < 0$ und $z' \cdot z_j > 0$.
- Fällt r'' in den Kreisvektor BC, so gilt $r' \cdot z_i < 0$ und $z' \cdot z_j < 0$.
- Fällt r'' in den Kreisvektor CD, so gilt $r' \cdot z_i > 0$ und $z' \cdot z_j < 0$.
- Fällt r'' in den Kreisvektor DA, so gilt $r' \cdot z_i > 0$ und $z' \cdot z_j > 0$.

Die Knoten v_i und v_j werden durch den Schnitt also genau dann getrennt, wenn r'' entweder in Sektor AB oder CD fällt. Beide haben einen Mittelpunktswinkel von θ_{ij}. Da r'' uniform zufällig auf dem Einheitskreis verteilt ist, beträgt die Wahrscheinlichkeit, dass es in einem der beiden Kreissektoren fällt, somit $\frac{2\theta_{ij}}{2\pi} = \frac{\theta_{ij}}{\pi}$. □

Wir haben bereits $W^* \geq \text{OPT}$ als den optimalen Wert des VPs eingeführt. Wir bezeichnen nun mit W das Gesamtgewicht des Schnittes, den SDPMaxCut berechnet. Sei außerdem

$$\alpha = \frac{2}{\pi} \cdot \min_{0<\theta\leq\pi} \frac{\theta}{1-\cos(\theta)}. \tag{6.4}$$

Lemma 6.8: Abschätzung Erwartungswert SDPMaxCut

Es gilt $\mathbb{E}[W] \geq \alpha W^* \geq \alpha\,\text{OPT}$.

Beweis. Mit Linearität des Erwartungswertes folgt aus Lemma 6.7

$$\begin{aligned}\mathbb{E}[W] &= \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} w_{ij} \cdot \mathbb{P}[v_i \text{ und } v_j \text{ werden durch den Schnitt } (U, V\setminus U) \text{ getrennt}] \\ &= \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} w_{ij} \cdot \frac{\theta_{ij}}{\pi}. \end{aligned} \tag{6.5}$$

Aus der Definition von α folgt

$$\theta_{ij} \geq \alpha \cdot \frac{\pi}{2}(1-\cos(\theta_{ij})).$$

Einsetzen in (6.5) liefert

$$\mathbb{E}[W] \geq \alpha \cdot \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} \frac{w_{ij}}{2}(1-\cos(\theta_{ij})) = \alpha W^* \geq \alpha\,\text{OPT}\,.$$ □

Lemma 6.9: Abschätzung α

Es gilt $\alpha \geq 0.878$.

Beweis. Wir verwenden Kurvendiskussion. Die Funktion $\frac{\theta}{1-\cos(\theta)}$ nimmt ihr Minimum im Intervall $[0,\pi]$ an der Stelle $\theta \approx 2.3311$ an. Für diesen Wert von θ gilt

$$\frac{2}{\pi} \cdot \frac{\theta}{1-\cos(\theta)} \geq 0.878.$$ □

Damit ergibt sich folgendes Resultat:

Satz 6.10: SDPMaxCut

SDPMaxCut ist ein 0.878*-Approximationsalgorithmus für das Maximum-Cut Problem.*

6.3 Max-2SAT

Wir betrachten nun den Spezialfall von Max-SAT, bei dem jede Klausel nur höchstens zwei Literale enthalten darf. Diesen bezeichnen wir mit Max-2SAT.

Man kann zwar in polynomieller Zeit testen, ob eine gegebene Formel eine Belegung der Variablen besitzt, die alle Klauseln erfüllt. Es ist aber NP-schwer, eine Belegung zu finden, die für eine gegebene Formel so viele Klauseln wie möglich erfüllt. Sind die Klauseln gewichtet, ist es ebenfalls NP-schwer, eine Belegung der Variablen zu finden, sodass das Gesamtgewicht der erfüllten Klauseln Maximal wird. In Abschnitt 5.1.2 für MAX-SAT einen $\frac{3}{4}$-Approximationsalgorithmus kennengelernt. Mit SDP können wir für MAX-2SAT aber einen besseren Approximationsfaktor zeigen.

Sei im Folgenden φ eine Eingabe für MAX-2SAT mit Variablen $x_1, \dots, x_n$, Klauseln $C_1, \dots, C_m$ und Gewichten $w_1, \dots, w_m$. Zunächst formulieren wir ein quadratisches Programm. Dazu führen wir Variablen $y_0, y_1, \dots, y_n$ ein, die nur die Werte -1 oder $+1$ annehmen können. Die Idee ist die folgende: Die Aussagenvariable x_i wird genau dann auf 1 gesetzt, wenn $y_i = y_0$ gilt. Wir definieren den Wert $v(C_j)$ der Klausel C_j als 1, wenn die Belegung der Aussagenvariablen, die durch $y_0, y_1, \dots, y_n$ codiert wird, die Klausel erfüllt, und sonst als 0. Für Klauseln der Länge 1 gilt

$$v(x_i) = \frac{1 + y_0 y_i}{2} \quad \text{und} \quad v(\neg x_i) = \frac{1 - y_0 y_i}{2}.$$

Betrachte nun eine Klausel der Länge 2 mit zwei positiven Literalen x_i und x_j. Dann:

$$\begin{aligned} v(x_i \vee x_j) &= 1 - v(\neg x_i)v(\neg x_j) = 1 - \frac{1 - y_0 y_i}{2} \cdot \frac{1 - y_0 y_j}{2} \\ &= 1 - \frac{1 - y_0 y_i - y_0 y_j + y_0^2 y_i y_j}{4} \\ &= \frac{3}{4} + \frac{y_0 y_i + y_0 y_j - y_i y_j}{4} \\ &= \frac{1 + y0 y_i}{4} + \frac{1 + y_0 y_j}{4} + \frac{1 - y_i y_j}{4}, \end{aligned}$$

da $y_0^2 = 1$. Somit können wir den Wert $v(x_i \vee x_j)$ als Linearkombination von Termen der Form $(1 + y_a y_b)$ und $(1 - y_a y_b)$ für $a, b \in \{0, 1 \dots, n\}$ darstellen. Das gilt auch, wenn die Klausel der Länge zwei mindestens ein negatives Literal enthält.

Die Zielfunktion, die wir maximieren möchten, ist $\sum_{j=1}^{m} w_j v(C_j)$. Da sich die Werte aller Klauseln als Linearkombination von Termen der Form $(1 + y_a y_b)$ und $(1 - y_a y_b)$ für $a, b \in \{0, 1 \dots, n\}$ darstellen lassen, können wir geeignete Koeffizienten a_{ij} und b_{ij} finden, sodass

$$\sum_{j=1}^{m} w_j v(C_j) = \sum_{i=0}^{n-1} \sum_{j=i+1}^{n} (a_{ij}(1 + y_i y_j) + b_{ij}(1 - y_i y_j))$$

gilt. Wir erhalten damit das quadratische Programm

$$\begin{aligned} \max \quad & \sum_{i=0}^{n-1} \sum_{j=i+1}^{n} (a_{ij}(1 + y_i y_j) + b_{ij}(1 - y_i y_j)) \\ s.t. \quad & y_i^2 = 1, && \forall i \in \{0, 1, \dots, n\}, \\ & y_i \in \mathbb{Z}, && \forall i \in \{0, 1, \dots, n\}. \end{aligned}$$

Auch hier betrachten wir wieder die Relaxierung als VP:

$$\begin{aligned} \max \quad & \sum_{i=0}^{n-1} \sum_{j=i+1}^{n} (a_{ij}(1 + z_i \cdot z_j) + b_{ij}(1 - z_i \cdot z_j)) \\ s.t. \quad & z_i \cdot z_i = 1, && \forall i \in \{0, 1, \dots, n\}, \\ & z_i \in \mathbb{R}^{n+1}, && \forall i \in \{0, 1, \dots, n\}. \end{aligned} \tag{6.6}$$

Der Approximationsalgorithmus SDP2SAT, den wir für MAX-2SAT betrachten, arbeitet voll-

kommen analog zu SDPMaxCut:

Algorithmus 6.11 SDP2SAT

Input: Wie in Problem 4.24, alle Klauseln haben jedoch Länge 2
Output: Wie in Problem 4.24

1: Löse das Vektorprogramm (6.6). Die optimale Lösung sei W^*
2: Wähle $r \in \mathbb{S}^n$ uniform zufällig
3: **for** $i \in \{0, \dots, n\}$ **do**
4: **if** $r \cdot z_i \geq 0$ **then** Setze $y_i = +1$
5: **else** Setze $y_i = -1$
6: **return** W als Gesamtgewicht aller erfüllten Klauseln

Lemma 6.12: Abschätzung Erwartungswert SDP2SAT

Es gilt $\mathbb{E}[W] \geq \alpha W^*$ für den Wert α aus (6.4).

Beweis. Mit Lineraität des Erwartungswert gilt

$$\mathbb{E}[W] = 2 \sum_{i=0}^{n-1} \sum_{j=i+1}^{n} (a_{ij} \cdot \mathbb{P}[y_i = y_j] + b_{ij} \cdot \mathbb{P}[y_i \neq y_j]).$$

Wir bezeichnen wieder mit θ_{ij} den Winkel zwischen z_i und z_j. Dann folgt analog zur Analyse von SDPMaxCut für alle $i, j \in \{0, \dots, n\}$

$$\mathbb{P}[y_i \neq y_j] = \frac{\theta_{ij}}{\pi} \geq \frac{\alpha}{2}(1 - \cos(\theta_{ij})).$$

Ebenfalls gilt

$$\mathbb{P}[y_i = y_j] = 1 - \frac{\theta_{ij}}{\pi} \geq \frac{\alpha}{2}(1 + \cos(\theta_{ij})).$$

Nutzen wir genau wie bei der SDPMaxCut-Analyse aus, dass $z_i \cdot z_j = \cos(\theta_{ij})$ gilt, so erhalten wir

$$\mathbb{E}[W] \geq \alpha \sum_{i=0}^{n-1} \sum_{j=i+1}^{n} (a_{ij} \cdot (1 + \cos(\theta_{ij})) + b_{ij} \cdot (1 - \cos(\theta_{ij}))) = \alpha W^*. \qquad \square$$

Zusammengefasst ergibt sich also Folgendes:

Satz 6.13: SDP2SAT

SDP2SAT ist ein 0.878-Approximationsalgorithmus für Max-2SAT.

[19.11.2025, Vorlesung 12]
[25.11.2025, Vorlesung 13]

Kapitel 8

Primal-Duale Algorithmen

8.1 Duale Lineare Programme

Wir betrachten folgendes LP:

$$\begin{array}{llrcrcrcl} \min & f(x) = & 4x_1 & + & 7x_2 & + & 2x_3 & & \\ s.t. & (y_1) & 3x_1 & + & 7x_2 & + & x_3 & \geq & 10 \\ & (y_2) & 2x_1 & + & 5x_2 & + & 3x_3 & \geq & 6 \\ & (y_3) & 2x_1 & + & x_2 & - & x_3 & \geq & 5 \\ & (y_4) & 4x_1 & + & x_2 & + & 3x_3 & \geq & 20 \\ & (y_5) & 4x_1 & + & 13x_2 & + & x_3 & \geq & 4 \\ & (y_6) & x_1, & & x_2, & & x_3 & \geq & 0 \end{array} \tag{8.1}$$

Wir suchen eine Möglichkeit, die Zielfunktion nach unten abzuschätzen. Beispielsweise gilt

$$\begin{aligned} f(x) &= 4x_1 + 7x_2 + 2x_3 \\ &\geq 3x_1 + 7x_2 + x_3 \geq 10, \end{aligned}$$

da alle x_i nichtnegativ sind. Somit hat die optimale Lösung keinen kleineren Wert als 10. Wir erhalten durch Addition der zweiten und dritten Schranke eine bessere Abschätzung:

$$\begin{aligned} f(x) &= 4x_1 + 7x_2 + 2x_3 \\ &\geq 4x_1 + 6x_2 + 2x_3 \\ &= (2x_1 + 5x_2 + 3x_3) + (2x_1 + 5x_2 - x_3) \\ &\geq 6 + 5 = 11. \end{aligned}$$

Eine bessere Schranke erhalten wir durch Gewichtung der dritten und vierten Ungleichung jeweils mit $\frac{1}{2}$:

$$\begin{aligned} f(x) &= 4x_1 + 7x_2 + 2x_3 \\ &\geq \frac{1}{2}(2x_1 + x_2 - x_3) + \frac{1}{2}(4x_1 + x_2 + 3x_3) \\ &\geq \frac{1}{2}(5 + 20) = 12.5 \end{aligned}$$

Sei x^* eine optimale Lösung. *Ziel:* Finde eine untere Schranke für $f(x^*)$. Wir setzen nun Variablen y_i ein (eine pro Gleichung), die alle nichtnegativ sein dürfen und jeweils darstellen, wie oft wir eine Zeile mitnehmen (siene (8.1)). Damit erhalten wir folgendes *Dualsystem*:

$$\begin{array}{lrcrcrcrcrcl} \max & 10y_1 & + & 6y_2 & + & 5y_3 & + & 20y_4 & + & 4y_5 & & \\ s.t. & 3y_1 & + & 2y_2 & + & 2y_3 & + & 4y_4 & + & 4y_5 & \leq & 4 \\ & 7y_1 & + & 5y_2 & + & y_3 & + & y_4 & + & 13y_5 & \leq & 7 \\ & y_1 & + & 3y_2 & - & y_3 & + & 3y_4 & + & y_5 & \leq & 2 \\ & 3y_1 & + & 2y_2 & + & 2y_3 & + & 4y_4 & + & 4y_5 & \leq & 4 \\ & y_1, & & y_2, & & y_3, & & y_4, & & y_5 & \geq & 0 \end{array} \tag{8.2}$$

Wir lösen (8.2) und erhalten $y^* = \left(0, 0, \frac{2}{5}, \frac{4}{5}, 0\right)$ und erhalten durch Gewichtung der Gleichungen tatsächlich $f(x) \geq \frac{2}{5} \cdot 5 + \frac{4}{5} \cdot 20 = 18$.

Wir verallgemeinern dies nun auf allgemeine lineare Programme. Wir können LPs schreiben als

$$\begin{aligned} \min \quad & f(x) = \sum_{j=1}^{n} c_j x_j & & \\ s.t. \quad & \sum_{j=1}^{n} a_{ij} x_j \geq b_i, & & \forall i \in \{1, \dots, m\}, \\ & x_j \geq 0, & & \forall j \in \{1, \dots, n\}. \end{aligned} \tag{8.3}$$

Wie oben erhalten wir im dualen LP eine Nebenbedingung pro Variable und eine Variable pro Nebenbedingung. Wir erhalten als Duales von (8.3)

$$\begin{aligned} \max \quad & g(y) = \sum_{i=1}^{m} b_i y_i & & \\ s.t. \quad & \sum_{i=1}^{1} a_{ij} y_i \leq c_j, & & \forall j \in \{1, \dots, n\}, \\ & y_i \geq 0, & & \forall i \in \{1, \dots, m\}. \end{aligned} \tag{8.4}$$

Satz 8.1: Schwache Dualität

Sei $x = (x_1, \dots, x_n)$ eine Lösung für das primale LP (8.3) und $y = (y_1, \dots, y_n)$ eine Lösung für das duale LP (8.4). Dann gilt $f(x) \geq g(y)$.

Beweis. Es gilt

$$f(x) = \sum_{j=1}^{n} c_j x_j \geq \sum_{j=1}^{n} \left(\sum_{i=1}^{m} a_{ij} y_i \right) x_j = \sum_{i=1}^{m} \left(\sum_{j=1}^{n} a_{ij} x_j \right) y_i \geq \sum_{i=1}^{m} b_i y_i = g(y), \tag{8.5}$$

wobei die Ungleichungen folgen, da y eine Lösung für (8.4) und x eine Lösung für (8.3) ist. □

Für die Analyse von Approximationsalgorithmen reicht uns tatsächlich schon Satz 8.1. Wir können aber auch noch die folgende stärkere Aussage formulieren:

Satz 8.2: Starke Dualität

Das primale LP (8.3) hat genau dann einen beschränkten, optimalen Wert wenn dies für das duale LP (8.4) der Fall ist.
Sei x^* eine optimale Lösung für das primale LP und y^* eine optimale Lösung für das duale LP. Dann gilt $f(x^*) = g(y^*)$.

8.2 Set Cover

Wir erinnern uns zurück an Set Cover aus Abschnitt 2.2.

Problem 8.3: Set Cover (Wiederholung)

Input: Menge S mit n Elementen, n Teilmengen $S_1, \dots, S_m \subseteq S$ mit $\bigcup_{i=1}^{m} S_i = S$, Kosten $c_1, \dots, c_m \in \mathbb{N}$.
Output: Alle $A \subseteq \{1, \dots, m\}$ mit $\bigcup_{i \in A} S_i = S$
Objective: Minimiere $\sum_{i \in A} c_i$

Formuliert als ILP erhalten wir

$$\begin{aligned} \min \quad & \sum_{j=1}^{m} w_j x_j \\ s.t. \quad & \sum_{j : i \in S_j} x_j \geq 1, \quad \forall i \in \{1, \dots, n\}, \\ & x_j \in \{0, 1\}, \quad \forall j \in \{1, \dots, m\}, \end{aligned}$$

bzw. relaxiert

$$\begin{aligned} \min \quad & \sum_{j=1}^{m} w_j x_j \\ s.t. \quad & \sum_{j : i \in S_j} x_j \geq 1, \quad \forall i \in \{1, \dots, n\}, \\ & x_j \geq 0, \quad \forall j \in \{1, \dots, m\}. \end{aligned} \tag{8.6}$$

Dualisieren ergibt

$$\begin{aligned} \max \quad & \sum_{i=1}^{n} y_i \\ s.t. \quad & \sum_{i \in S_j} y_i \leq w_j, \quad \forall j \in \{1, \dots, m\}, \\ & y_i \geq 0, \quad \forall i \in \{1, \dots, n\}. \end{aligned} \tag{8.7}$$

Algorithmus 8.4 PRIMALDUAL-SC

Input: Wie in Problem 8.3
Output: Wie in Problem 8.3

1: $y := 0$ ▷ *Gültige Lösung für* (8.7)
2: $I := \emptyset$
3: **while** $\exists\, i \notin \bigcup_{j \in I} S_j$ **do**
4: Erhöhe y_i solange, bis für ein j mit $i \in S_j$ gilt $\sum_{\ell \in S_j} y_\ell = w_\ell$
5: $I := I \cup \{j\}$
6: **return** I

Satz 8.5: PrimalDual-SC

Sei $f = \max_i |\{j \mid i \in S_j\}|$ die maximale Anzahl an Vorkommen eines Elements in den Teilmengen. PRIMALDUAL-SC ist ein f-Approximationsalgorithmus für Set Cover.

Beweis. Sei OPT der Wert eines optimalen Set Covers und Z^* der optimale Wert des primalen LP. Dann gilt $Z^* \leq \text{OPT}$.

Da y eine gültige Lösung im dualen LP ist, gilt

$$\sum_{i=1}^{n} y_i \leq Z^* \leq \mathrm{OPT}\,.$$

Damit genügt es zu zeigen, dass $\sum_{j\in I} w_j \leq f \cdot \sum_{i=1}^{n} y_i$. Es gilt

$$\sum_{j\in I} w_j = \sum_{j\in I}\left(\sum_{i\in S_j} y_i\right) = \sum_{i\in S} y_i \cdot |\{j \in I \mid i \in S_j\}| \leq f \cdot \sum_{i\in S} y_i \leq f \cdot \mathrm{OPT},$$

da j nur in I aufgenommen wird, wenn w_j mit Gleichheit erreicht wird. □

Algorithmen, die nach dem Schema von PrimalDual-SC ablaufen, nennt man *primal-duale Algorithmen*. Diese haben gemeinsam, dass sie stets eine gültige Lösung des dualen LPs mitführen und dabei eine (im Allgemeinen nicht optimale) ganzzahlige Lösung für das primale LP konstruieren. Dabei werden die Lösungen für das primale und duale LP vom Algorithmus explizit konstruiert, ohne auf allgemeine Algorithmen zur Lösung von linearen Programmen zurückzugreifen. Die Dualitätstheorie ist wesentlich für die Analyse der Algorithmen.

8.3 Feedback Vertex Set

Problem 8.6: Feedback Vertex Set

Input: ungerichteter Graph $G = (V, E)$, Gewichte $w : V \to \mathbb{R}_{>0}$
Output: $S \subseteq V$, sodass $G[V \setminus S]$ kreisfrei ist
Objective: Minimiere $\sum_{v\in S} w(v)$

Wir modellieren dies als ILP, wobei $\mathcal{C}$ die Menge aller Kreise in G sei. Dabei sei $w_v = w(v)$ für alle $v \in V$.

$$\begin{aligned} \min \quad & \sum_{v\in V} w_v x_v \\ s.t. \quad & \sum_{v\in C} x_v \geq 1, \quad \forall C \in \mathcal{C}, \\ & x_v \in \{0,1\}, \quad \forall v \in V, \end{aligned}$$

bzw. relaxiert

$$\begin{aligned} \min \quad & \sum_{v\in V} w_v x_v \\ s.t. \quad & \sum_{v\in C} x_v \geq 1, \quad \forall C \in \mathcal{C}, \\ & x_v \geq 0, \qquad \forall v \in V. \end{aligned} \tag{8.8}$$

Es könnte sein, dass wir exponentiell viele Nebenbedingungen haben. Das ist letztlich aber egal, siehe morgen!

[25.11.2025, Vorlesung 13]
[26.11.2025, Vorlesung 14]

Duailsiert erhalten wir

$$\begin{aligned} \max \quad & \sum_{C\in\mathcal{C}} y_C \\ s.t. \quad & \sum_{C: v\in C} y_C \leq w(v), \quad \forall v \in V, \\ & y_C \geq 0, \qquad \forall C \in \mathcal{C}. \end{aligned} \tag{8.9}$$

Sei wieder OPT der Wert einer optimalen Lösung für die gegebene Instanz von Feedback Vertex Set, Z^* ein optimaler Wert des dualen LP (8.9). Sei y eine zulässige Lösung für das duale LP. Dann gilt wieder

$$\text{OPT} \geq Z^* \geq \sum_{C \in \mathcal{C}} y_C.$$

Algorithmus 8.7 PrimalDual-FVS

Input: Wie in Problem 8.6
Output: Wie in Problem 8.6

1: $y := 0$ ▷ *Lösung von* (8.9)
2: $S := \emptyset$
3: **while** $\exists$ Kreis C^* in G **do**
4: Entferne rekursiv alle Knoten mit Grad 1
5: Erhöhe y_{C^*}, bis für ein $v \in C^*$ gilt, dass $\sum_{C: v \in C} y_C = w_v$
6: $S := S \cup \{v\}$
7: Entferne Knoten v und zu v inzidente Kanten aus G
8: **return** S

Es gilt

$$\sum_{v \in S} w_v = \sum_{v \in S} \left(\sum_{C: v \in C} y_C \right) = \sum_{C \in \mathcal{C}} y_C \cdot |S \cap C|$$

denn für jeden Knoten, der in S und C liegt, wird y_C einmal aufaddiert.

Angenommen, es ist $|S \cap C| \leq \alpha$ für ein α. Dann wäre $\sum_{v \in S} w_v \leq \alpha \sum_{c \in \mathcal{C}} y_C \leq \alpha \cdot \text{OPT}$. Leider ist das aber a priori nicht klar. Wir modifizieren bzw. konkretisieren, *wie* wir C^* auswählen:

Algorithmus 8.8 PrimalDual-FVS2

Input: Wie in Problem 8.6
Output: Wie in Problem 8.6

1: $y := 0$ ▷ *Lösung von* (8.9)
2: $S := \emptyset$
3: **while** $\exists$ Kreis C^* in G **do**
4: Entferne rekursiv alle Knoten mit Grad 1
5: Finde einen Kreis C^* mit höchstens $2 \cdot \lceil \log_2(n) \rceil$ Knoten mit Grad ≥ 3.
6: Erhöhe y_{C^*}, bis für ein $v \in C^*$ gilt, dass $\sum_{C: v \in C} y_C = w_v$
7: $S := S \cup \{v\}$
8: Entferne Knoten v und zu v inzidente Kanten aus G
9: **return** S

Hier bleibt die Frage, warum dieser Kreis existieren muss.

Lemma 8.9: Graph hat Kreis mit $\leq 2\lceil \log_2(n) \rceil$ Knoten mit Grad ≥ 3

In jedem Graphen mit n Knoten, der keine Knoten mit Grad 1, aber mindestens eine Kante hat, existiert ein Kreis mit höchstens $2 \cdot \lceil \log_2(n) \rceil$ Knoten mit Grad ≥ 3.

Beweis. Zunächste entfernen wir aus G alle Knoten mit Grad 2, inden wir nach und nach jeden Knoten mit Grad 2 durch eine direkte Kante zwischen seinen beiden Nachbarknoten ersetzen. Es bezeichne G' den resultirenden Graphen. Jeder Kreis in G' entspricht einem Kreis in G.

Wir machen Breitensuche in G' beginnend bei einem beliebigen Knoten. Solange kein Knoten in der Breitensuche mehrfach besucht wird, gibt es auf Level $i \geq 1$ mindetsens $3 \cdot 2^{i-1}$ Knoten, insgesamt also

$$1 + \sum_{i=1}^{k} 3 \cdot 2^{i-1} = 1 + 3 \cdot (2^k - 1) = 3 \cdot 2^k - 2.$$

Für $K = \left\lceil \log_2\left(\frac{n+2}{3}\right)\right\rceil$ gilt $3 \cdot 2^K - 2 \geq n$. Damit ergibt sich ein Kreis der Länge $2K+1$, da die BFS spätestens auf Tiefe K den ersten Kreis liefert. In G entspricht das einem Kreis mit höchstens $2K+1$ Knoten mit Grad ≥ 3. Es gilt

$$2 \cdot \left\lceil \log_2\left(\frac{n+2}{3}\right)\right\rceil + 1 \leq 2 \cdot \lceil \log_2(n) \rceil. \qquad \square$$

Ziel: Beschränke $|S \cap C|$ für alle Kreise C mit $y_C > 0$.

Knoten mit Grad ≥ 3 teilen C in maximal $2\lceil \log_2(n) \rceil$ Segmente ein, wobei jedes Segment nur Grad-2-Knoten enthält. Folglich kann S höchstens alle Grad-$\geq$ 3-Knoten von C und pro Segment einen weiteren Knoten enthalten. Damit folgt $|S \cap C| \leq 4\lceil \log_2(n) \rceil$ und wir erhalten

$$\sum_{v \in S} w_v = \sum_{v \in S} \left(\sum_{C : v \in C} y_C \right) = \sum_{C \in \mathcal{C}} y_C \cdot |S \cap C| \leq 4 \cdot \lceil \log_2(n) \rceil \cdot \sum_{C \in \mathcal{C}} y_C \leq 4 \cdot \lceil \log_2(n) \rceil \cdot \mathrm{OPT}\,.$$

Wir erhalten zusammengefasst folgenden Satz:

Satz 8.10: PrimalDual-FVS2

PrimalDual-FVS2 ist ein $4\lceil \log_2(n) \rceil$-Approximationsalgorithmus für Feedback Vertex Set.

8.4 Das Kürzeste-Wege-Problem

Problem 8.11: Kürzeste-Wege-Problem

Input: Ungerichteter Graph $G = (V, E)$, Kantengewichte $w : E \to \mathbb{R}_{>0}$, Startknoten s, Zielknoten t
Output: s-t-Pfad in G
Objective: Minimiere $\sum_{e \in P} w(e)$

Wir formulieren dies wieder als ILP (für $w_e = w(e)$ und $\mathcal{S} = \{S \subseteq V \mid s \in S, t \notin S\}$ als Menge der s-t-Schnitte, mit $\delta(S) = \{\{x, y\} \in E \mid |\{x, y\} \cap S| = 1\}$ für $S \in \mathcal{S}$)

$$\begin{aligned} \min \quad & \sum_{e \in E} w_e x_e & & \\ s.t. \quad & \sum_{e \in \delta(S)} x_e \geq 1, & \forall S \in \mathcal{S} & \qquad (8.10) \\ & x_e \in \{0, 1\}, & \forall e \in E, & \end{aligned}$$

bzw. relaxiert

$$\begin{aligned} \min \quad & \sum_{e \in E} w_e x_e & & \\ s.t. \quad & \sum_{e \in \delta(S)} x_e \geq 1, & \forall S \in \mathcal{S} & \qquad (8.11) \\ & x_e \geq 0, & \forall e \in E. & \end{aligned}$$

[26.11.2025, Vorlesung 14]
[02.12.2025, Vorlesung 15]

Sei P ein s-t-Weg. Setze $x_e = \mathbb{1}_{\{e \in P\}}(e)$ für alle $e \in E$. Dann ist x eine gültige Lösung von (8.10).

Sei andersrum x eine zulässige Lösung von (8.10). Dann codiert x eine Knotenauswahl, die einen s-t-Weg enthält.

Dualisieren von (8.11) ergibt

$$\begin{aligned} \max \quad & \sum_{S \in \mathcal{S}} y_S \\ s.t. \quad & \sum_{S: e \in \delta(S)} y_S \geq w_e, \quad \forall e \in E \\ & y_S \geq 0, \qquad\quad \forall S \in \mathcal{S}. \end{aligned} \tag{8.12}$$

Algorithmus 8.12 PrimalDual-ShortestPath

Input: Wie in Problem 8.11
Output: Wie in Problem 8.11

1: $y := 0$ ▷ *Lösung von* (8.12)
2: $F := \emptyset$
3: **while** $\nexists$ s-t-Weg in (V, F) **do**
4: $\quad S := \{v \in V \mid \exists\ s\text{-}v\text{-Weg in } (V, F)\}$
5: $\quad$ Erhöhe y_S bis für ein $e \in \delta(S)$ gilt $\sum_{S': e \in \delta(S)} y_{S'} = w_e$
6: $\quad F := F \cup \{e\}$
7: Sei P ein beliebiger s-t-Weg in (V, F)
8: **return** P

Lemma 8.13: PrimalDual-ShortestPath wählt Baum aus

Zu jedem Zeitpunkt gilt, dass F einen Baum auf den von s in (V, F) erreichbaren Knoten bildet.

Beweis. Zu Beginn ist $F = \emptyset$ und die Aussage ist offensichtlich wahr.

Nun werde $e = \{x, y\}$ der Menge F hinzugefügt. Es ist also $e \in \delta(S)$ für die Menge S, der von s aus erreichbaren Knoten in (V, F). Sei $x \in S$ und $y \notin S$. Da y von s aus über F nicht erreichbar ist, kann die Hinzunahme von e nicht zu einem Kreis führen (da der Graph ungerichtet ist). Damit ist die Invariante gezeigt. □

Satz 8.14: PrimalDual-ShortestPath

PrimalDual-ShortestPath ist ein 1-Approximationsalgorithmus für das Kürzeste-Wege-Problem, also wird stets ein kürzester Weg ausgegeben.

Beweis. Es gilt

$$\sum_{e \in P} w_e = \sum_{e \in P} \sum_{S: e \in \delta(S)} y_S = \sum_{S \in \mathcal{S}} y_S \cdot |P \cap \delta(S)|.$$

Es bleibt also zu zeigen, dass $|P \cap \delta(S)| \leq 1$ für alle S mit $y_S > 0$, durch schwache Dualität (Satz 8.1) folgt damit nämlich

$$\sum_{e \in P} w_e \leq \sum_{e \in P} y_S \leq \mathrm{OPT}\,.$$

Angenommen, es gibt einen Schnitt $S \in \mathcal{S}$ mit $y_S > 0$ und $|P \cap \delta(S)| \geq 2$. Dann gilt sogar $|P \cap \delta(S)| \geq 3$, da der Schnitt immer ungerade oft gekreuzt wird. Sei w der erste Knoten, den wir beim Wiedereintritt in S erreichen. Da zu jedem Zeitpunkt die Kanten F einen Baum bildet, und $w \in S$, gibt es einen Weg von s zu w innerhalb von S. Damit gibt es also einen Kreis. Da nach 8.13 (V, F) aber kreisfrei ist, folgt der Widerspruch. □

FunFact: PrimalDual-ShortestPath ist literally Dijkstra.

8.5 Steinerwaldproblem

Problem 8.15: Steinerwaldproblem

Input: Ungerichteter Graph $G = (V, E)$ mit Kantengewichten $w : E \to \mathbb{R}_{>0}$, Knotenpaare (s_i, t_i) für $i \in \{1, \dots, k\}$ des Graphen
Output: Kantenauswahl $F \subseteq E$, sodass für alle i ein s_i-t_i-Weg in F existiert
Objective: Minimiere $\sum_{e \in F} w(e)$

Wir modellieren das Steinerwaldproblem nun wieder mit Linearen Programmen. Es geht sehr analog zu Abschnitt 8.4. Wir setzen $\mathcal{S}_i := \{S \subseteq V \mid |S \cap \{s_i, t_i\}| = 1\}$ und $\mathcal{S} = \mathcal{S}_1 \cup \dots \cup \mathcal{S}_k$. Dann können wir die LPs übernehmen und erhalten

$$\begin{aligned} \min \quad & \sum_{e \in E} w_e x_e \\ s.t. \quad & \sum_{e \in \delta(S)} x_e \geq 1, \quad \forall S \in \mathcal{S} \\ & x_e \in \{0, 1\}, \qquad \forall e \in E, \end{aligned}$$

bzw. relaxiert

$$\begin{aligned} \min \quad & \sum_{e \in E} w_e x_e \\ s.t. \quad & \sum_{e \in \delta(S)} x_e \geq 1, \quad \forall S \in \mathcal{S} \\ & x_e \geq 0, \qquad \forall e \in E. \end{aligned} \tag{8.13}$$

Dualisieren ergibt wieder

$$\begin{aligned} \max \quad & \sum_{S \in \mathcal{S}} y_S \\ s.t. \quad & \sum_{S : e \in \delta(S)} y_S \geq w_e, \quad \forall e \in E \\ & y_S \geq 0, \qquad \forall S \in \mathcal{S}. \end{aligned} \tag{8.14}$$

Wir werden versuchen, einen sehr analogen Algorithmus zu PrimalDual-ShortestPath aufzustellen:

Algorithmus 8.16 PrimalDual-SteinerForest1

Input: Wie in Problem 8.15
Output: Wie in Problem 8.15

1: $y := 0$ ▷ *Lösung von* (8.14)
2: $F := \emptyset$
3: **while** $\exists\, i : \nexists\, s_i$-$t_i$-Weg in (V, F) **do**
4: Sei S eine Zusammenhangskomponente von (V, F) mit $|S \cap \{s_i, t_i\}| = 1$
5: Erhöhe y_S bis für ein $e \in \delta(S)$ gilt $\sum_{S' : e \in \delta(S)} y_{S'} = w_e$
6: $F := F \cup \{e\}$
7: **return** F

Es gilt wieder

$$\sum_{e \in F} w_e = \sum_{e \in F} \sum_{S : e \in \delta(S)} y_S = \sum_{S \in \mathcal{S}} y_S \cdot |F \cap \delta(S)|.$$

Betrachte folgendes Beispiel: Wir haben eine $(k+1)$-Clique, wobei $s_1 = \dots = s_k = v$ und die t_i alle verschieden sind und alle Kanten die Länge 1 haben. Sei $S = \{v\}$ der erste Schnitt. Nun

wird y_S erhöht, bis $y_S = 1$ ist. Dann sind plötzlich alle Knoten erreichbar, und wir fügen eine dieser Kanten zu F hinzu. Nun wird wieder eine Zusammenhangskomponente ausgesucht, und eine weitere duale Variable erhöht. Es kann aber tatsächlich für keine weitere Menge $S' \in \mathcal{S}$ die Variable $y_{S'}$ noch um mehr als 0 erhöht werden. Damit ist für $S = \{v\}$ am Ende $|F \cap \delta(S)| = k$. Durch diese Analyse bekommen wir also keinen besseren Approximationsfaktor als durch die triviale Betrachtung, einfach alle k kürzesten Wege zu wählen.

Auch ein konkreteres Wählen der Zusammenhangskomponente S funktioniert nicht trivial sinnvoll. Wir brauchen einen anderen Ansatz, und erhöhen die dualen Variablen aller Zusammenhangskomponenten parallel gleichzeitig:

[02.12.2025, Vorlesung 15]
[09.12.2025, Vorlesung 16]

Algorithmus 8.17 PrimalDual-SteinerForest2

Input: Wie in Problem 8.15
Output: Wie in Problem 8.15

1: $y := 0$ ▷ *Lösung von* (8.14)
2: $F := \emptyset$
3: $\ell := 0$ ▷ *Schleifenindex*
4: **while** $\exists\, i : \nexists s_i$-$t_i$-Weg in (V, F) **do**
5: $\quad \ell := \ell + 1$
6: $\quad C := \{\text{Zusammenhangskomponente } S \text{ von } (V, F) \mid \exists\, i : |S \cap \{s_i, t_i\}| = 1\}$
7: $\quad$ Erhöhe y_S gleichmäßig für alle $S \in C$, bis für ein S aus C und eine Kante $e_\ell \in \delta(S)$ gilt $\sum_{S' : e_\ell \in \delta(S')} y_{S'} = w_{e_\ell}$.
8: $\quad F := F \cup \{e_\ell\}$
9: $F' := F$
10: **for** $k = \ell, \ell - 1, \ldots, 1$ **do**
11: $\quad$ **if** $F' \setminus \{e_k\}$ gültige Lösung **then**
12: $\quad\quad F' := F' \setminus \{e_k\}$
13: **return** F'

Lemma 8.18: PrimalDual-SteinerForest2: Iterationen

In jeder Iteration von PrimalDual-SteinerForest2 gilt

$$\sum_{S \in C} |F' \cap \delta(S)| \leq 2|C|.$$

Satz 8.19: PrimalDual-SteinerForest2

PrimalDual-SteinerForest2 ist ein 2-Approximationsalgorithmus.

Beweis. Es gilt

$$\sum_{e \in F'} w_e = \sum_{e \in F'} \sum_{S' : e \in \delta(S)} y_S = \sum_{S \in \mathcal{S}} y_S \cdot |F' \cap \delta(S)|.$$

Wir wollen also zeigen, dass

$$\sum_{S \in \mathcal{S}} y_S |F' \cap \delta(S)| \leq 2 \sum_{S \in \mathcal{S}} y_S$$

gilt. Per Induktion über ℓ: Für den Induktionsanfang gilt $y_S = 0$ für alle $S \in \mathcal{S}$, es ist also nichts zu zeigen. Für den Induktionsschritt betrachte die Iteration, in der jedes y_S für $S \in C$ um ε erhöht wird. Dann wird $\sum_{S \in \mathcal{S}} y_S \cdot |F' \cap \delta(S)|$ um $\varepsilon \cdot \sum_{S \in C} |F' \cap \delta(S)|$ erhöht. Die rechte Seite $2 \sum_{S \in \mathcal{S}} y_S$ wird um $2\varepsilon|C|$ erhöht. Mit Lemma 8.18 wird die linke Seite höchstens so stark erhöht wie die rechte Seite. □

Beweis von Lemma 8.18. Sei $F_\ell = \{e_1, \dots, e_{\ell-1}\}$ und $H = F' \setminus F_\ell$. Dann ist $F_\ell \cup H = F_\ell \cup F'$ eine Lösung. Betrachte Iteraiton ℓ und lege einen Graphen G' an. Dabei setzen wir die Knoten V' als die Zusammenhangskomponenten von (V, F_ℓ), und die Kanten entsprechen den Kanten aus H.

Jede Kante $e \in H$ verläuft zwischen verschiedenen Zusammenhangskomponenten von (V, F_ℓ). G' ist außerdem kein Multigraph, also zwischen zwei Knoten aus G' gibt maximal eine Kante.

Für $v \in V'$ sei $d(v)$ der Grad von v. Wir sagen, $v \in V'$ ist *aktiv*, wenn für die entsprechende Zusammenhangskomponente S gilt, dass $S \in C$. Wir sagen, $v \in V'$ ist *passiv*, wenn v nicht aktiv ist, aber $d(v) > 0$ gilt. Die Menge $A \subseteq V'$ beschreibt die aktiven Knoten, $P \subseteq V'$ beshreibt die passiven Knoten.

Wir wollen zeigen:

$$\sum_{S \in C} |F' \cap \delta(S)| \leq 2|C|, \tag{8.15}$$

dabei ist $|F' \cap \delta(S)| = \delta(v_S)$ für v_S den entsprechenden Knoten in G', die rechte Seite ist das gleiche wie $2 \cdot |A|$, somit ist (8.15) äquivalent zu

$$\sum_{v \in A} d(v) \leq 2|A|. \tag{8.16}$$

Es gilt

$$\sum_{v \in A} d(v) = \sum_{v \in A \cup P} d(v) - \sum_{v \in P} d(v).$$

Wäre $d(v) \geq 2$ für alle $v \in P$, dann gilt

$$\sum_{v \in A} d(v) = \sum_{v \in A \cup P} d(v) - \sum_{v \in P} d(v) \leq 2 \cdot (|A| + |P|) - 2|P| = 2|A|,$$

wobei $\sum_{v \in A \cup P} d(v) \leq 2 \cdot (|A| + |P|)$, weil G' kreisfrei (und kein Multigraph) ist und somit $|E(G')| \leq |V(G')| - 1 \leq |A| + |P| - 1 \leq |A| + |P|$.

Angenommen, für $v \in P$ ist $d(v) = 1$. Der Fall $d(v) = 0$ kann nach Definition passiver Knoten nicht auftreten. Sei e die zu v inzidente Kante. e gehört zu H und damit zu F'. Damit wird e gebraucht, um Zusammenhang zwischen s_i und t_i für ein i herzustellen. Dann ist e aktiv. $\lightning$ □

Teil II

Online-Algorithmen

Kapitel 11

Einleitung

In den vergangenen Problemen kannten wir von Anfang an die komplette Eingabe. Nun stellen wir uns vor, dass im Laufe eines Algorithmus weitere Eingaben bekannt werden bzw. dazukommen, beispielsweise auf dem Aktienmarkt. Ein Algorithmus dafür heißt *Online-Algorithmus*. Mehr dazu nächste Woche.

[09.12.2025, Vorlesung 16]
[16.12.2025, Vorlesung 17]

Das klassische Einführungsproblem ist das fiktive *Ski Rental*-Problem: Entweder zahlen wir 50 pro Tag, oder 800 als Flat Rate. Wir wissen aber vorher nicht, wie gut das Wetter ist und ob man an diesem Tag Ski fahren möchte. Angenommen, wir gehen an x Tagen Ski fahren.

1. **Strategie** Wir kaufen die Flat Rate am ersten Tag. Dann ist $\mathrm{OPT} = \min\{800, 50x\}$ und $\frac{800}{\mathrm{OPT}} \leq 16$.
2. **Strategie** Wir kaufen die Flat Rate nie. Dann ist $\frac{50x}{\min\{800,50x\}} \overset{x\to\infty}{\longrightarrow} \infty$.
3. **Strategie** Wir kaufen die Flat Rate am 17. Tag. Dann bezahlen wir $50x$ für $x \leq 16$ und 1600 für $x \geq 16$ und wir erhalten $\frac{\text{Kosten}}{\mathrm{OPT}} \leq 2$.

Definition 11.1: Kompetitiver Faktor

Ein Online-Algorithmus A für ein Minimierungsproblem *erreicht einen kompetitiven Faktor* $r \geq 1$ (oder *ist* r*-kompetitiv*), wenn

$$\exists \tau \in \mathbb{R}\ \forall \sigma \in \mathcal{I}_\Pi : \quad w_A(\sigma) \leq r \cdot \mathrm{OPT}(\sigma) + \tau.$$

Gilt das sogar für $\tau = 0$, so heißt A *strikt* r*-kompetitiv*.

Ein strikt r-kompetitiver, polynomieller Online-Algorithmus ist tatsächlich auch ein r-Approximastionsalgorithmus. Andersrum gilt das im Allgemeinen nicht.

Kapitel 12

Paging

Wir betrachten die Frage *Cache vs. RAM*. Der Cache habe eine Größe k. Es sei $\sigma = (\sigma_1, \sigma_2, \ldots)$ mit $\sigma_i \in \mathbb{N}$. Dieses entspricht einer aufgerufenen Seite. Wir können also nur k Seiten gleichzeitig in unserem Cache aktiv haben. Die Kosten sind nun die Anzahl der Zugriffe, bei denen eine Seite in den Cache geladen werden muss. Außerdem sei der Cache am Anfang leer.

Problem 12.1: Paging

Input: Seiten $\{1, \ldots, m\}$, Cache-Größe k, Eingabe $\sigma = (\sigma_1, \ldots, \sigma_n) \in \{1, \ldots, m\}^n$
Output: Verteilung der aktuell im Cache befindlichen Seiten
Objective: Minimiere Anzahl der Seitenladungen

12.1 Deterministische Algorithmen

Wir haben folgende Algorithmen-Ideen:

LRU (Least Recently Used)

FIFO (First In, First Out)

FWF (Flush When Full)

RANDOM *nicht deterministisch*

LFU (Least Frequently Used)

LIFO (Last In, First Out)

LFD (Longest Forward Distance) *keine Online-Strategie*

12.1.1 Markierungsalgorithmen

Wir betrachten eine *Phaseneinteilung*: Sei $k = 3$ und $\sigma = (1, 2, 4, 2, 1, 3, 5, 2, 3, 5, 1, 2, 3, 4)$. Wir unterteilen dies in Phasen, in denen nur auf k verschiedene Seiten zugegriffen wird (mit maximaler Länge).

Beispiel 12.2. Für obiges Beispiel erhalten wir die folgende Phaseneinteilung:

$$\sigma = (\overbrace{1, 2, 4, 2, 1}^{P1}, \overbrace{3, 5, 2, 3, 5}^{P2}, \overbrace{1, 2, 3}^{P3}, \overbrace{4}^{P4}).$$

Definition 12.3: Markierungsalgorithmus

Ein Algorithmus ist ein *Markierungsalgorithmus*, wenn wir nie eine Seite verdrängen, auf die in der aktuellen Phase bereits zugegriffen wurde.

Satz 12.4: Beispiele: Markierungsalgorithmen auf Paging

LRU und FWF sind Markierungsalgorithmen.

Beweis. **LRU:** Sei σ eine Eingabe, auf der LRU eine markierte Seite x verdränkt, zum Zeitpunkt t in Phase i. Dann wird Seite x (das vor Seite σ_t, zum Zeitpunkt $t' < t$ vorkommt) entfernt. Damit müssen zwischen $\sigma_{t'}$ und σ_t $k-1$ Seiten $\neq x$ vorkommen. Damit gibt es aber $k+1$ verschiedene Seiten in Phase i. $\lightning$

FWF: Wenn die erste Phase vorbei ist, wird der gesamte Cache geleert. Somit besteht der Cache gerade aus den Seiten, die in der aktuellen Phase bereits vorgekommen sind.

□

Satz 12.5: Markierungsalgorithmen auf Paging

Jeder Markierungsalgorithmus ist strikt k-kompetitiv.

Beweis. Sei A ein Markierungsalgorithmus und σ eine Eingabe. Sei ℓ die Anzahl der Phasen in σ und sei $\ell \geq 2$. Es gilt $w_A(\sigma) \leq k \cdot \ell$, da in einer Phase höchstens k Seiten neu in den Cache geladen werden (sonst hätte die Phase mehr als k Seiten!).

Wir teilen eine Eingabe in *Teilsequenzen* auf, wobei Teilsequenz i (mit $i \in \{1, \ldots, \ell-2\}$) mit dem zweiten Zugriff der $(i+1)$-ten Phase beginnt und mit dem ersten Zugriff in Phase $i+2$.

Behauptung: Jeder Algorithmus muss in jeder Teilsequenz mindestens einen Seitenfehler produzieren: Mindestens eine der Seiten aus der Teilsequenz ist zu Beginn der Teilsequenz nicht im Cache.

Somit erhalten wir $\mathrm{OPT}(\sigma) \geq k + (\ell - 2) = k + \ell - 2 \geq \ell$, da wir in der ersten Phase k Seiten in den leeren Cache laden.

Somit erhalten wir $w_A(\sigma) \leq k \cdot \ell \leq \mathrm{OPT}(\sigma) \cdot k$. □

Korollar. *LRU und FWF sind strikt k-kompetitiv.*

12.1.2 Untere Schranken

Satz 12.6: LFU, LIFO

LFU und LIFO sind nicht kompetitiv.

Das heißt, wir finden keinen konstanten Kompetitiven Faktor.

Beweis.

LIFO: Sei $\sigma = (1, 2, \ldots, k, (k+1, k)^\ell)$. Dann ist $w_{\mathrm{LIFO}}(\sigma) = 2\ell + (k-1)$ und $\mathrm{OPT}(\sigma) = k+1$.

LFU: Sei $\sigma = (1^{\ell+1}, 2^{\ell+1}, \ldots, (k-1)^{\ell+1}, k, (k+1, k)^\ell)$. Dann ist ebenfalls $w_{\mathrm{LFU}}(\sigma) = 2\ell + (k-1)$ und $\mathrm{OPT}(\sigma) = k+1$.

□

Satz 12.7: Deterministische Online-Algorithmen für Paging

Für kein $r < k$ gibt es einen deterministischen *r-kompetitiven Online-Algorithmus für Paging.*

Beweis. Sei A ein beliebiger Online-Algorithmus. Wir konstruieren σ wie folgt:

$\sigma^{(\ell)} = (1, \dots, k, \sigma_{k+1}, \dots, \sigma_\ell)$, und σ_i mit $i > k$ entspricht der *eindeutigen* Seite aus $\{1, \dots, k+1\}$, die A nicht im Cache hält. Dann ist $w_A(\sigma) = \ell$, da in jedem Schritt ein Seitenfehler passiert.

[16.12.2025, Vorlesung 17]
[17.12.2025, Vorlesung 18]

Andererseits ist $\mathrm{OPT} \leq w_{\mathrm{LFD}}(\sigma^\ell) \leq k + \left\lceil \frac{\ell}{k} \right\rceil$ Nun gibt es für alle $r < k$ und für alle τ ein ℓ, sodass $w_A(\sigma^{(\ell)}) > \tau + r \cdot \mathrm{OPT}(\sigma^{(\ell)})$ – wähle $\ell > \tau + r \cdot \left(k + \left\lceil \frac{\ell}{k} \right\rceil\right)$ und wir erhalten somit den Widerspruch. ↯ □

12.2 Randomisierte Algorithmen

Wir betrachten folgende Algorithmen:

RANDOM Verdränge bei Seitenfehler bei vollem Cache eine uniform zufällig gewählte Seite

MARK Verdränge bei Seitenfehler bei vollem Cache uniform zufällig eine nicht-markierte Seite

Definition 12.8: Kompetitiver Faktor für randomisierte Algorithmen

Ein randomisierter Online-Algorithmus A für ein Minimierungsproblem *erreicht einen kompetitiven Faktor* $r \geq 1$ (oder *ist* r*-kompetitiv*), wenn

$$\exists \tau \in \mathbb{R}\ \forall \sigma \in \mathcal{I}_\Pi : \quad \mathbb{E}[w_A(\sigma)] \leq r \cdot \mathrm{OPT}(\sigma) + \tau.$$

Gilt das sogar für $\tau = 0$, so heißt A *strikt* r*-kompetitiv.*

12.2.1 Potentialfunktionen

Sei A ein Online-Algorithmus und $\sigma = (\sigma_1, \dots, \sigma_n)$ eine Eingabe. Wir wollen für A und σ Potentiale $\Phi_0, \dots, \Phi_n \in \mathbb{R}$ finden, die intuitiv einen „Kontostand“ darstellen sollen – ist Φ positiv, haben wir gegenüber OPT mehr Kosten gebraucht, für Φ negativ analog.

Definition 12.9: Amortisierte Kosten

Seien A_i und OPT_i die Kosten von A bzw. OPT bei Ereignis σ_i. Wir sagen,

$$a_i := A_i + \Phi_i - \Phi_{i-1}$$

sind die *amortisierten Kosten* von A für σ_i.

Satz 12.10: Potentialfunktion

Sei A ein Online-Algorithmus und $r \geq 1$. Gibt es eine Konstante $b \geq 0$ und eine Potentialfunktion Φ mit den folgenden Eigenschaften, so ist A r-kompetitiv:

1. *Für alle $i \geq 1$ gilt $\mathbb{E}[a_i] \leq r \cdot \mathrm{OPT}_i$.*
2. *$\mathbb{E}[\Phi_0] \leq b$.*
3. *Für alle $i \geq 1$ gilt $\mathbb{E}[\Phi_i] \geq -b$.*

Beweis. Es gilt

$$\begin{aligned}\mathbb{E}[w_A(\sigma)] &= \mathbb{E}\left[\sum_{i=1}^{n} A_i\right] = \mathbb{E}\left[\sum_{i=1}^{n}(a_i - \Phi_i + \Phi_{i-1})\right] \\ &= \sum_{i=1}^{n}\mathbb{E}[a_i] - \mathbb{E}[\Phi_i] + \mathbb{E}[\Phi_{i-1}] = \left(\sum_{i=1}^{n}\mathbb{E}[a_i]\right) + \mathbb{E}[\Phi_0] - \mathbb{E}[\Phi_n] \\ &\leq \sum_{i=1}^{n} r\cdot \mathrm{OPT}_i + b + b = r\cdot\sum_{i=1}^{n}\mathrm{OPT}_i + 2b = r\cdot\mathrm{OPT} + 2b.\end{aligned}$$

mit Linearität des Erwartungswerts und den gegebenen Bedingungen. □

12.2.2 Analyse von RANDOM

Wur definieren das Potential basierend auf den Cacheinhalten von RANDOM und OPT. Sei z die Anzahl an gemeinsamen Seiten. Wir setzen dabei $\Phi(z) = k(k-z)$.

Satz 12.11: RANDOM ist k-kompetitiv

RANDOM ist k-kompetitiv.

Beweis. Es ist $\Phi_0 = \Phi(0) = k^2$, da beide Caches leer sind und entsprechend keine gemeinsamen Seiten haben. Ebenso gilt $\Phi_i \geq 0$ für alle i, dementsprechend sind die hinteren beiden Eigenschaften erfüllt.

Zur ersten Eigenschaft: Zu zeigen ist, dass

$$\mathbb{E}[a_i] \leq r\cdot\mathrm{OPT}_i \iff \mathbb{E}[A_i+\Phi_i-\Phi_{i-1}] \leq r\cdot\mathrm{OPT} \iff \mathbb{E}[\Phi_i-\Phi_{i-1}] \leq r\cdot\mathrm{OPT}_i - \mathbb{E}[A_i], \quad (12.1)$$

wobei $A = \mathrm{RANDOM}$.

Sei das Verhalten von RANDOM auf $\sigma_1,\ldots,\sigma_{i-1}$ bereits aufgedeckt, und sei z_{i-1} die Anzahl an gemeinsamen Seiten nach $\sigma_1,\ldots,\sigma_{i-1}$, und P mit $|P| = z_{i-1}$ die gemeinsamen Seiten, und sei $\sigma_i = p$.

Fall 1: p ist im Cache von RANDOM. Dann ist $A_i = 0$ und $z_i - z_{i-1} \in \{0,1\}$. Dementsprechend ist $\Phi_i - \Phi_{i-1} \in \{0,-k\}$. Damit ist die linke Seite von (12.1) nichtpositiv und die rechte Seite nichtnegativ.

Fall 2: p ist nicht im Cache von RANDOM, aber im Cache von OPT. Dann ist $A_i = 1$ und $z_i - z_{i-1} \in \{0,1\}$. Entsprechend ist $\Phi_i - \Phi_{i-1} \in \{0,-k\}$. Die Wahrscheinlichkeit, eine Seite zu verdrängen, die nicht zu P gehört, beträgt $\frac{k-z_{i-1}}{k}$. Damit gilt

$$\mathbb{E}[\Phi_i - \Phi_{i-1}] = \frac{k-z_{i-1}}{k}\cdot(-k) = z_{i-1} - k \leq -1, \quad (12.2)$$

wobei $z_{i-1} \leq k-1$, da p nur im Cache von OPT enthalten ist. Mit $k\cdot\mathrm{OPT} - A_i = -1$ ist (12.1) damit bewiesen.

Fall 3: p ist weder im Cache von RANDOM noch im Cache von OPT. Dann nimmt die rechte Seite von (12.1) den Wert $k-1$ an.

Fall 3.1: OPT verdrängt eine Seite, die nicht zu P gehört. Dann ist $\Phi_i - \Phi_{i-1} \in \{0,-k\}$, je nachdem, ob RANDOM eine Seite aus P oder eine andere Seite verdrängt. Somit ist die linke Seite von (12.1) kleiner als $k-1$.

Fall 3.2: OPT verdrängt eine Seite aus P. Verdrängt RANDOM dieselbe Seite, so ändert sich das Potential nicht, ebenso wenn RANDOM eine andere Seite, die nicht zu P gehört, verdrängt. Nur der Fall, dass RANDOM eine andere Seite aus P als OPT

verdrängt, erhöht sich das Potential um k – eintreten tut dies mit Wahrscheinlichkeit $\frac{z_{i-1}-1}{k}$, daraus folgt

$$\mathbb{E}[\Phi_i - \Phi_{i-1}] = \frac{z_{i-1}-1}{k} \cdot k \leq k-1. \tag{12.3}$$

Damit ist (12.1) gezeigt für jedes P. □

[16.12.2025, Vorlesung 18]
[07.01.2026, Vorlesung 19]

12.2.3 Analyse von MARK

[07.01.2026, Vorlesung 19]
[13.01.2026, Vorlesung 20]

12.2.4 Untere Schranke für randomisierte Online-Algorithmen

Satz 12.12: Randomisierte Online-Algorithmen für Paging

Es gibt keinen randomisierten Online-Algorithmus für Paging, der besser als H_k-kompetitiv ist.

Beweis. Sei A ein beliebiger, randomisierter Online-Algorithmus. Konstruiere eine Eingabe σ mit $\sigma_i \in \{1,\dots,k+1\}$ und $\sigma = (1,\dots,k,\dots)$.

Für σ gibt es eine Wahrscheinlichkeitsverteilung:

$$\forall i \in \{1,\dots,k+1\} \text{ definiere } p_i := p_i^\sigma := \mathbb{P}[\text{Seite } i \text{ nach } \sigma \text{ nicht im Cache}].$$

Dann gilt $\sum_{i=1}^{k+1} p_i^\sigma = 1$.

Erste Idee: Wähle $\sigma_i = j$ für das j, für das $p_j^{\sigma_1,\dots,\sigma_{i-1}}$ maximal ist. Das Problem: Für RANDOM gilt stets $p_{\sigma_i}^{\sigma_1,\dots,\sigma_i} = 0$ und für alle $j \neq \sigma_i$ ist $p_j^{\sigma_1,\dots,\sigma_i} = \frac{1}{k}$. Erwartete Kosten von RANDOM sind pro Schritt $\frac{1}{k}$. Somit erwartet RANDOM pro Schritt Kosten von $\frac{1}{k}$. Je nach Tiebreaking verursacht auch OPT in jedem k-ten Schritt einen Seitenfehler.

Konstruktion einer Phase σ': Wir unterteilen nun σ wieder in Phasen $\{\sigma'\}$, diese bestehen jeweils aus mehreren Teilphasen $\sigma'_1,\dots,\sigma'_k$. Eine Seite heißt *markiert*, wenn sie in der aktuellen Phase schon vorgekommen ist oder wenn es die letzte Seite aus der vorherigen Phase ist.

Eine Teilphase σ'_i besteht aus einer (ggf. leeren) Sequenz von Zugriffen auf markierte Seiten, gefolgt von einer unmarkierten Seite. Dann ist $\mathrm{OPT}(\sigma) \leq (k+1) + (m-1)$, wobei m die Gesamtzahl der Phasen ist.

Konstruktion von Teilphase σ'_j: Zu Beginn von σ'_j gibt es $u = k - j + 1$ unmarkierte Seiten. Wir setzen $\gamma := \sum_{i \in M} p_i$, wobei M die menge der markierten Seiten ist.

Fall 1: $\gamma = 0$. Dann gibt es $a \notin M$ mit $p_a \geq \frac{1}{u}$. Dann ist $\sigma'_j = (a)$.

Fall 2: $\gamma > 0$. Sei $a \in M$ mit $p_a > 0$ beliebig und sei $\varepsilon := p_a$. Dann beginne σ'_j mit Zugriff auf a. Wir betrachten folgenden Algorithmus:

1: $E := p_a$
2: **while** $E < \frac{1}{u}$ **and** $\gamma > u\varepsilon$ **do**
3: Hänge Zugriff $b \in M$ mit $b := \arg\max_{i \in M} p_i$ an σ'_j an.
4: $E := E + p_b$
5: Aktualisiere $p_1,\dots,p_{k+1}$ und $\gamma = \sum_{i \in M} p_i$.
6: Hänge Zugriff auf $b' \notin M$ mit $b' := \arg\max_{i \notin M} p_i$ an σ'_j an.

Terminiert die Schleife wegen $E \geq \frac{1}{u}$, so betragen die erwarteten Kosten von a auf σ'_j mindestens $\frac{1}{u}$. Betrachte also den Fall, dass die Schleife wegen $\gamma \leq u\varepsilon$ terminiert. Die erwarteten Kosten von A auf σ'_j betragen mindestens

$$p_a + p_{b'} = \varepsilon + \frac{1-\gamma}{u} \geq \varepsilon + \frac{1-u\varepsilon}{u} = \frac{1}{u}.$$

Damit betragen die erwarteten Kosten von Algorithmus A auf der Teilphase σ'_j mindestens $\frac{1}{u_j}$ mit $u_j := k - j + 1$. Aufsummiert ergibt sich

$$\mathbb{E}[\text{Kosten von } A \text{ auf Phase } i \geq 2] \geq \sum_{j=1}^{k} \frac{1}{u_j} = \sum_{j=1}^{k} \frac{1}{k-j+1} = H_k.$$

Als Gesamtkosten erhalten wir

$$\mathbb{E}[w_A(\sigma)] \geq k + 1 + (m-1)H_k, \qquad \text{OPT} = k + m$$

und für m groß gilt $\frac{\mathbb{E}[w_A(\sigma)]}{\text{OPT}} \to H_k$. □

Kapitel 13

Das k-Server-Problem

Wir betrachten einen metrischen Raum $\mathcal{M} := (M, d)$ für einen endlichen Raum M und eine Metrik d.

Problem 13.1: k-Server

Input: Enldicher Metrischer Raum $\mathcal{M} = (M, d)$, Folge $\sigma = (\sigma_1, \ldots, \sigma_n)$ von Anfragen $\sigma_i \in M$, k Server mit Startstandorten in M
Output: Folge von Serverstandorten, sodass bei jeder Anfrage ein Server bei diesem Standort steht
Objective: Minimiere Gesamtkosten der Serververschiebungen

Das Paging-Problem ist ein Spezialfall des k-Server-Problems, wo $d(x, y) = 1$ für alle $x \neq y$ ist.

Algorithmus 13.2 Greedy-KServ

Input: Wie in Problem 13.1
Output: Wie in Problem 13.1

```
1: for σ_i ∈ σ do
2:     if Kein Server befindet sich bei σ_i then
3:         Verschiebe den nächsten Server zu σ_i
```

Dieser Algorithmus hat das Problem, dass eine Eingabe $\sigma = (c, a, b, a, b, a, b, \ldots)$ beliebig hohe Kosten erzeugen kann (für $k = 2$ und $d(a, b) < \min\{d(a, c), d(b, c)\}$ und a, b und c auf einer Linie).

[13.01.2026, Vorlesung 20]
[14.01.2026, Vorlesung 21]

13.1 Untere Schranke für deterministische Algorithmen

Satz 13.3: Untere deterministische Schranke für k-Server

Sei $\mathcal{M} = (M, d)$ ein metrischer Raum mit $|M| \geq k + 1$. Dann gibt es auf $\mathcal{M}$ keinen r-kompetitiven, deterministischen Online-Algorithmus für $r < k$.

Beweis. Sei A ein beliebiger, deterministischer Online-Algorithmus. OBdA sei A *faul*, das heißt, A bewegt nur dann einen Server, wenn die aktuelle Anfrage nicht abgedeckt ist, und dann auch nur einen.

Sei $B := \{b_1, \ldots, b_{k+1}\} \subseteq M$ so gewählt, dass zu Beginn alle Server auf Punkten aus $B \setminus \{b_1\}$ stehen und immer genau ein Punkt $b \in B$ nicht durch einen Server abgedeckt ist (also sich an einem Punkt niemals zwei Server gleichzeitig befinden).

Konstruktion von σ: Jedes $\sigma_i \in B$ sei so gewählt, dass nach $\sigma_1, \ldots, \sigma_{i-1}$ der Punkt σ_i nicht abgedeckt ist. Außerdem habe σ die Länge n.

Kosten von A auf σ: Es gilt $w_A(\sigma) \geq \sum_{i=1}^{n-1} d(\sigma_i, \sigma_{i+1})$, da unsere Eingabe immer so gewählt ist, dass im nächsten Schritt σ_{i+1} unbesetzt ist – entsprechend also in Schritt i ein Server von σ_{i+1} zu σ_i bewegt wird.

Kosten von OPT auf σ: Definiere für jedes $S \subseteq B$ mit $b_1 \in S$ und $|S| = k$ einen Algorithmus C_S.

Verhalten von C_S:

- Ist σ_i nicht abgedeckt, dann verschiebe Server σ_{i-1} zu σ_i.
- Zu Beginn seien von C_S genau die Punkte aus S abgedeckt.

Die Anzahl an Mengen $S \subseteq B$ mit $b_1 \in S$ und $|S| = k$ ist genau k, da $b_1 \in S$ und vom Rest genau ein Element von B fehlt.

Beobachtung: C_S und $C_{S'}$ mit $S \neq S'$ decken niemals im Laufe von σ exakt dieselben Positionen ab. (Induktion).

Daraus folgt: Zu jedem Zeitpunkt $\sigma_1, \ldots, \sigma_i$ gibt es für jedes $b \in B$ mit $b \neq \sigma_i$ genau einen Algorithmus, der b nicht abdeckt.

Für die Gesamtkosten all dieser Algorithmen erhalten wir

$$\sum_{\substack{S: b_1 \in S \\ |S|=k}} w_{C_S}(\sigma) = \sum_{i=2}^{n} d(\sigma_{i-1}, \sigma_i) = \sum_{i=1}^{n-1} d(\sigma_i, \sigma_{i+1}),$$

da pro Iteration jeweils genau ein Algorithmus einen Server verschiebt. Somit kostet C_S im Durchschnitt $\frac{1}{k}\sum_{i=1}^{n-1} d(\sigma_i, \sigma_{i+1})$ und folglich gibt es $S, b_1 \in S, |S| = k$ mit

$$w_{C_S}(\sigma) \leq \frac{1}{k} \sum_{i=1}^{n-1} d(\sigma_i, \sigma_{i+1}).$$

Somit erhalten wir $\frac{w_A}{\mathrm{OPT}} \to k$. □

13.2 Das k-Server-Problem auf Linien und Bäumen

Wir betrachten $\mathcal{M} = ([0,1], \|\cdot\|_2)$, das heißt, eine Linie. Wir betrachten den DC-Algorithmus (*Double Coverage*):

Algorithmus 13.4 DC

Input: Wie in Problem 13.1
Output: Wie in Problem 13.1

```
1: for σ_i ∈ σ do
2:     if Alle Server befinden sich links (oder rechts) von σ_i then
3:         Bewege nächsten Server zu σ_i
4:     else
5:         Bewege jeweils den nächsten Server links und rechts mit gleicher Geschwindigkeit auf
           σ_i zu, bis σ_i erreicht ist
```

13.2.1 Analyse von DC auf der Linie

Satz 13.5: DC auf der Linie

DC ist k-kompetitiv für das k-Server-Problem auf der Linie.

Beweis. Sei σ eine beliebige Eingabe. Seien $o_1, \dots, o_k$ die aktuellen Positionen von OPT und $s_1, \dots, s_k$ die aktuellen Positionen von DC. Wir definieren

$$M_{\min} := \min_{\pi \in \mathcal{S}_k} \sum_{i=1}^{k} d(s_i, o_{\pi(i)}), \qquad \Sigma_{\mathrm{DC}} := \sum_{i=1}^{k-1} \sum_{j=i+1}^{k} d(s_i, s_j), \qquad \Phi := k \cdot M_{\min} + \Sigma_{\mathrm{DC}} \in \left[0, 2k^2\right].$$

Seien DC_i und OPT_i die Kosten von DC bzw. OPT bei Bearbeitung von σ_i. Wir erhalten als amortisierte Kosten $a_i = \mathrm{DC}_i + \Phi_i - \Phi_{i-1}$. Wir müssen nun noch die Ungleichungen der Potentialfunktionen (wie in Satz 12.10) zeigen:

1. Wir müssen zeigen, dass $\Phi_i - \Phi_{i-1} \leq k \cdot \mathrm{OPT}_i - \mathrm{DC}_i$.

   Seien $o_1, \dots, o_k$ und $s_1, \dots, s_k$ die Positionen von OPT und DC nach σ_{i-1} und $o'_1, \dots, o'_k$ bzw. $s'_1, \dots, s'_k$ die Positionen von OPT und DC nach σ_i. Dementsprechend resultiert Φ_{i-1} aus den Positionen nach σ_{i-1} und Φ_i resultiert aus den Positionen nach σ_i.

   Wir führen das Zwischenpotential Φ'_{i-1} ein: Dies sei das Potential bezüglich $o'_1, \dots, o'_k$ und $s_1, \dots, s_k$.

   Lemma 13.6: DC-Zwischenpotential I (Linie)

   Es gilt $\Phi'_{i-1} \leq \Phi_{i-1} + k \cdot \mathrm{OPT}_i$.

   Beweis. Es ist $\Delta\Sigma_{\mathrm{DC}} = 0$ und $\Delta M_{\min} \leq \mathrm{OPT}_i$, da OPT faul ist und somit höchstens einen Server um höchstens OPT_i vom ihm zugewiesenen DC-Server entfernt. □

   Lemma 13.7: DC-Zwischenpotential II (Linie)

   Es gilt $\Phi_i \leq \Phi'_{i-1} - \mathrm{DC}_i$.

   Beweis.

   Fall 1: DC **bewegt genau einen Server.** Dann erhalten wir $\Delta\Sigma_{\mathrm{DC}} = (k-1) \cdot \mathrm{DC}_i$ und $\Delta M_{\min} \leq -\mathrm{DC}_i$. Dann folgt

   $$\Phi_i - \Phi'_{i-1} \leq -k \cdot \mathrm{DC}_i + (k-1) \cdot \mathrm{DC}_i = -\mathrm{DC}_i.$$

   Fall 2: DC **bewegt genau zwei Server.** Dann bewegen sich die beiden Server links und rechts um jeweils $\frac{\mathrm{DC}_i}{2}$. Dann ist $\Delta\Sigma_{\mathrm{DC}} = -\mathrm{DC}_i$, da für jeden nicht bewegten Server einer der bewegten Server sich entfernt und einer sich annähert. Effektiv verringern die beiden sich bewegenden Server ihren Abstand also um DC_i. Außerdem ist $\Delta M_{\min} \leq 0$, da der eine Server auf seinen ihm zugewiesenen Server zugeht, und der andere sich möglicherweise von seinem Server entfernt. Damit ist auch hier

   $$\Phi_i - \Phi'_{i-1} \leq -\mathrm{DC}_i.$$

   □

   Aus Lemma 13.6 und Lemma 13.7 folgt

   $$\Phi_i \leq \Phi'_{i-1} - \mathrm{DC}_i \leq \Phi_{i-1} + k \cdot \mathrm{OPT}_i - \mathrm{DC}_i \iff \Phi_i - \Phi_{i-1} \leq k \cdot \mathrm{OPT}_i - \mathrm{DC}_i.$$

2. gilt für $b = 2k^2$.
3. gilt für $b = 2k^2$.

□

[14.01.2026, Vorlesung 21]
[20.01.2026, Vorlesung 22]

13.2.2 Analyse von DC auf allgemeinen Bäumen

Wir schauen auf die Verwandschaft zwischen endlichen Graphen und endlichen Metriken: Die eine Richtung ist sehr trivial: Aus einer endlichen Metrik (V, d) wähle den vollständigen Grpahen $G = (V, E)$ mit $w : E \to \mathbb{R}_{\geq 0}$ so, dass $w(\{u, v\}) = d(u, v)$.

Für die andere Richtung sei $G = (V, E)$ zusammenhängend mit Kantengewichten $w : E \to \mathbb{R}_{\geq 0}$. Dann setzen wir $d(x, y)$ als die Länge eines kürzesten x-y-Weges in G.

(V, d) heißt *Baummetrik*, wenn sie mithilfe eines Baumes auf V beschrieben werden kann.

Wir sagen, Server s_j heißt *zur Anfgage σ_i benachbart*, wenn auf dem eindeutigen s_j-σ_i-Weg kein anderer Server steht. Stehen mehrere Server auf einer zur Anfrage benachbarten Position, so definiere genau einen beliebigen davon als benachbart.

Auf der Linie wählt der DC-Algorithmus alle zur Anfrage benachbarten Server und bewegt sie mit gleicher Geschwindigkeit auf die Anfrage zu, bis die Anfrage erreicht ist. Wir wollen das sehr ähnlich machen. Dabei lassen wir einen Server stehen, sobald er nicht mehr benachbart zur Anfrage ist (beispielsweise wenn ein anderer Server schneller war). Ebenso können wir Server auch „zwischen“ Knoten stehen lassen.

Satz 13.8: DC auf Bäumen

Der DC-Algorithmus ist k-kompetitiv auf Baummetriken.

Beweis. Wir erinnern uns an unsere Potentialfunktionen. Es sind $M_{\min} = \min_{\pi \in \mathcal{S}_k} \sum_{i=1}^{n} d(s_i, o_{\pi(i)})$ und $\Sigma_{\mathrm{DC}} = \sum_{i=1}^{k-1} \sum_{j=i+1}^{k} d(s_i, s_j)$, sowie $\Phi = k \cdot M_{\min} + \Sigma_{\mathrm{DC}}$.

Wir zeigen wieder die drei Ungleichungen aus Satz 12.10 zeigen:

Es gilt wieder wie oben $\Phi \in \left[0, 2k^2 \max_{x,y \in V} d(x, y)\right]$. Somit bleibt zu zeigen, dass für alle i insgesamt $\Phi_i - \Phi_{i-1} \leq k \cdot \mathrm{OPT}_i - \mathrm{DC}_i$ gelten muss. Wieder seien die Server nach Anfrage σ_{i-1} bei (s_j) bzw. (o_j), und nach σ_i bei (s'_j) bzw. (o'_j). Das Potential Φ_{i-1} sei das Potential bezüglich (s_j) und (o_j), Φ_i bezüglich (s'_j) und (o'_j), und dazwischen befindet sich Φ'_{i-1} bezüglich (s_j) und (o'_j).

Lemma 13.9: DC-Zwischenpotential I (Bäume)

Es gilt $\Phi'_{i-1} \leq \Phi_{i-1} + k \cdot \mathrm{OPT}$.

Beweis. Exakt wie in Lemma 13.6. □

Lemma 13.10: DC-Zwischenpotential II (Bäume)

Es gilt $\Phi_i \leq \Phi'_{i-1} - \mathrm{DC}_i$.

Beweis. Wir teilen die Bearbeitung von σ_i durch DC in Phasen ein. Eine neue Phase beginnt immer, wenn sich die Nachbarschaft ändert.

Wir betrachten eine Phase, in der sich m Server um jeweils d bewegen.

Für $\Delta M_{\min}$: Es steht ein OPT-Server $o_{\pi(j)}$ bereits an der Anfrage (für s_j benachbart zur Anfrage). Wir erhalten $\Delta M_{\min} \leq (m-1)d - d = (m-2)d$, da sich $m-1$ Server wegbewegen und ein Server sich auf den entsprechenden OPT-Server zubewegt.

Es ist $\Delta\Sigma_{DC} \leq -\binom{m}{2}2d + (k-m)\Big(d - (m-1)d\Big) = -m(m-1)d - (k-m)(m-2)d$, da für jedes Paar sich bewegender Server diese sich aufeinander zubewegen (da wir einen Baum haben). Für

jeden nicht zur Anfrage benachbarten Server s_j gilt, dass alle bis auf einer der aktiven Server sich auf diesen Server s_j zu bewegen (da nur ein eindeutiger, zur Anfrage benachbarter Server s_j im Weg steht.)

Wir erhalten

$$\Delta\Phi \leq d\Big(k(m-2) - m(m-1) - (k-m)(m-2)\Big) = d\Big((m-2)m - m(m-1)\Big) = -md,$$

und Summieren über alle Phasen liefert die Gesamtkosten von $-\mathrm{DC}_i$ auf der rechten Seite und auf $\Phi_i - \Phi'_{i-1}$ auf der linken Seite. □

Damit folgt die fehlende Ungleichung für Satz 12.10. □

13.2.3 Anwendung des DC-Algorithmus

Wir betrachten das gewichtete Paging-Problem, wobei jede Seite i noch Kosten w_i erhält, die das Laden kostet. Dazu modellieren wir das normale Paging-Problem wie folgt als Baum:

Betrachte einen Sterngraphen, wo am Anfang alle Server auf dem Mittelpunkt stehen, und alle Kanten Kosten $\frac{1}{2}$ haben. Sind nun alle Server außen und eine nicht-abgedeckte Seite fragt an, so bewegen sich alle Server gleichzeitig in die Mitte (und nur einer geht weiter). Das entspricht etwa dem FWF-Algorithmus. Im gewichteten Fall wird es einfacher – es haben nicht mehr alle Kanten Kosten $\frac{1}{2}$, sondern $\frac{w_i}{2}$, dann flushen wir nicht mehr immer.

Wir betrachten nun eine beliebige Metrik $\mathcal{M} = (V, d)$ mit $G = (V, E)$ und $w : E \to \mathbb{R}_{\geq 0}$ kanonisch als gegeben. Wir erzeugen eine Baummetrik (V, d_T) mithilfe eines minimalen Spannbaums T von G. Es gilt für alle $x, y \in V$ gerade $d(x, y) \leq d_T(x, y) \leq (N-1)d(x, y)$ (für $N = |V|$).

Sei σ eine Eingabe für (V, d). Wende DC auf σ bezüglich (V, d_T) an, das Resultat sei S. Dann erhalten wir als Kostenabschätzung

$$\begin{aligned} w_{d_T}(S) &\leq k \cdot \mathrm{OPT}_{d_T} + \tau \\ w_d(S) \leq w_{d_T}(S) &\leq k \cdot \mathrm{OPT}_{d_T} + \tau \leq k \cdot (N-1) \cdot \mathrm{OPT}_d + \tau. \end{aligned}$$

Damit erhalten wir folgendes Resultat:

Satz 13.11: DC auf allgemeinen Metriken

DC ist $k(N-1)$-kompetitiv auf allgemeinen Metriken.

Fun Fact: Mittels Randomisierung werden wir den kompetitiven Faktor auf $k \cdot \log(N)$ beschränken. Es gibt aber tatsächlich (komplexe) $2k$-kompetitive deterministische Algorithmen.

[20.01.2026, Vorlesung 22]
[21.01.2026, Vorlesung 23]

Kapitel 14

Approximation von Metriken

Definition 14.1: Dominanz und Approximation von Metriken

Sei $\mathcal{M} = (M, d)$ eine Metrik. Wir sagen, eine Metrik $\mathcal{M}' = (M', d')$ mit $M' \supseteq M$ *dominiert die Metrik* $\mathcal{M}$, wenn für alle $x, y \in M$ gilt $d'(x, y) \geq d(x, y)$.
Sei $\mathcal{S}$ eine Menge von Metriken, die jeweils $\mathcal{M}$ dominieren, und sei $\mathcal{D}$ eine Wahrscheinlichkeitsverteilung auf $\mathcal{S}$. Wir sagen, $(\mathcal{S}, \mathcal{D})$ ist eine α-*Approximation*, wenn für alle $x, y \in M$

$$\mathbb{E}_{(M',d')\sim\mathcal{D}}[d'(x, y)] \leq \alpha d(x, y)$$

gilt. Das bedeutet, für jedes feste Paar (x, y) von Punkten aus M darf die erwartete Entfernung in einer zufällig gemäß der Verteilung $\mathcal{D}$ gewählten Metrik höchstens $\alpha d(x, y)$ betragen.
Ist $(\mathcal{S}, \mathcal{D})$ eine α-Approximation einer Metrik $\mathcal{M}$, sagen wir, $\mathcal{M}$ *ist in* $\mathcal{S}$ *eingebettet* und α ist der *Streckungsfaktor* der Einbettung.

14.1 Approximation durch Baummetriken

Für $N = |M|$ ist $\Omega(N)$ bestmöglichste deterministische Apprixomation für Baummetriken M'.

Satz 14.2: Approximation durch Baummetriken

Für jede Metrik $\mathcal{M}$ gibt es eine Menge $\mathcal{S}$ von Baummetriken, *die $\mathcal{M}$ dominieren, und $\mathcal{D}$, sodass $(\mathcal{S}, \mathcal{D})$ eine $\mathcal{O}(\log(N))$-Approximation von $\mathcal{M}$ ist.*
Außerdem gibt es einen effizienten Algorithmus, um gemäß $\mathcal{D}$ aus $\mathcal{S}$ ein Sample zu ziehen.

Der Beweis dauert etwas länger.

14.1.1 Hierarchische Partitionen und Baummetriken

Definition 14.3: Partition

Eine *Partition* von $\mathcal{M} = (M, d)$ mit Radius $r \geq 1$ ist eine *Partition* $V_1, \ldots, V_\ell$ disjunkt von V, sodass für alle $i \in \{1, \ldots, \ell\}$ ein $c_i \in V$ existiert, sodass für alle $v \in V_i$ gilt $d(c, v) \leq r$.

Wir setzen $\Delta := \max_{x,y\in V} d(x, y)$ und $\delta \in \mathbb{N}$ so, dass $2^{\delta-1} < \Delta \leq 2^\delta$ gilt.

Definition 14.4: Hierarchische Partition

Eine *hierarchische Partition* von $\mathcal{M} = (M, d)$ ist eine Folge $D_0, D_1, \ldots, D_\delta$ von $\delta + 1$ Partitionen von V, sodass folgendes gilt:

1. $D_\delta = \{V\}$, das heißt, D_δ ist die triviale Partition mit Radius höchstens 2^δ.
2. Für jedes $i < \delta$ ist D_i eine Partition mit Radius 2^i, die D_{i+1} *verfeinert*, also jede Klasse von D_i ist eine Teilmenge einer Klasse von D_{i+1}.

Nun erzeugen wir aus einer hierarchischen Partition eine Baummetrik: Wir wählen einen Baum, wo jede Ebene die Klassen von D_i (absteigend) darstellt, wobei jede Klasse einen eigenen Knoten erhält. Sie werden entsprechend aufsteigend verbunden, wie sie partitionszugehörig sind. Damit entsprechen Blätter des Baumes genau Knoten aus V. Wir wählen dabei pro Kante im Baum, die Ebenen D_{i-1} und D_i verbindet, das Gewicht 2^i.

Lemma 14.5: hierarchische Partition dominiert Baummetrik

Sei eine beliebige, hierarchische Partition von $\mathcal{M} = (V, d)$ gegeben. Erzeuge daraus wie beschrieben eine Baummetrik (V', d_T). Dann gilt für alle $x, y \in V$, dass $d_T(x, y) \geq d(x, y)$.

Beweis. Der Durchmesser einer Klasse von D_i beträgt höchstens $2 \cdot 2^i = 2^{i+1}$. Daraus folgt: In D_j mit $2^{j+1} < d(x, y)$ liegen x und y in unterschiedlichen Klassen. Das gilt insbesondere für $j = \lceil \log_2 (d(x, y)) \rceil - 2$, denn

$$2^{j+1} = 2^{\lceil \log_2(d(x,y)) \rceil - 1} < 2^{\log_2(d(x,y))+1-1} = d(x, y).$$

Das bedeutet, x und y sind frühestens (von unten aus gesehen) in D_{j+1} in derselben Klasse. Dann ist

$$d_T(x, y) \geq 2 \left(\sum_{i=1}^{j+1} 2^i \right) \geq 2 \cdot 2^{j+1} = 2^{j+2} = 2^{\lceil \log_2(d(x,y)) \rceil} \geq d(x, y). \qquad \square$$

14.1.2 Erzeugung von hierarchischen Partitionen

Wir definieren

$$\mathcal{B}(X, v, r) = \{x \in X \mid d(v, x) \leq r\}$$

als den Ball mit Radius r um v in X.

Algorithmus 14.6 HIERPART

Input: Metrik $\mathcal{M} = (V, d)$
Output: Hierarchische Partition von V

```
1:  Wähle β uniform zufällig aus [1, 2]
2:  Wähle uniform zufällige Partition π von {1, ..., N}
3:  D_δ := {V}
4:  for i = δ − 1 to 0 do
5:      if D_{i+1} enthält eine Klasse mit mehr als einem Element then
6:          β_i := 2^{i−1} β
7:          D_i := Partition(M, D_{i+1}, β_i, π)
8:      else
9:          D_i := D_{i+1}
10: return D_0, D_1, ..., D_δ
```

Da PARTITION in jedem Aufruf in Zeile 7 eine Partition von V mit Radius $\beta_i = 2^{i-1}\beta \leq 2^i$ berechnet, die D_{i+1} verfeinert, ist $(D_0, \ldots, D_\delta)$ eine hierarchische Partition.

Algorithmus 14.7 Partition

Input: Metrik $\mathcal{M}$, Partition D_{i+1}, Faktor β, Permutation π
Output: Partition D_i gemäß hierarchischer Ordnung

```
1: D' := ∅
2: for Klasse K aus D do
3:     for i = 1 to N do
4:         B_π(i) := B(X, v_π(i), β)
5:         X := X \ B_π(i)
6:         if B_π(i) ≠ ∅ then Füge B_π(i) zu D' hinzu
7: return D'
```

[21.01.2026, Vorlesung 23]
[27.01.2026, Vorlesung 24]

[27.01.2026, Vorlesung 24]
[28.01.2026, Vorlesung 25]

Anhang

Liste der Definitionen

1.1 Optimierungsproblem, Approximationsalgorithmus . . . 3
2.1 Vertex Cover . . . 4
2.2 Matching . . . 4
2.5 Set Cover . . . 5
2.9 Scheduling auf identischen Maschinen . . . 6
2.16 Rucksackproblem (Knapsack Problem, KP) . . . 8
3.1 Approximationsschema, PTAS, FPATS . . . 10
3.10 Bin-Packing . . . 13
3.15 Stark NP-schwer . . . 15
4.1 Maximum-Cut Problem . . . 17
4.3 Wahrscheinlichkeitsraum . . . 17
4.5 Unabhängige Ereignisse . . . 18
4.7 Bedingte Wahrscheinlichkeit . . . 18
4.9 Zufallsvariable . . . 19
4.11 Unabhängigkeit (Zufallsvariablen) . . . 19
4.12 Erwartungswert . . . 19
4.16 Randomisierter Approximationsalgorithmus . . . 22
4.24 Max-SAT . . . 26
4.26 Bedingte Erwartung . . . 27
5.1 Lineares Programm (LP) . . . 31
5.10 Integrality Gap . . . 36
5.11 Facility Location (ohne Kapazitäten) . . . 36
5.15 Scheduling auf allgemeinen Maschinen . . . 40
6.1 Positiv semidefinit . . . 44
8.3 Set Cover (Wiederholung) . . . 53
8.6 Feedback Vertex Set . . . 54
8.11 Kürzeste-Wege-Problem . . . 56
8.15 Steinerwaldproblem . . . 58
11.1 Kompetitiver Faktor . . . 62
12.1 Paging . . . 63
12.3 Markierungsalgorithmus . . . 63
12.8 Kompetitiver Faktor für randomisierte Algorithmen . . . 65
12.9 Amortisierte Kosten . . . 65
13.1 k-Server . . . 69
14.1 Dominanz und Approximation von Metriken . . . 74
14.3 Partition . . . 74
14.4 Hierarchische Partition . . . 75

Liste der Aussagen

2.4 Matching-VC 4
2.7 Greedy-SC 5
2.8 Preis in Greedy-SC 6
2.11 LeastLoaded 7
2.14 LPT 7
2.18 Greedy-KP 8
3.4 DynKP 11
3.6 FPTAS-KP 11
3.9 GroßKleinScheduling 12
3.11 Bin-Packing 14
3.13 $B_\varepsilon(T)$ 15
3.14 PTAS Scheduling 15
3.16 Scheduling hat keinen FPTAS 15
4.14 Linearität des Erwartungswerts 20
4.15 Markov 21
4.17 RandMaxCut 22
4.18 Binomialverteilung 23
4.19 Erwartungswert natürliche Zufallsvariable 23
4.20 Geometrische Verteilung 23
4.21 Gedächtnislosigkeit geometrischer Verteilung 24
4.22 Coupon Collector's Problem 24
4.23 RandVertexCover 25
4.25 RandMaxSAT 26
4.27 Zufällige Belegung mit Wahrscheinlichkeit p 29
4.28 Approximationsalgorithmus durch zufällige Belegung mit Wahrscheinlichkeit p . 29
4.29 OPT ist höchstens alle Gewichte minus negative Einheitsklauseln 29
4.30 randomisierter $\frac{\sqrt{5}-1}{2}$-Approximationsalgorithmus 30
5.4 RandomizedRounding 32
5.6 BetterOfTwo 34
5.8 RandRounding$_f$ 35
5.12 Facility Location 37
5.13 Kunde von Hälfte der Nachbarschaft beliefert 38
5.14 Metrisches Facility Location 39
5.17 Zulässige Basislösung von LP$_{T^*}$ 42
5.19 Allokationsgraph ist Pseudowald 42
5.20 H hat einseitig perfektes Matching 43
5.21 2-Approximationsalgorithmus für Scheduling auf allgemeinen Maschinen 43
6.2 Eigenschaften positiv semidefiniter Matrizen 44

6.3 Äquivalenz von Vektor- und semidefiniten Programmen 45
6.4 Fehlerabschätzung optimale Lösung von SDPs 45
6.6 Uniformverteilung auf $\mathbb{S}^{n-1}$ 47
6.7 Wahrscheinlichkeit, dass Knoten getrent werden 47
6.8 Abschätzung Erwartungswert SDPMaxCut 48
6.9 Abschätzung α 48
6.10 SDPMaxCut 48
6.12 Abschätzung Erwartungswert SDP2SAT 50
6.13 SDP2SAT 50
8.1 Schwache Dualität 52
8.2 Starke Dualität 52
8.5 PrimalDual-SC 53
8.9 Graph hat Kreis mit $\leq 2\lceil \log_2(n) \rceil$ Knoten mit Grad ≥ 3 55
8.10 PrimalDual-FVS2 56
8.13 PrimalDual-ShortestPath wählt Baum aus 57
8.14 PrimalDual-ShortestPath 57
8.18 PrimalDual-SteinerForest2: Iterationen 59
8.19 PrimalDual-SteinerForest2 59
12.4 Beispiele: Markierungsalgorithmen auf Paging 64
12.5 Markierungsalgorithmen auf Paging 64
12.6 LFU, LIFO 64
12.7 Deterministische Online-Algorithmen für Paging 64
12.10Potentialfunktion 65
12.11RANDOM ist k-kompetitiv 66
12.12Randomisierte Online-Algorithmen für Paging 67
13.3 Untere deterministische Schranke für k-Server 69
13.5 DC auf der Linie 70
13.6 DC-Zwischenpotential I (Linie) 71
13.7 DC-Zwischenpotential II (Linie) 71
13.8 DC auf Bäumen 72
13.9 DC-Zwischenpotential I (Bäume) 72
13.10DC-Zwischenpotential II (Bäume) 72
13.11DC auf allgemeinen Metriken 73
14.2 Approximation durch Baummetriken 74
14.5 hierarchische Partition dominiert Baummetrik 75

Liste der Algorithmen

2.3 Matching-VC 4
2.6 Greedy-SC 5
2.10 LeastLoaded 6
2.13 LongestProcessingTime (LPT) 7
2.17 Greedy-KP 8
3.3 DynKP 11
3.5 FPTAS-KP 11
3.8 GrossKleinScheduling 12
3.12 $B_\varepsilon(T)$ 14
4.2 RandMaxCut 17
5.3 RandomizedRounding 32
5.5 BetterOfTwo 34
5.7 RandRounding$_f$ 35
6.5 SDPMaxCut 46
6.11 SDP2SAT 50
8.4 PrimalDual-SC 53
8.7 PrimalDual-FVS 55
8.8 PrimalDual-FVS2 55
8.12 PrimalDual-ShortestPath 57
8.16 PrimalDual-SteinerForest1 58
8.17 PrimalDual-SteinerForest2 59
13.2 Greedy-KServ 69
13.4 DC 70
14.6 HierPart 75
14.7 Partition 76

Index

Allokationsgraph, 42
Amortisierte Kosten, 65
Approximationsalgorithmus, 3
    derandomisierter -, 27
    randomisierter r-, 22
Approximationsfaktor, 22
Approximationsschema, 10
    polynomielles -, 10
    voll-polynomielles -, 10

Baummetrik, 72
bedingte Wahrscheinlichkeit, 18
bedingter Erwartungswert, 27
BetterOfTwo, 34
Bin-Packing, 13
Binomialverteilung, 23

Coupon Collector's Problem, 24

derandomisiert, 27
Disjunktion, 26
Duales LP, 51
DynKP, 11

Ecke, 41
Elementarereignis, 17
Ereignis, 17
    Gegen-, 17
    unabhängige -se, 18
Ergebnismenge Ω, 17
Erwartungswert, 19
    bedinger -, 27

Facility Location, 36
Feedback Vertex Set, 54
Fläche, 41
    Seiten-, 41
FPATS, 10
FPTAS-KP, 11

Gedächtnislosigkeit, 24
geometrische Verteilung, 23
    Gedächtnislosigkeit, 24
Greedy-KP, 8
Greedy-SC, 5

Hierarchische Partition, 75
HierPart, 75

Integrality Gap, 36

k-Server-Problem, 69
Kante, 41
Klausel, 26
    negative Einheits-, 28
Knapsack-Problem, 8, 10
Kompetitiver Faktor, 62, 65
    randomisierter -, 65
    strikt -, 62, 65
Konjunktion, 26
konjunktive Normalform, 26
konvexes Polyeder, 41
KP, 8, 10
Kürzeste-Wege-Problem, 56

LeastLoaded, 6
Lineares Programm (LP), 31
    Allokationsgraph, 42
    Duales System, 51
    ganzzahliges - (ILP), 31
        Integrality Gap eines -es, 36
    Lösungspolyeder, 41
    relaxiertes -, 32
Literal, 26
    negatives -, 26
    positives -, 26
LongestProcessingTime, 7
LPT, 7
Lösungspolyeder, 41

Markierungsalgorithmus, 63
Matching, 4
    inklusionsmaximales -, 4
Matching-VC, 4
Matrix
    positiv semidefinite -, 44
    symmetrische -, 44
Max-2SAT, 48
Max-SAT, 26
    Max-2SAT, 48
MaxCut, 17
Metrik
    α-Approximation, 74
    Dominanz, 74
    Einbettung, 74
    Streckungsfaktor, 74

negative Einheitsklausel, 28

Online-Algorithmus, 62

Paging, 63
  FIFO, 63
  FWF, 63, 73
  LFD, 63
  LFU, 63
  LIFO, 63
  LRU, 63
  MARK, 65
  RANDOM, 63, 65, 66
Partition, 74
  - (Algorithmus), 76
  hierarchische -, 75
Phaseneinteilung, 63
Polyeder
  Ecke, 41
  Fläche, 41
    Seiten-, 41
  Kante, 41
  konvexes -, 41
  Lösungs-, 41
  Seitenfläche, 41
  stützende Hyberebene, 41
primal-dualer Algorithmus, 54
PrimalDual-FVS, 55
PrimalDual-FVS2, 55
PrimalDual-SC, 53
PrimalDual-ShortestPath, 57
PrimalDual-SteinerForest1, 58
PrimalDual-SteinerForest2, 59
Pseudobaum, 42
Pseudowald, 42
PTAS, 10

RandRounding$_f$, 35
RandMaxCut, 17
RandMaxSAT, 26
RandomizedRounding, 32
RandVertexCover, 25
relaxieren, 32
Rucksackproblem, 8, 10

Scheduling, 6, 12, 40
  auf allgemeinen Maschinen, 40
  auf identischen Maschinen, 6, 12
SDP2SAT, 49
SDPMaxCut, 46
Seitenfläche, 41
Semidefinites Programm (SDP), 44
Set Cover, 5, 53
σ-additiv, 17
stark NP-schwer, 15
Steinerwaldproblem, 58
stützende Hyperebene, 41

Unabhängigkeit, 19
  von Zufallsvariablen, 19

Vektorprogramm (VP), 44
Vertex Cover, 4

Wahrscheinlichkeit, 17
  bedingte -, 18
Wahrscheinlichkeitsmaß, 17
Wahrscheinlichkeitsraum, 17
  Elementarereignis, 17
  Ereignis, 17
    unabhängige -se, 18
  Ergebnismenge Ω, 17
  Produktraum, 18
  σ-additiv, 17
  Wahrscheinlichkeit, 17
  Wahrscheinlichkeitsmaß, 17

Zufallsvariable, 19
  binomialverteilte -n, 23
  Erwartungswert, 19
    bedingter -, 27
  geometrisch verteilte -n, 23
    Gedächtnislosigkeit, 24
  unabhängige -n, 19